

VPJ-Abschlussbericht

Gewerk3

Antriebs- und Regelungstechnik

vorgelegt am 04.01.2026
Hannah Deppe und Alexander Prehn

Betreuer*in
Jochen Maaß

**HOCHSCHULE FÜR ANGEWANDTE
WISSENSCHAFTEN HAMBURG**
Department Informations- und Elektrotechnik
Berliner Tor 7
20099 Hamburg

Inhaltsverzeichnis

Abkürzungsverzeichnis	V
Abbildungsverzeichnis	VI
Tabellenverzeichnis	VIII
1 Einleitung.....	1
1.1 Problemumfeld.....	1
1.2 Aufgaben und Ziele.....	2
2 Konzeptionierung	3
2.1 Programmstruktur	3
2.2 Schnittstellenbeschreibung.....	4
2.3 Kommunikation	5
2.3.1 Extern.....	5
2.3.2 Intern.....	6
3 Controller.....	8
3.1 Konzepterstellung	8
3.1.1 Ziel und Aufgabe der Controller-Task	8
3.1.2 Grobkonzept und Simulation.....	8
3.2 Eigene Variablen über Data Unit Types	11
3.3 Programmierung am HAWino	12
3.3.1 Aufbau des Programms	12
3.3.2 Statemachine des Controllers	16
3.4 Testbetrieb.....	30
4 Trajektoriengenerierung	33
4.1 Konzeption und Simulation	33
4.2 Implementierung der Trajektoriengenerierung	35
4.2.1 Beschreibung des Trajectory-Programms.....	36
4.2.2 Berechnung der Trajektorie für die Drehung in θ	40
4.2.3 Berechnung der Trajektorie für Fahrten in x- und y-Richtung.....	44
5 Bahnregelung.....	49

5.1	Konzeption und Simulation	49
5.2	Positionsregelung.....	50
5.2.1	Berechnung der Regeldifferenz	50
5.2.2	Regelalgorithmus mit Gain-Scheduling	51
5.3	Geschwindigkeitsregelung.....	52
5.3.1	Vorsteuerung	52
5.3.2	Koordinatentransformationen	52
5.3.3	Regelalgorithmus.....	53
5.4	Abschließende Betrachtung der Bahnregelung.....	55
6	Luenberger Beobachter.....	57
6.1	Konzeptionierung.....	57
6.2	Totzeitermittlung.....	58
6.2.1	Kameraanalyse.....	59
6.2.2	Ermittlung der Totzeit	60
6.3	Umsetzung in der Programmierung	62
6.4	Experimentelle Ermittlung des Proportionalitätsfaktors.....	65
6.4.1	Verschiebung der Beobachterposition um die Totzeit.....	65
6.4.2	Experimentelle Bestimmung über den Weg	67
6.4.3	Schätzung und Anpassung über Szenario im Controller	68
6.5	Optimierungsversuche	69
6.6	Fehlerbetrachtung und abschließende Gedanken.....	71
7	Fazit	74
	Literaturverzeichnis.....	75
	Anhang	76
A	Tabellen der Fehlerauswertung	76
B	Ladestation	77
B.-1	Problembeschreibung.....	77
B.-2	Analyse des Systems	78
B.-3	Erkenntnisse	79
B.-4	Abschließende Anmerkungen	80

C	Encoder.....	82
C.-1	Der Funktionsbaustein FB_GetEncoderSpeed.....	82
C.-2	Beobachtungen, Fehlersuche und Ergebnis	82
D	Programmierung des Regelkreises über PID.....	84

Abkürzungsverzeichnis

ARP	<i>Address Resolution Protocol</i>
DKT	<i>Direkte Kinematik Transformation</i>
HAW	<i>Hochschule für Angewandte Wissenschaften</i>
HMI	<i>Human Machine Interface (Mensch-Maschine-Schnittstelle)</i>
IKT	<i>Inverse Kinematik Transformation</i>
MQTT.....	<i>Message Queuing Telemetry Transport</i>
RFID.....	<i>Radio Frequency Identification</i>
ST	<i>Strukturierter Text</i>
TLS.....	<i>Transport Layer Security</i>
VPJ	<i>Verbundprojekt</i>

Abbildungsverzeichnis

Abbildung 1 Aufbau der Programmstruktur.....	3
Abbildung 2 Grobkonzept für das Einlesen und Verarbeiten von Koordinaten.....	9
Abbildung 3 Umschaltung in Simulink.....	10
Abbildung 4 Umschaltung an der Strecke.....	11
Abbildung 5 Umschaltung in der Geschwindigkeit	11
Abbildung 6 Überblick über das Programm RobotController.....	13
Abbildung 7 Prüfung auf Teilverlust.....	14
Abbildung 8 Prüfung auf Bumper	15
Abbildung 9 Prüfung, ob es einen Abbruch gab	16
Abbildung 10 Ablaufdiagramm des States Init	17
Abbildung 11 Ablaufdiagramm des States Standby.....	18
Abbildung 12 Übersetzung der Wegpunkte aus Gewerk 2	19
Abbildung 13 Ablaufdiagramm des States InputWaypointArray	20
Abbildung 14 Abbildung 13 Ablaufdiagramm des States LogicCheck	21
Abbildung 15 Phänomen der erreichten Fahrt.....	22
Abbildung 16 Problematik der Winkelstellung.....	23
Abbildung 17 Normalisierungslogik über ATAN2.....	24
Abbildung 18 Ablaufdiagramm des States SendingReglervalue.....	25
Abbildung 19 Ablaufdiagramm des States TargetReachedControll	26
Abbildung 20 Ablaufdiagramm des States ControllGripper	28
Abbildung 21 Verlagerung der ursprünglichen Programmierung in die Statemachine	29
Abbildung 22 ScanRequest und Scanning vereinfacht.....	29
Abbildung 23 RobotController Testbetrieb.....	31
Abbildung 24 Beispiele für die testValues.....	32
Abbildung 25 Konzept für die Bewegungspfade der HAWinos	33
Abbildung 26 Umsetzung der Trajektoriengenerierung in MATLAB Simulink	35
Abbildung 27 Ablaufdiagramm des Programms Trajectory	38
Abbildung 28 Auswahl des Reglermodus im Programm Trajectory	39
Abbildung 29 Vereinfachtes Ablaufdiagramm des FB_TrajectoryOneAxis.....	41
Abbildung 30 Vereinfachtes Ablaufdiagramm des FB_CalculateProfileOneAxis	42
Abbildung 31 Code zur Auswahl der Beschleunigung	43
Abbildung 32 Berechnete Trajektorien für eine Drehung in θ	44
Abbildung 33 Vereinfachtes Ablaufdiagramm des States NoTraj im FB_TrajectoryTwoAxis.....	44
Abbildung 34 Vereinfachtes Ablaufdiagramm des States Active im FB_TrajectoryTwoAxis.....	45
Abbildung 35 Vereinfachtes Ablaufdiagramm des States BlendingX	47

Abbildung 36 Ergänzung im FB_CalculateProfileTwoAxis zum FB_CalculateProfileOneAxis	47
Abbildung 37 Berechnete Weg- und Geschwindigkeitsprofile für einen Fahrauftrag eines HAWinos	48
Abbildung 38 Konzept für den Regelkreis	49
Abbildung 39 Code des FB_PositionError	50
Abbildung 40 Gain-Scheduling im Positionsregler	51
Abbildung 41 Fahrtverlauf in x- und y-Richtung	55
Abbildung 42 Verlauf einer Drehung in θ	56
Abbildung 43 Luenberger in der Simulation	57
Abbildung 44 Eine Fahrt aus der Simulation	58
Abbildung 45 Kameraanalyse mit Roboter 3	59
Abbildung 46 Kameraanalyse mit Roboter 4	60
Abbildung 47 Sinus-Funktion für das Totzeitexperiment	60
Abbildung 48 Messung für die Totzeitermittlung	61
Abbildung 49 Vergleich der Totzeitentwicklungen bei Programmaufruf in verschiedenen Tasks	62
Abbildung 50 Berechnung der Korrektur	63
Abbildung 51 Berechnung des Luenberger Beobachters	64
Abbildung 52 Verschiebung des Puffers um einen Wert	64
Abbildung 53 Positionsverlauf in Beckhoff für die x-Achse	65
Abbildung 54 Positionsverlauf in Beckhoff für Theta	66
Abbildung 55 Das neue Experiment	67
Abbildung 56 Einstellung über ein Szenario	68
Abbildung 57 Verschiebung in θ	71
Abbildung 58 Verschiebung in x	71
Abbildung 59 Rotation um die eigene Achse	72
Abbildung 60 Fehlermeldung der Ladestation	77
Abbildung 61 Vergleich der Uhrzeiten auf dem HMI und den Laborrechnern	78
Abbildung 62 „Kommunikationsversuche der Ladestation bei einem Fehlerfall“	79
Abbildung 63 Ladestation: Temperaturfehler 1	81
Abbildung 64 Ladestation: Temperaturfehler 2	81
Abbildung 65 Darstellung des <i>FB_PID_for_Array</i>	84

Tabellenverzeichnis

Tabelle 1 Beispiel der Auftragsübergabe	4
Tabelle 2 Definition der Aufgabe.....	4
Tabelle 3 Bedeutung der Statusmeldung.....	5
Tabelle 4 Bedeutung der GlobalVar zwischen den Gewerken.....	5
Tabelle 5 Fehlermeldungen für das Debuggen in den Gewerken 1 und 2.....	6
Tabelle 6 Bedeutung der GlobalVar.stateSinc	6
Tabelle 7 Bewegungsgleichungen für eine gleichmäßig beschleunigte Bewegung.....	34
Tabelle 8 Formeln für die Trajektoriengenerierung nach Profil.....	34
Tabelle 9 Austausch zwischen Trajectory, RobotController und Main_MotorTask.....	36
Tabelle 10 Rotationsmatrizen für die Umrechnung der Koordinatensysteme [2].....	53
Tabelle 11 Formeln der IKT und DKT [3].....	53
Tabelle 12 Messergebnisse der Versuche für die Ermittlung der Kameratzeit	62
Tabelle 13 Messwerte für die X-Achse	76
Tabelle 14 Messwerte für die Y-Achse	76
Tabelle 15 Messwerte für Ausrichtung Theta	76

1 Einleitung

Das Verbundprojekt der Hochschule für Angewandte Wissenschaften (HAW) Hamburg im Master Automatisierung befasst sich mit der Umsetzung eines Fertigungsprozesses anhand einer modellhaften Anlage. Diese Anlage besteht aus vier Fertigungsstationen, einer Ladestation mit zwei Ladepunkten und vier autonom fahrenden Robotern, den HAWinos, die entsprechend dem Prozess die Stationen bestücken.

Der vorliegende Bericht beschreibt die Arbeit des Gewerks 3. Dieses Gewerk beschäftigt sich mit der Aufgabe, ein Konzept für die Bahnregelung der HAWinos zu entwickeln, umzusetzen und anschließend zu verifizieren.

1.1 Problemumfeld

Das Verbundprojekt (VPJ) gliedert sich in vier Gewerke, die innerhalb des Gesamtprojekts jeweils unterschiedliche Aufgabenbereiche übernehmen. Ziel des Projekts ist die Entwicklung eines vollständigen Fertigungsprozesses, bei dem aus Rohteilen durch mehrere, verschieden kombinierte Fertigungsschritte unterschiedliche Produkte hergestellt und im Endproduktlager abgelegt werden. Für den Transport zwischen den Stationen werden vier autonome HAWinos eingesetzt.

Die Positionen der HAWinos werden über ein an der Decke des Raums angebrachtes Kamerasystem ermittelt. Um eine Überwachung des Prozesses zu ermöglichen, besitzt jedes Werkstück einen Radio Frequency Identification (RFID)-Tag, das an den einzelnen Stationen über RFID-Reader gelesen oder beschrieben wird, um die Werkstücke an den Stationen ein- oder auszuchecken. Dadurch ist es jederzeit möglich, den aktuellen Produktionsschritt eines Werkstücks zu erfassen und den Produktionsprozess zu überwachen.

Gewerk 1 übernimmt die zentrale Fertigungsplanung sowie die Steuerung und Überwachung des Gesamtprozesses. Dazu zählen die Generierung und Verwaltung von Transport- und Ladeaufträgen, die Verarbeitung der RFID-Daten, die Speicherung aller Fertigungsdaten in einer Datenbank sowie die Entwicklung einer HMI (Human Machine Interface) zur Bedienung und Überwachung der Anlage. Die Kommunikation erfolgt über eine Message Queuing Telemetry Transport (MQTT)-Schnittstelle.

Die Bahnplanung für die von Gewerk 1 erzeugten Aufträge wird von Gewerk 2 durchgeführt. Die Hauptaufgabe von Gewerk 2 besteht darin, einen geeigneten Algorithmus auszuwählen, der auf Basis der empfangenen Auftragsdaten eine Pfadberechnung durchführt, um kollisionsfreie Bahnen für alle HAWinos zu bestimmen. Diese Bahnen werden anschließend an die Regelungstechnik übergeben.

Für den Aufgabenbereich der Regelungstechnik ist Gewerk 3 zuständig. Der Schwerpunkt liegt auf der exakten Positionsbestimmung, der Trajektoriengenerierung sowie der Entwicklung und Parametrierung der Antriebsregelung. Hierzu wird ein Beobachter implementiert, der die genaue Position der HAWinos

im Raum ermittelt. Auf Basis der von Gewerk 2 geplanten Pfade werden Trajektorien generiert, die von den HAWinos abgefahren werden. Die Umsetzung dieser Aufgaben ist Gegenstand dieses Berichts.

Gewerk 4 befasst sich mit dem Energiemanagement der HAWinos. Dazu zählen die Erneuerung des Algorithmus für die Berechnung des Ladezustands (State of Charge (SoC)) sowie die Umsetzung einer Akkuschutzschaltung mit akustischem Warnsignal bei niedrigem Akkustand.

1.2 Aufgaben und Ziele

Um die Aufgaben und Ziele dieser Arbeit zu erläutern, wird nachfolgend die Struktur der Arbeit betrachtet.

Zu Beginn des Projekts erfolgt die Konzeption. Dabei wird ein Konzept für die Regelungstechnik entwickelt, auf Basis dessen die Umsetzung erfolgt. In dieser Phase werden mehrere einzelne Simulationen, ebenso wie eine gesamte Simulation des Regelkreises aufgebaut, um die erstellten Konzepte zu verifizieren. Diese Simulationen werden im Laufe des Berichts an verschiedenen Stellen referenziert, da sie die Basis für die Entwicklung des Projekts bilden. Des Weiteren wird eine Programmstruktur konzipiert, sowie die Schnittstellen zu anderen Gewerken und zwischen den einzelnen Programmen der Regelungstechnik definiert.

Darauffolgend wird das Programm *RobotController* betrachtet, welches zum Ziel hat die Kommunikation zu Gewerk 2 herzustellen und die korrekte Abfolge der Wegpunkte an die Trajektoriengenerierung zu übermitteln. Zudem bildet dieses Programm die Struktur der Fahrtabläufe, steuert die Greifer und reagiert auf Notfall-Stops und andere Fehlerfälle.

Der nächste Abschnitt des Berichts befasst sich mit der Trajektoriengenerierung, die zum Ziel hat, aus den Wegpunkten von Gewerk 2, die Sollwerte für die Regelalgorithmen zu berechnen.

Die verwendeten Regelalgorithmen, die das Ziel haben, eine möglichst genaue Positionierung und Bahnverfolgung zu ermöglichen, werden in dem darauffolgenden Kapitel erläutert.

Für die genaue Positionsbestimmung mithilfe des Kamerasystems wird ein Beobachter benötigt. Dieser hat zum Ziel Störungen des Kamerasystems zu kompensieren und dadurch jederzeit die exakten Positionen der HAWinos zu liefern.

Zuletzt wird ein abschließendes Fazit zur Arbeit und zum VPJ gezogen.

Im Anhang B dieser Arbeit ist ein zusätzliches Kapitel beigelegt, das das Startproblem der Ladestation analysiert und mögliche Lösungsansätze betrachtet.

2 Konzeptionierung

In diesem Kapitel wird ein allgemeiner Überblick über das Zusammenspiel der einzelnen Tasks sowie deren zeitliche Abfolge gegeben. Darüber hinaus wird auf die Schnittstellen zu den anderen Gewerken, deren Bedeutung sowie die zugrunde liegende Kommunikation eingegangen.

2.1 Programmstruktur

Die Realisierung der Regelungsaufgabe erfolgt durch das koordinierte Zusammenwirken von drei separaten Tasks, dem Controller-Task, dem Trajectory-Task und dem MainMotor-Task. Der strukturelle Aufbau sowie der abstrakte Ablauf des Systems sind in Abbildung 1 schematisch dargestellt.

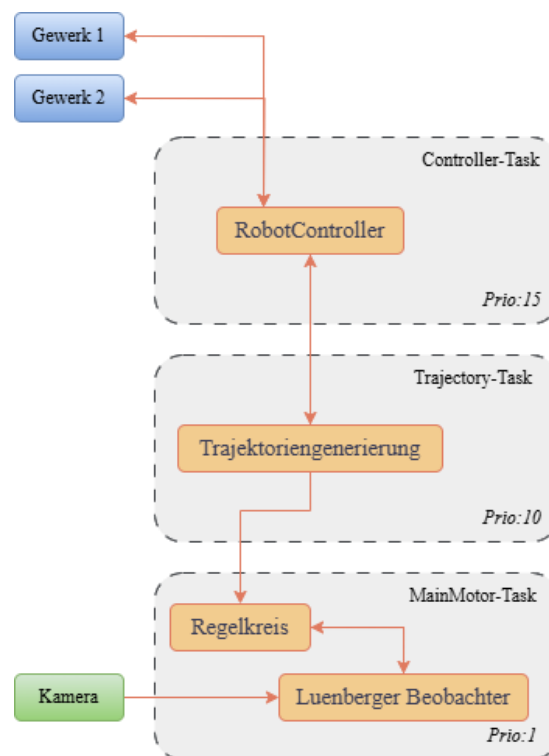


Abbildung 1 Aufbau der Programmstruktur

Die Gewerke 1 und 2 kommunizieren mit der Ablaufsteuerung, dem *Controller-Task*, welcher den *RobotController* beinhaltet. Dieser ist neben der Kommunikation zwischen den Gewerken auch für die Koordination innerhalb des eigenen Gewerkes zuständig. Der *RobotController* übergibt Koordinaten an die Trajektoriengenerierung in dem *Trajectory-Task*. Aufgabe dieser ist die Erzeugung eines Weg-, Geschwindigkeits- und Beschleunigungsprofils. Die Sollwerte für die Fahrt werden an den Regelkreis in dem *MainMotor-Task* weitergegeben. Dieser übernimmt die Positions- und Geschwindigkeitsregelung in Zusammenarbeit mit dem Luenberger Beobachter. Eingebettet ist hier auch die Kamera, sie liefert die verzögerte Ist-Position des jeweiligen HAWinos.

2.2 Schnittstellenbeschreibung

Für den Austausch von Informationen sowie die Abarbeitung von Aufträgen ist es erforderlich, im Vorfeld eine konsistente und einheitliche Schnittstellendefinition festzulegen. Im Rahmen der Konzeptionierung ist eine solche in Zusammenarbeit mit Gewerk 1 und Gewerk 2 erarbeitet worden.

Die erste Definition bezieht sich auf die Übergabe der Auftragsdaten. Diese müssen strukturiert und gleichbleibend übergeben werden. Die Struktur ist beispielhaft dargestellt in der folgenden Tabelle 1.

Tabelle 1 Beispiel der Auftragsübergabe

Koordinate in der X-Achse
Koordinate in der Y-Achse
Koordinate in der Ausrichtung θ
Aufgabentyp oder Aktion

Diese Informationen sind notwendig, um eine Strecke von einer Station zur nächsten zu bewältigen. Für den Aufgabentyp oder die Aktion folgt die Definition in der folgenden Tabelle 2.

Tabelle 2 Definition der Aufgabe

Beschreibung	Nr.	threshold [mm]
Globalfahrt	1	340
Ablegefahrt	2	6
Abholfahrt	3	2
Scannfahrt	4	2
Warten	5	50
Ladefahrt	6	2
Lokalfahrt	7	5
Pendelfahrt	8	210

Über den Aufgabentyp wird das Verhalten des HAWinos beim Erreichen eines Wegpunkts gesteuert. Die Aufgabentypen definieren die im Controller-Task auszuführenden Aktionen und beeinflussen indirekt die Überschleifkurven sowie die Positioniergenauigkeit. Die Überprüfung der Erreichung einer Koordinate erfolgt mithilfe eines Schwellwerts, der als *threshold* bezeichnet wird. Jeder Aufgabe ist ein eigener Schwellwert zugewiesen.

Darüber hinaus sind die Statusmeldungen des HAWinos vordefiniert. Dieser soll in der Lage sein, die in der folgenden Tabelle 3 aufgezählten Zustände mitzuschreiben.

Tabelle 3 Bedeutung der Statusmeldung

Kürzel	Bedeutung	Nr
Working	Der Auftrag wird abgearbeitet	1
Target_Reached	Das Ziel ist erreicht	2
Waiting	Es wird gewartet	3
Error	Es existiert ein Fehler	4

Sie dienen der Fehlersuche während der Programmierung und sind als interne Zustände für die Anzeige in Gewerk 3 gedacht.

2.3 Kommunikation

Der Fokus liegt auf der Kommunikation mit den beteiligten Gewerken sowie auf dem Informationsaustausch innerhalb der Regelung. Dabei wird zwischen interner und externer Kommunikation unterschieden. Das zugrunde liegende Kommunikations- und Scheduling-Konzept dient der Synchronisation der Prozesse und stellt durch prioritätsbasiertes Scheduling sicher, dass konkurrierende Programmabläufe kontrolliert verdrängt werden und Fehlverhalten vermieden wird.

2.3.1 Extern

Die externe Kommunikation, also die direkte Rückmeldung zu Gewerk 1 und 2 findet über die *GlobalVar* statt. Die Bedeutung dieser ist in der folgenden Tabelle 4 abgebildet. Hier dargestellt ist eine Auswahl an Variablen und deren Bedeutung.

Tabelle 4 Bedeutung der GlobalVar zwischen den Gewerken

GlobalVar	Bedeutung
ResetBumper	Halte- und Rücksetzbedingung für den Nothalt.
jobAbort	Abbruch des aktuellen Fahrauftrages.
stepCounter	Wegpunktzähler des RobotController.
numberWayPoints	Anzahl der gültigen Wegpunkte aus der Pfandplanung.
turningPointsX	Wegpunkte in der X-Achse.
turningPointsY	Wegpunkte in der Y-Achse
turningPointsTheta	Wegpunkt für die Orientierung in Theta.
taskType	Art der Fahrt für den aktuellen Wegpunkt.

Über die Schnittstellen lässt sich der Ablauf der Controller-Task von außen steuern, wodurch direkt die Verzweigungen in der Statemachine beeinflusst werden können. Zur Überwachung der Programmierung und zur Nachvollziehbarkeit von Fehlerfällen existieren die Fehlermeldungen (*error messages*). Diese werden vom RobotController bei Störungen an die beteiligten Gewerke übermittelt und sind im MQTT-Client einsehbar. Die jeweilige Bedeutung der Fehlermeldungen ist in Tabelle 5 zusammengefasst.

Tabelle 5 Fehlermeldungen für das Debuggen in den Gewerken 1 und 2

GlobalVar.errorMessage	Bedeutung
GrippingFailed	Der HAWino konnte das Werkstück nicht erfolgreich aus der Tasche abholen.
PartLost	Während der Fahrt ging das Werkstück verloren.
BumperTriggered	Der Bumper des HAWinos ist betätigt worden.
TargetReachedFailed	Die Zeitüberwachung der Zielverfolgung hat ausgelöst. Der Auftrag wird nicht abgearbeitet.
TooManyWaypoints	Der RobotController verfügt über mehr Wegpunkte als die Planung bereitgestellt hat.
EmptyPocket	Die angefahrene Tasche ist leer.

2.3.2 Intern

Die Synchronisation von Trajektoriengenerierung, Regler und Ablaufsteuerung erfolgt über den Austausch von Variablen, die als Semaphoren dienen. Die Bedeutung dieser Variablen findet sich in der folgenden Tabelle 6.

Tabelle 6 Bedeutung der GlobalVar.stateSinc

GlobalVar.stateSinc	Bedeutung
0	Initialwert nach Inbetriebnahme des HAWinos. Der Reset ist noch nicht erfolgt.
1	Ein Reset der Trajektoriengenerierung muss durchgeführt werden.
2	Der Reset ist durchgeführt und die Trajektoriengenerierung ist bereit für Koordinaten.
3	Der RobotController ist im Standby, die Trajektoriengenerierung kann aufhören zu arbeiten.
4	Normalbetrieb, die Trajektorien werden generiert.

5	Fehler bei der Übermittlung. Erneutes Übergeben der Wegpunkte an Trajektorie notwendig.
---	---

Mit dieser eigenen Konvention ist es möglich ein Austausch von Informationen zwischen dem Prozessablauf und der Trajektorie herzustellen. Gleichzeitig dient sie als Semaphore, um die Tasks zu synchronisieren und unnötige Rechenlast während eines Stillstands zu vermeiden.

3 Controller

In diesem Kapitel wird auf die angewandte Kommunikation zwischen den Gewerken, dem Datenaustausch und der Koordination der Fahrten eingegangen. Auch wird Bezug auf die Konzeptionierung und Ausarbeitung der *Controller-Task* genommen.

3.1 Konzepterstellung

In der Phase der Konzepterstellung werden unterschiedliche Ansätze zur Umsetzung der Fahraufträge untersucht und validiert. Zudem wird über eine vereinfachte Simulation überprüft, ob der gewählte Ansatz zur erfolgreichen Bewältigung der Fahraufgabe geeignet ist.

3.1.1 Ziel und Aufgabe der Controller-Task

Das Hauptziel der Controller-Task besteht im Einlesen der von der Pfadplanung des Gewerks 2 bereitgestellten Wegpunkte. Diese werden gewerksintern weitergegeben und zu zusammenhängenden Strecken für die Trajektoriengenerierung kombiniert. Dabei muss zu jedem Zeitpunkt erkannt werden, ob ein Zielpunkt erreicht oder ein Fahrauftrag abgeschlossen ist.

Eine wesentliche und für die erfolgreiche Umsetzung des Verbundprojekts notwendige Aufgabe stellt die Kommunikation mit den beteiligten Gewerken dar. Diese erfolgt zentral über den *RobotController* und wird von dort aus an die entsprechenden Teilbereiche weitergeleitet. Darüber hinaus ist die mit dem jeweiligen Fahrauftrag verbundene Aufgabe auszuführen. Hierbei kommen sämtliche mechatronischen Komponenten des HAWinos zum Einsatz, darunter der Bumper, die Sensorik sowie der Greifer.

3.1.2 Grobkonzept und Simulation

Das Grobkonzept des Controllers muss in der Lage sein die Basisaufgaben zu erfüllen, es muss damit eine Strecke weitergeben können und erkennen, ob der Zielpunkt erreicht ist. Ist dieser erreicht muss ein neuer Zielpunkt an die Trajektorie übergeben werden. Dieser Vorgang wird fortlaufend als Umschalten bezeichnet. Der Controller darf nur so lange arbeiten, wie es Wegpunkte gibt.

Begonnen wird die Ausarbeitung der Simulation mit dem Einlesen einer Koordinatenliste, dem extrahieren der gewünschten Wegpunkte und der Übergabe an die Trajektoriengenerierung. Dieser Teil der Simulation ist in der folgenden Abbildung 2 dargestellt und zeigt einen Ausschnitt aus der anfänglichen Simulation für das Grobkonzept.

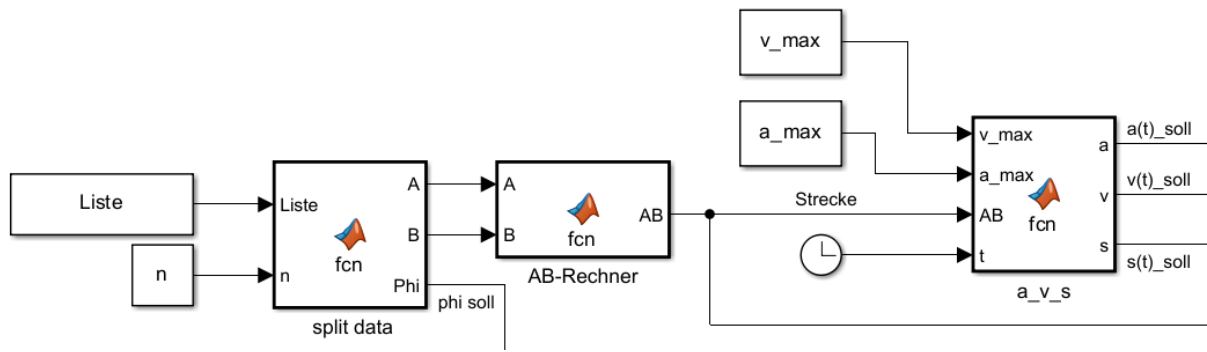


Abbildung 2 Grobkonzept für das Einlesen und Verarbeiten von Koordinaten

Die Simulation nutzt eine Matrix der Größe $2 \times m$ mit dem Namen *Liste*. In dieser befinden sich die testweise gewählten Wegpunkte. Diese Koordinaten werden, zusammen mit einem Zähler n , in den Block *split data* überführt. Die Matrix wird dort an dem gewählten Wegpunkt n in zwei Spaltenvektoren, dem Startpunkt $\vec{A}$ und dem Zielpunkt $\vec{B}$, aufgeteilt. Dabei sei $\vec{A}$ der Spaltenvektor an der Stelle n und $\vec{B}$ der Spaltenvektor an der Stelle $n + 1$.

Aus diesen wird mit der folgenden Formel (1) in dem Block *AB-Rechner* die Strecke $\overline{AB}$ berechnet.

$$\overline{AB} = \vec{B} - \vec{A} \quad (1)$$

Diese Strecke kann an die Trajektoriengenerierung übergeben werden und für die Planung eines Fahrprofils verwendet werden.

Auf diese erste Simulation wird weiter aufgebaut. Hierzu wird die Umschaltung der Wegpunkte bei dem Erreichen eines Zielpunkts ergänzt. Dargestellt ist dieser Teil der Simulation in der folgenden Abbildung 3.

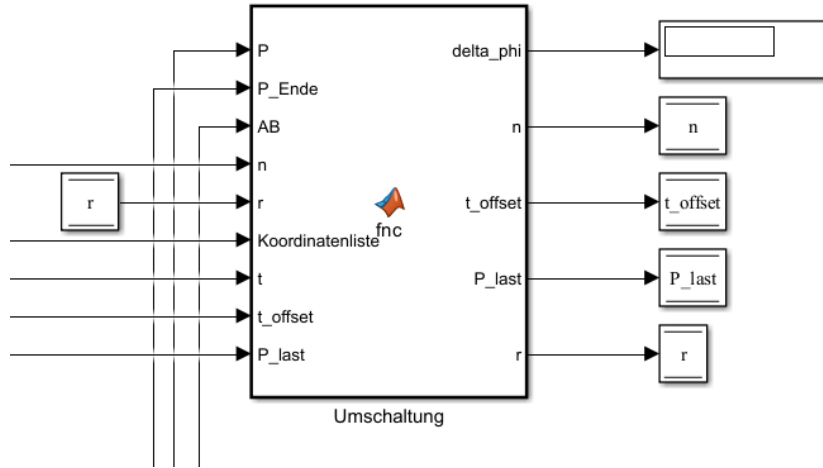


Abbildung 3 Umschaltung in Simulink

Der dargestellte Block *Umschaltung* verfolgt eine schlichte Logik. Nähert sich der HAWino dem Zielpunkt $\vec{B}$ verringert sich der Abstand von der Ist-Position zu diesem. Berechnet wird der Abstand durch Verwendung der folgenden Formel (2).

$$\overline{PB} = \sqrt{(x_B - x_P)^2 + (y_B - y_P)^2} \quad (2)$$

Dabei wird für die Berechnung in Simulink die Ist-Position $\vec{P}$ und der Endpunkt $\vec{B}$ verwendet. Die resultierende Reststrecke $\overline{PB}$ kann nun mit einem *threshold* verglichen werden. Der threshold ist hier als Variable *r* mit dem Wert 0.3 m definiert worden.

Mit diesen Informationen lässt sich der Zähler *n* inkrementieren, sobald die in der folgenden Formel (3) dargestellte Bedingung erfüllt ist.

$$n_{k+1} = \begin{cases} n_k + 1 & , falls r > threshold \\ n_k & , sonst \end{cases} \quad (3)$$

Da die Länge der Zeilen in der Wegpunktliste im Voraus gezählt werden kann, ist es auch möglich zu erkennen, wann eine Fahrt beendet ist. Dies geschieht durch das Prüfen der Bedingung in der folgenden Formel (4).

$$Fahrtende := n_{k+1} > len \quad (4)$$

Hierbei seien *len* die Länge der Wegpunktliste und die Variable *Fahrtende* eine boolesche Variable. Diese kann, wenn sie wahr ist, das *Fahrtende* auslösen.

Die Simulation kann das Konzept und die verfolgten Ansätze in ihrer Funktionalität bestätigen. Es ist möglich die Koordinaten in dem Format der Schnittstellendefinition an die Trajektoriengenerierung zu übergeben und das Erreichen eines Zielpunkts, sowie das Ende einer Fahrt zu detektieren. Belegt werden kann dies durch die Visualisierung der Fahrt in den folgenden Abbildung 4 und Abbildung 5. Dort ist eine erfolgreiche Umschaltung dargestellt.

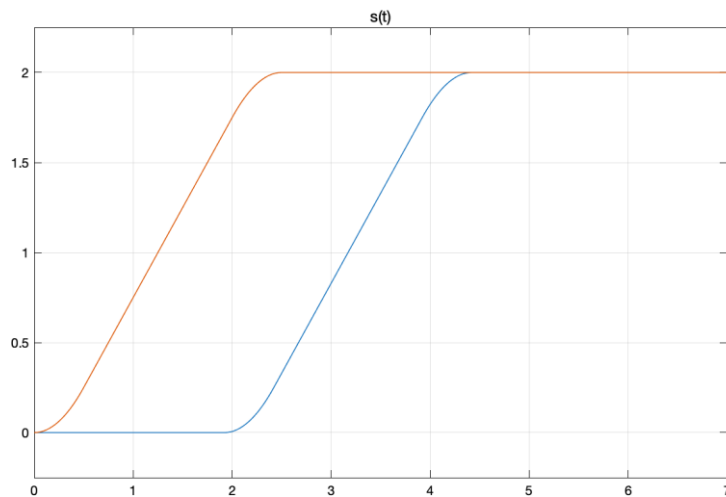


Abbildung 4 Umschaltung an der Strecke

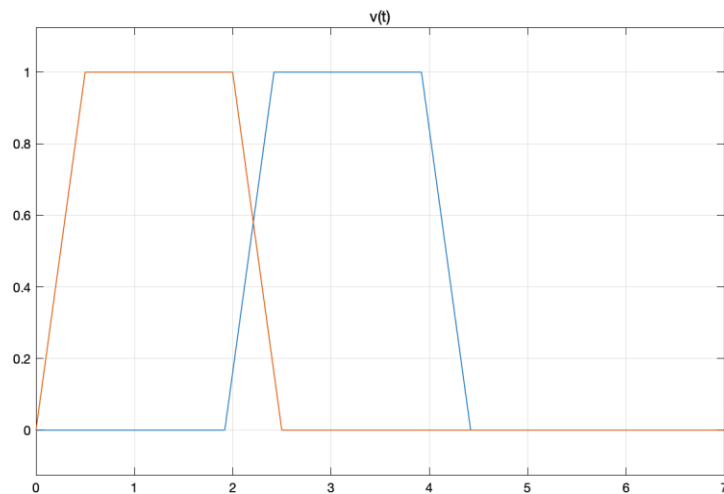


Abbildung 5 Umschaltung in der Geschwindigkeit

Hier ist deutlich zu erkennen, dass die Umschaltung der einzelnen Koordinaten funktioniert. Es zeigt sich eine Überlappung der Profile aus der Trajektoriengenerierung. In der Simulation lag der Schwerpunkt auf der Erprobung der Umschaltung und der gewählten Methodiken. Alle weiteren Funktionalitäten der Controller-Task sind direkt auf dem HAWino implementiert.

3.2 Eigene Variablen über Data Unit Types

Die Einhaltung einer strukturierten und nachvollziehbaren Arbeitsweise ist mit der Definition eigener Dateneinheitstypen gelungen. Dateneinheitstypen sind in der Programmierung besser bekannt als Data Unit Types oder kürzer DUTs.

Ein DUT [1] beschreibt den Typ und die inhaltliche Bedeutung einer Dateneinheit innerhalb eines Kommunikations- oder Verarbeitungssystems. Er legt fest, welche Art von Informationen eine

Dateneinheit enthält und wie diese interpretiert sowie weiterverarbeitet werden muss. Durch die eindeutige Klassifikation von Daten ermöglicht der DUT eine strukturierte und zuverlässige Kommunikation zwischen den beteiligten Systemkomponenten. Für den *RobotController* sind drei wichtige DUTs definiert worden.

Das DUT *Path*, welches die Koordinaten und die Ausrichtung in θ beinhaltet, besteht aus den einzelnen Elementen der in Kapitel 3.1.2 erwähnten Spaltenvektoren.

Das zweite DUT ist der *positonValueBuffer* und dient als Puffer für die Koordinaten, die an die Trajektoriengenerierung gesendet werden sollen. Es ist eine vereinfachte Abwandlung der DUT *Path* und wird flexibel genutzt.

Letztes DUT ist das *RobotStatusValues*, es ist verkürzt als *Robot* in den Controller-Task eingefügt worden. Es bildet den aktuellen Zustand des Roboters in strukturierter Form ab. Es enthält Identifikations- und Aufgabeninformationen (*name*, *taskType*, *taskState*), Zustandsgrößen zur Beschreibung von Ist- und Soll-Position sowie Orientierung (*CurrentX*, *CurrentY*, *CurrentTheta*, *TargetX*, *TargetY*, *TargetTheta*, *OffsetX*, *OffsetY*, *TargetThreshold*) und boolesche Variablen zur Erfassung von Sensor-, Aktor- und Sicherheitszuständen (*Slider*, *LightBarrier*, *GripperLoaded*, *E_STOP*, *TargetReached*, *Gripper*). Damit stellt das DUT eine konsistente Zustandsverwaltung sicher und bildet die Grundlage für einen stabilen und sicheren Betrieb des Robotersystems.

3.3 Programmierung am HAWino

Die Controller-Task hat mit Priorität 15 die niedrigste Priorität aller Tasks und wird in einem Zeitintervall von 10 ms ausgeführt. Diese niedrige Priorität resultiert daraus, dass sie von allen anderen Tasks diejenige mit der geringsten Dringlichkeit ist. Ihre Hauptaufgaben bestehen darin, die Werte für die geplanten Fahrten des HAWinos zu empfangen, diese zu verarbeiten und die Erreichung der Zielkoordinaten zu überprüfen. Die Mechanismen zur Positionskontrolle reduzieren den Bedarf an einer höheren Priorisierung.

3.3.1 Aufbau des Programms

Der Aufbau des Programms wird in der folgenden Abbildung 6 als stark vereinfachtes Ablaufdiagramm dargestellt. Hier soll die generelle Ablauflogik und die Operationsweise des Programms verdeutlicht werden.

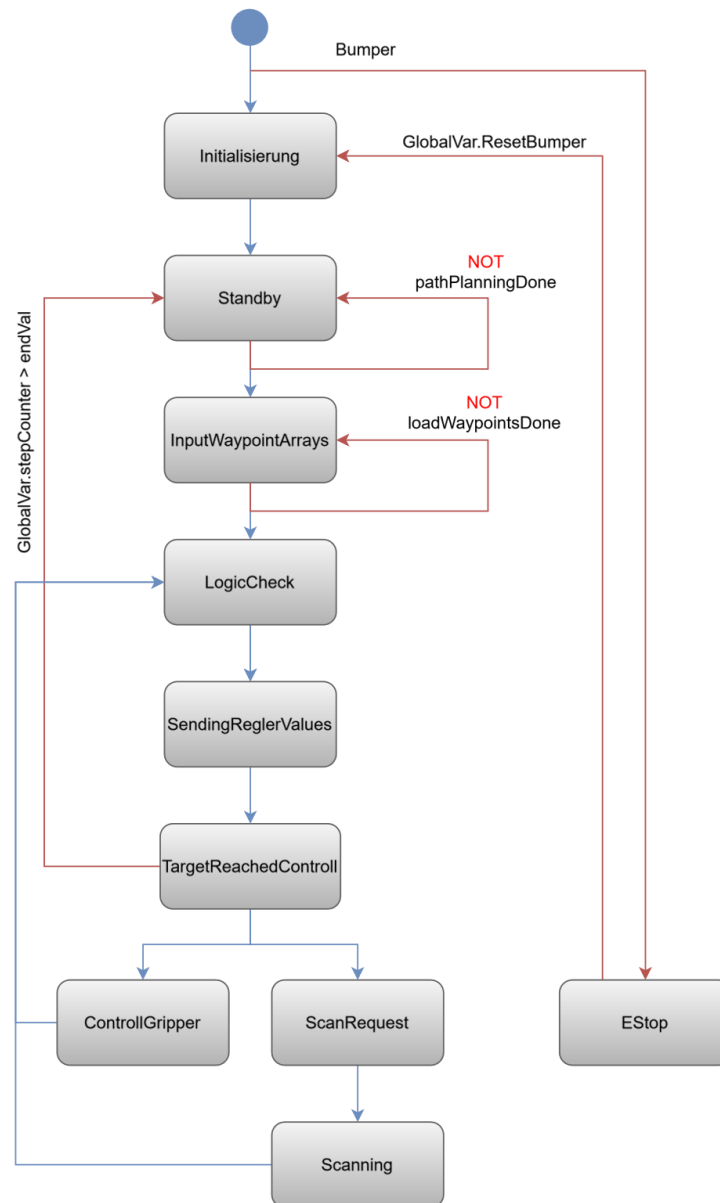


Abbildung 6 Überblick über das Programm RobotController

Der blaue Punkt im oberen Teil der Abbildung 6 stellt dabei den Programmstart dar. Die erste Verzweigung verdeutlicht die vorgeschaltete Prüfung des HAWinos und des Fahrauftrags selbst. Diese wird für eine bessere Übersicht erst nach dem Programmdurchlauf erläutert, hier folgt der planmäßige Durchlauf in Kurzfassung. Anschließend werden die Prüfungen und Vorarbeiten genauer erklärt. Im State *Init* wird der HAWino auf einen sicher verwendbaren Grundzustand gebracht. Von dort aus geht es in den State *Standby*, dort wird lediglich auf die Anweisungen aus anderen Gewerken gewartet. Ist ein Auftrag von den anderen Gewerken geplant worden, so wird über die Variable *pathPlanningDone* eine Transition in den nächsten State *InputWaypointsArray* möglich. Hier werden die Wegpunkte für die Verwendung vorbereitet und überprüft. Nach erfolgreicher Prüfung geht es in den *LogicCheck*. Dort werden verschiedene, für die Fahrten und Aufgaben relevante, Entscheidungen getroffen. Diese beeinflussen anschließend den Regler und die mechatronischen Elemente. Die Wegpunkte werden dann in *SendingReglerValues* an den Regler übergeben. Darauf erfolgt die Überprüfung, ob das Ziel erreicht worden ist. Ihr

nachgeschaltet werden die verschiedenen Varianten der Fahrt, damit gemeint sind die Aktionen, die nach Beendigung der Fahrt ausgeführt werden müssen. Beim Beenden des Auftrags geht es zurück in den State *Standby*. Bei einem Scan geht es in den *ScanRequest* und nach Bestätigung von Gewerk 2 in den State *Scanning*. Dieser wartet auf die Bestätigung von Gewerk 1 und endet mit dem *LogicCheck*. Auch der Greifer kann hier gesteuert werden. Dafür gedacht ist der State *ControllGripper*.

Begonnen wird mit der Aktualisierung der Prozessdaten. Dabei wird zuerst der Namen des HAWinos über die Variable *Var_HAWIno.Robot_Number* abgegriffen. Diese Integervariable wird über eine Funktion in einen String umgewandelt, dieser String ist die direkte Verknüpfung von Roboter Nummer und Kurzbezeichnung. Darauf folgt das permanente Übermitteln des Greiferzustands, ist dieser beladen oder nicht.

Damit innerhalb der Statemachine ein externer Befehl oder eine Fehlfunktion detektiert werden kann, ist eine mehrstufige Prüfung vorgeschaltet worden. Die erste konkrete Überprüfung des HAWinos stellt die Überwachung des Greifers dar. Diese ist über die Erkennung einer ansteigenden Flanke realisiert und überwacht die Lichtschranke im Greifer selbst. In Kombination mit dem Greiferzustand kann die in der folgenden Abbildung 7 dargestellte Arbeitsabfolge ausgelöst werden.

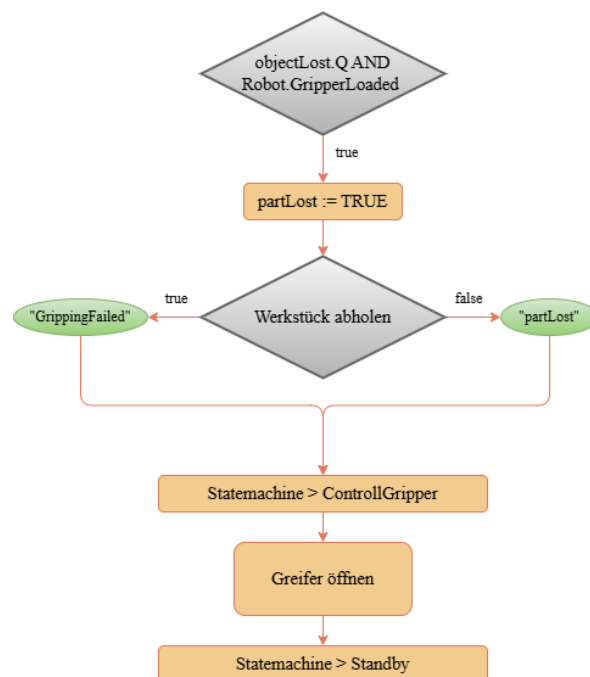


Abbildung 7 Prüfung auf Teilverlust

Ist der Greifer beladen und die ansteigende Flanke wird erkannt, wird eine Variable *partLost* als Bedingung für die spätere Abfrage im Greifer gesetzt. Gleiches gilt für das misslungene Greifen eines Werkstücks in einer Tasche. Es wird zwischen dem Verlust während der Fahrt und dem Fehlgriff unterschieden. Der aktuelle State wechselt sofort auf den State *ControllGripper* und alle vorherigen Aktionen oder folgenden States werden übersprungen. Dort angekommen wird der Greifer in den geöffneten Zustand zurückgesetzt. Abschließend wechselt der Controller in den State *Standby* und wartet auf Anweisungen.

Anschließend folgt die Abfrage der Bumperleiste. Wird diese ausgelöst, ist das als Notfall zu werten und der HAWino muss sofort in den Nothalt wechseln. Besser dargestellt in der folgenden Abbildung 8.

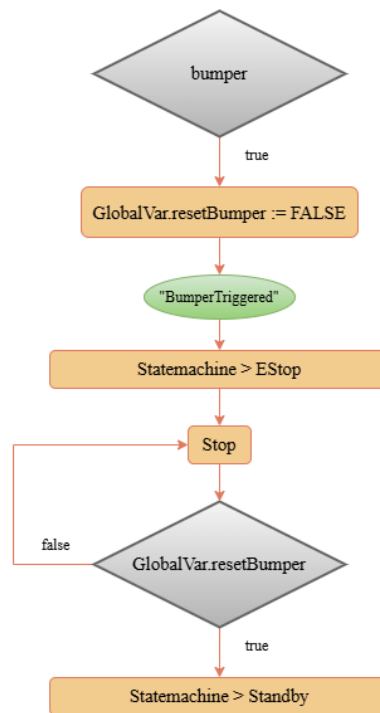


Abbildung 8 Prüfung auf Bumper

Wird der Bumper aktiviert, erfolgt die entsprechende Rückmeldung über die definierten Fehlermeldung. Der Controller wechselt in den State *EStop* um von dort aus die Motoren zu deaktivieren. Die Variable *GlobalVar.resetBumper* kann nur durch die anderen Gewerke zurückgesetzt werden, der Controller verbleibt also in diesem State bis die Gewerke den Nothalt freigeben. Nach der erteilten Freigabe wechselt der Controller in den *LogicCheck* und arbeitet den Fahrauftrag weiter ab. Anzumerken sei, dass sich Gewerk 2 dazu entschieden hat die Fahrplanung nach einem solchen Nothalt neu zu generieren. Der Auftrag wird damit abgebrochen und ein weiteres Abarbeiten ist unterdrückt.

Die Letzte Prüfung bildet das *JobAbort*, welches in den Gewerken als Abbruchbedingung für den Fahrauftrag gilt. Der Mechanismus dahinter findet sich in der folgenden Abbildung 9.

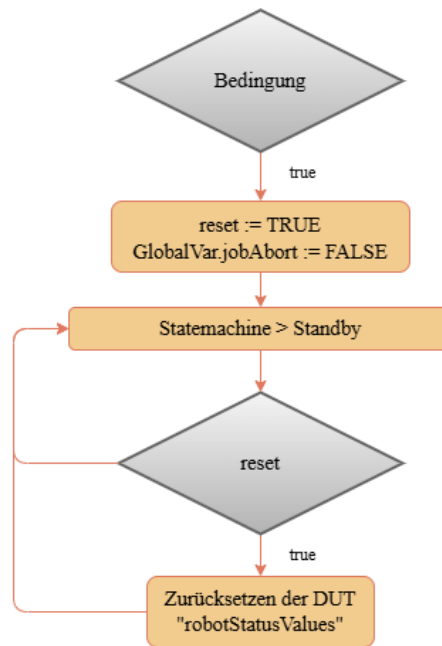


Abbildung 9 Prüfung, ob es einen Abbruch gab

Die in der Abbildung mit *Bedingung* abgekürzte Abfrage setzt sich zusammen aus der in der folgenden Formel (5) dargestellten Logik.

$$Bedingung := (GlobalVar.jobAbort \vee testJobAbort) \wedge \neg partLost \wedge GlobalVar.ResetBumper \quad (5)$$

Wird die Variable *GlobalVar.jobAbort* von den beteiligten Gewerken gesetzt, dazu ist das Bauteil nicht verloren und der Bumper ist nicht aktiviert, wird der Abbruch erkannt. Der Controller setzt eine Bedingung, welche nachfolgend für ein Zurückführen der Steuerung in den Grundzustand notwendig ist.

3.3.2 Statemachine des Controllers

Die Statemachine ist in insgesamt zehn verschiedene States unterteilt, von denen jeder eine spezifische Aufgabe innerhalb des Regelungsablaufs übernimmt. Der Übergang zwischen den States erfolgt auf Grundlage definierter Transitionsbedingungen. Diese Bedingungen werden durch das Scheduling, interne Überprüfungen der Controller-Task oder externe Inputs von den beteiligten Gewerken bestimmt.

Der State: Init

Dieser State bildet den initialen Zustand des RobotControllers und dient der Initialisierung sowie der Rückführung des HAWinos in einen definierten Grundzustand. Ziel ist die Sicherstellung konsistenter und gültiger Systemzustände als Grundlage für den weiteren Ablauf der Robotersteuerung. Der gesamte Ablauf findet sich in der folgenden Abbildung 10.

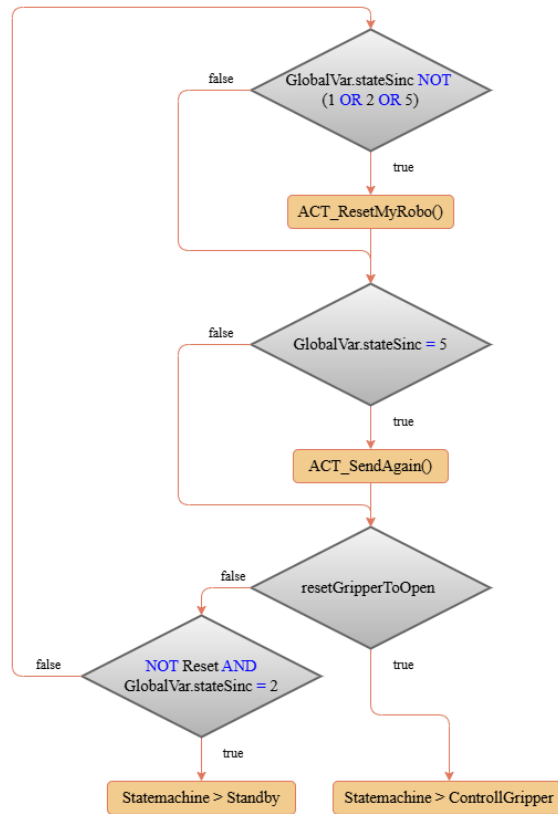


Abbildung 10 Ablaufdiagramm des States Init

Über die Action *ACT_ResetMyRobo* wird das DUT zu Beginn in einen definierten Ausgangszustand überführt. Das erfolgt jedoch nur, wenn sich die Trajektoriengenerierung im Initialzustand, im Standby oder im Normalbetrieb befindet. Innerhalb dieser Action wird zunächst die Variable *Reset* zurückgesetzt, diese ist möglicherweise in Fehlerfällen gesetzt worden. Anschließend erfolgt eine Überprüfung des Greiferzustands mittels einer booleschen Logik, diese findet sich in den folgenden Formeln (6) und (7).

$$var := (Robot.GripperLoaded \wedge LightBarrier) \vee GlobalVar.ResetGripperToOpen \quad (6)$$

$$resetGripperToOpen := var \vee initialUseOfRobot \quad (7)$$

Die Formel (6) ist eine Hilfestellung für die Darstellung in Formel (7). Ist der Greifer als beladen markiert und die Lichtschranke liefert ein durchgehendes Signal oder wünscht eines der Gewerke das Öffnen oder handelt es sich um das erstmalige Starten des HAWinos, so muss der Greifer geöffnet werden. Die spätere Abfragebedingung hierfür ist die Variable *resetGripperToOpen*.

Die darauf folgende Abzweigung ist die Abfrage auf das Erhalten der Wegpunkte. Hat die Trajektoriengenerierung die erneute Übermittlung der Wegpunkte gefordert, so geschieht dies hier über die Action *ACT_sendAgain*.

Muss der Greifer geöffnet werden, so wird nach dem erneuten Senden in den entsprechenden State gewechselt. Hat die Trajektoriengenerierung die Zurücksetzung beendet und ist bereit für neue Wegpunktübertragung, wird direkt in den State *Standby* gewechselt.

Der State: Standby

In dem State *Standby* wird auf einen Startimpuls gewartet. Über das DUT wird dieser Status auch im RobotController angezeigt. Der genaue Ablauf findet sich in der folgenden Abbildung 11.

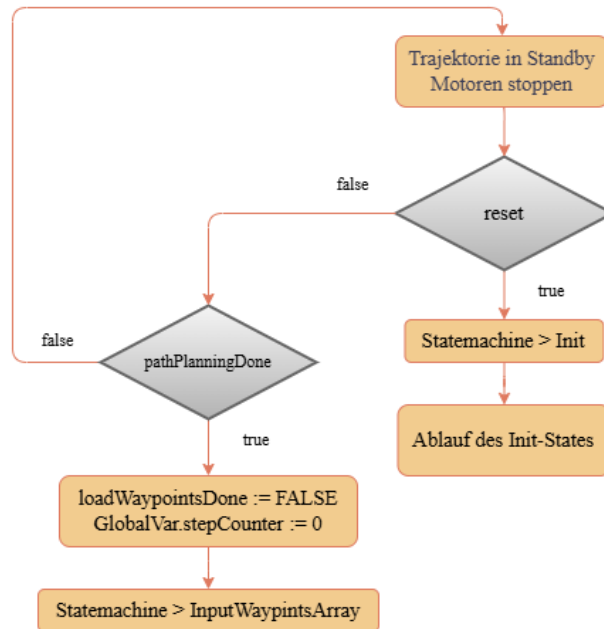


Abbildung 11 Ablaufdiagramm des States Standby

Die Trajektoriengenerierung wird unterbrochen und die Motoren im Regler werden gestoppt. Ist in vorangegangenen Aktionen die Variable *reset* gesetzt worden, so wird von hieraus zurück in den State *init* gewechselt. Sobald die Planung der Fahrt abgeschlossen ist und das Signal *MQTT_Bahnplanung_Sync.pathPlanningDone* für den HAWino freigegeben ist, kann der Roboter den State wechseln. Diese boolesche Variable dient hier als Semaphore und signalisiert, dass Gewerk 2 ihre Arbeit beendet hat und die Daten bereit zur Abholung stehen. Die Prüfbedingung für den nachfolgenden State wird vorsorglich zurückgesetzt und der Wegpunktzähler wird auf einen Anfangswert 0 geschrieben.

Der State: InputWaypointsArray

Der State *InputWaypointsArray* liest alle gegebenen und für die Fahrt notwendigen Daten in den Robot-Controller ein. Dabei sei darauf hingewiesen, dass es Unterschiede in der Realisierung der Schnittstellendefinition zwischen den Gewerken gibt. Da eine Matrix oder ein Vektor in variabler Länge nicht an eine Funktion übergeben werden kann und diese Hürde erst bei der Fehlersuche bemerkt worden ist, unterscheiden sich die beiden Ansätze. In Gewerk 2 werden einzelne Vektoren verwendet und in Gewerk 3 eine einzige Matrix. Die Unterschiede sind minimal, jedoch bedarf es einer Übersetzung damit der HAWino problemlos arbeiten kann und der Code nicht erneut überarbeitet werden muss.

Die Übersetzung geschieht gemäß den folgenden Formeln (8) und (9) .

$$N := GlobalVar.numberWayPoints \quad (8)$$

$$Waypoints_{i,j} = \begin{cases} (X_i, Y_i, \theta_i, T_i) & , falls i < N - 1 \\ (X_{N-1}, Y_{N-1}, \theta_{N-1}, T_{N-1}) & , sonst \end{cases} \quad (9)$$

Hierbei seien N die Anzahl der übergebenen Wegpunkte, X und Y die Koordinaten, θ die Ausrichtung zu Beginn jeder Fahrt und T markiert die mit dem Wegpunkt verbundene Aktion. Besser verstanden werden kann diese Übersetzung unter Betrachtung der folgenden Abbildung 12, dort ist die angewandte Logik visualisiert.

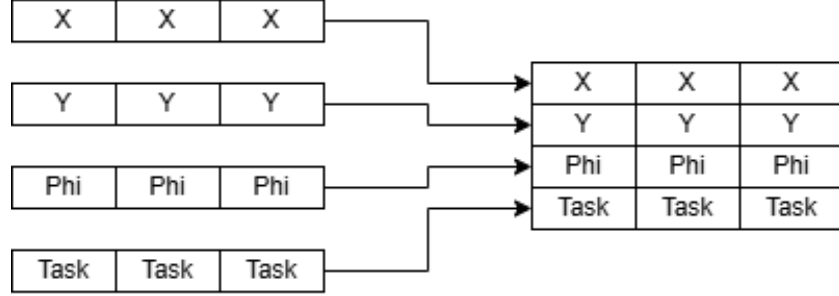


Abbildung 12 Übersetzung der Wegpunkte aus Gewerk 2

Die zweite große Besonderheit ist die Länge der Vektoren aus Gewerk 2, diese kann je nach Fahrt stark variieren. Jedoch sind die übergebenen Wegpunkte in der Länge immer 256 Einträge breit und werden vor jeder Fahrtplanung mit einem Nullwert in jedem Element initialisiert.

Während der Durchführung des Verbundprojekts hat sich die Fehleranfälligkeit des Systems bemerkbar gemacht. Bei der Optimierung der Fahrtplanung kam es gelegentlich zur Übergabe von Nulleinträgen innerhalb der Vektoren, in einigen Fällen stimmte die Längenangabe nicht. Zur Eindämmung dieser Problematik sind verschiedene Eingrenzungsmöglichkeiten verwendet worden. Um die Übergabe von Nullwerten an den RobotController zu vermeiden, wird die 4×256 Matrix nach Einlesen des letzten gültigen Wegpunkts mit dem letzten Wegpunkt gefüllt und nicht mit den folgenden Einträgen aus Gewerk 2.

Als zusätzliche Sicherheit und zur Prüfung der Ergebnisse aus Gewerk 2 ist die Transitionsbedingung zum nachfolgenden State an eine triviale Rechnung gebunden. Diese ist in den folgenden Formeln (10) und (11) dargestellt. Die Darstellung erfolgt aus Gründen der Übersicht vereinfacht.

$$checkValue = \sum_{i=0}^3 \sum_{j=0}^{255} c \begin{cases} c = 1 & , falls Waypoints_{i,j} \neq 0 \vee |Waypoints_{2,j}| < \pi \\ c = 0 & , sonst \end{cases} \quad (10)$$

$$WaypointsLoaded := (1026 = checkValue) \quad (11)$$

In Worten ausgedrückt wird hierbei ein Zähler mit dem Namen *checkValue* inkrementiert, wenn die Einträge der Positionen und die Art der Fahrt ungleich Null sind und der Wert für θ in seinem Betrag kleiner als π ist. Dieser Wert kann mit dem Wert 1026 verglichen werden. Es sind 1026 Werte, da es vier Vektoren mit jeweils 256 Einträgen sind. Erfüllen diese die Bedingungen, so kann in den nächsten State gewechselt werden. Das erfolgreiche Laden der Wegpunkte wird an Gewerk 2 übermittelt.

Der gesamte State ist dargestellt in der folgenden Abbildung 13, dort ersichtlich ist das Ablaufdiagramm.

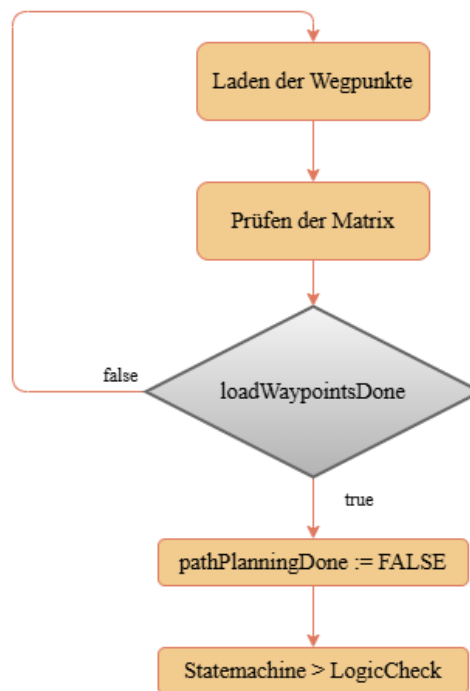


Abbildung 13 Ablaufdiagramm des States InputWaypointArray

Der State: LogicCheck

Dieser State erfüllt mehrere Aufgaben, welche für die Fahrt selbst und den Regler, sowie für die Trajektorie wichtig sind. Hauptaufgabe ist das Unterscheiden zwischen translatorischen und rotatorischen Bewegungen, da diese nicht simultan stattfinden. Dazu kommen kleinere Berechnungen und Prüfungen, welche die Robustheit positiv beeinflussen. Das Ablaufdiagramm des States ist in der folgenden Abbildung 14 dargestellt.

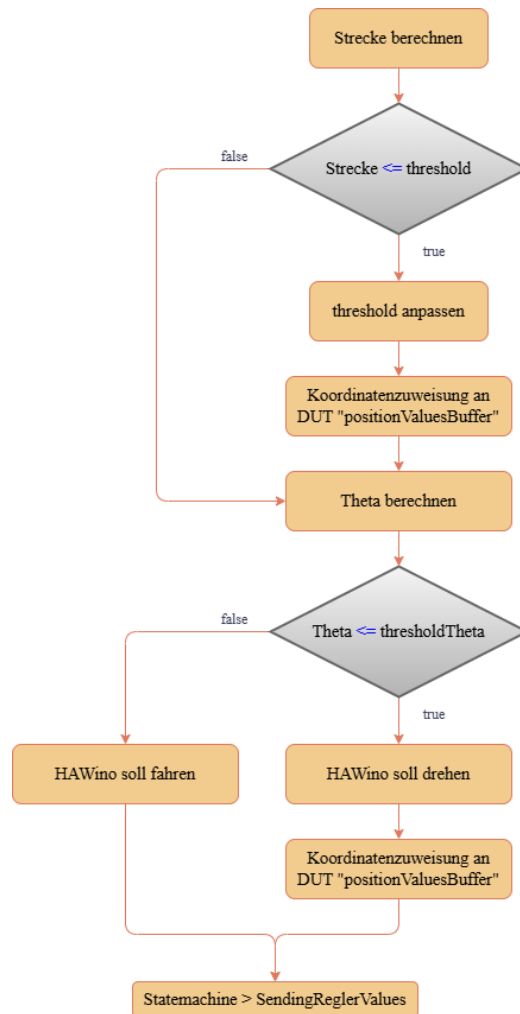


Abbildung 14 Abbildung 13 Ablaufdiagramm des States LogicCheck

Zuerst wird die Strecke zwischen den Eckpunkten berechnet, diese Strecke wird für eine Kontrollrechnung verwendet, um folgendes Phänomen zu verhindern. Bei dem Überschleifen der Wegpunkte kommt es zu dem seltenen Ereignis, dass es Wegpunkte gibt, die vor Beginn der Fahrt bereits erreicht sind. Das Problem dahinter ist in der folgenden Abbildung 15 besser dargestellt.

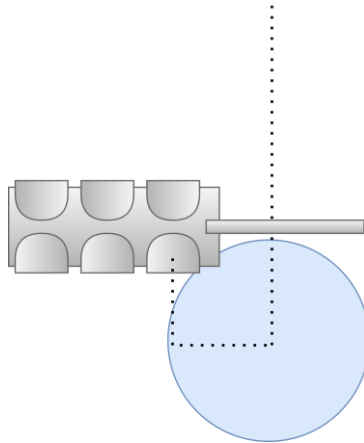


Abbildung 15 Phänomen der erreichten Fahrt

Der blaue Kreis symbolisiert den *threshold*, dieser ist so groß, dass er den nächsten Wegpunkt beinhaltet. Häufig tritt dies innerhalb oder an den Stationen auf, dort wo die Fahrtplanung auf engem Raum agieren muss.

Um dieses Phänomen zu unterbinden, wird der *threshold* vor dem Fahrtbeginn überprüft. Dafür wird der Abstand der beiden Wegpunkte über die bereits bekannte Formel (2) berechnet. Ist die Strecke kleiner als der *threshold* wird ersatzweise die Hälfte der Strecke als neuer *threshold* angenommen. Ist dieser zu klein wird der *thresholdMin* verwendet.

Nach erfolgreicher Prüfung und gegebenenfalls der Anpassung, werden die Koordinaten in das DUT *positionValueBuffer* zur flexiblen Verwendung geschrieben. Darauffolgend wird der Winkel zwischen dem HAWino und der Startausrichtung, sowie den nächsten zwei Wegpunkten berechnet. Anschaulicher ist das damit verbundene Problem in der folgenden Abbildung 16 dargestellt.

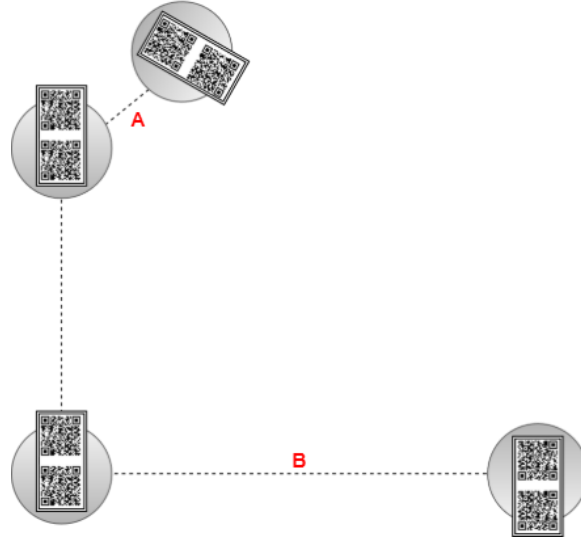


Abbildung 16 Problematik der Winkelstellung

Bei der mit A markierten Fahrt ist eine Diskrepanz in der Startausrichtung zu erkennen. Diese wird nicht während einer translatorischen Fahrbewegung ausgeregelt, sie erfolgt gesondert. Bei der mit B markierten Strecke ist ein Wechsel der Ausrichtung zu erkennen. Dieser erfolgt beispielsweise bei der Fahrt zur Ladestation und muss rechtzeitig erkannt werden.

Die hierfür verwendete Systematik hat zu Beginn des Projekts große Hürden mit sich gebracht und konnte mit einer trivialen Lösung überwunden werden. Die große Hürde ist der Kipppunkt des Winkels bei $\pm\pi$ und die Veränderung der Ausrichtung innerhalb der Fahrt. Auch problematisch ist die Normalisierung des Winkels auf diesen Wertebereich. In der anfänglichen Prüfung galt die in der folgenden Formel (12) verwendete Berechnung.

$$turningFirst := (thresholdTheta > ||\theta_{n+1}| - |\theta_n||) \quad (12)$$

Diese Logik ist für drei Berechnungen verwendet worden. Auf diese hat eine aufwändige und anfällige Fallunterscheidung zur Feststellung der Ausgangssituation gefolgt.

Die endgültige Lösung ist erreicht durch die Nachahmung des Befehles *ATAN2*, dieser wird verwendet, um die Quadranten zu bestimmen und den Winkel zu normalisieren. Die folgende Formel (13) ist dabei Gedankenstütze der Berechnung, da sie die Beziehung zwischen den Kreiskoordinaten und dem Winkel herstellt.

$$\Delta\theta = \operatorname{atan}\left(\frac{\sin(\Delta\theta)}{\cos(\Delta\theta)}\right), \quad \theta \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \quad (13)$$

Diese Formel ist aus Berechnungssicht sinnlos, da hier die Tangensfunktion mit dem Winkel berechnet wird und aus dem Ergebnis über die Umkehrung wieder der Winkel. Die Berechnung liefert nicht den korrekten Quadranten und damit das korrekte Vorzeichen der Berechnung. Über die nachprogrammierte Funktion *ATAN2* ist es möglich dies auszugleichen, wie in der folgenden Formel (20) dargestellt.

$$\Delta\theta = \operatorname{atan2}\left(\frac{\sin(\Delta\theta)}{\cos(\Delta\theta)}\right), \quad \theta \in -\pi, \pi] \quad (14)$$

Mit dem Betrag des $\Delta\theta$ aus der Funktion ATAN2 ist diese Überprüfung mit der Formel (12) stark vereinfacht worden und über eine FOR-Schleife ist der Programmieraufwand zusätzlich verkleinert. Dargestellt ist die Normalisierung in der folgenden Abbildung 17.

```
// delta Theta für x1 - current und x2 - current
FOR i := 0 TO 1 DO
    deltaTheta[i] := waypoints[GlobalVar.stepCounter + i,2] - Robot.CurrentTheta;
    deltaTheta[i] := ABS(ATAN2(SIN(deltaTheta[i]), COS(deltaTheta[i])));
END_FOR
// delta Theta zwischen x1 und x2
deltaThetaWaypoints := waypoints[GlobalVar.stepCounter + 1,2] - waypoints[GlobalVar.stepCounter,2];
deltaThetaWaypoints := ABS(ATAN2(SIN(deltaThetaWaypoints), COS(deltaThetaWaypoints)));
```

Abbildung 17 Normalisierungslogik über ATAN2

Mit den normalisierten Winkeln lässt sich nun nachvollziehbar ein Unterschied in der Ausrichtung erkennen. Ist dieser erkannt wird das DUT zur Speicherung der Wegpunkte mit der Ist-Position in x und y überschrieben. Der HAWino muss auf der aktuellen Position drehen. Ist keine Notwendigkeit für eine Drehung erkannt, so wird der State dementsprechend gewechselt.

Der State: SendingReglerValues

Dieser State übergibt die DUT aus dem LogicCheck an die DUT der Trajektorie. Der Ablauf dafür findet sich in der folgenden Abbildung 18.

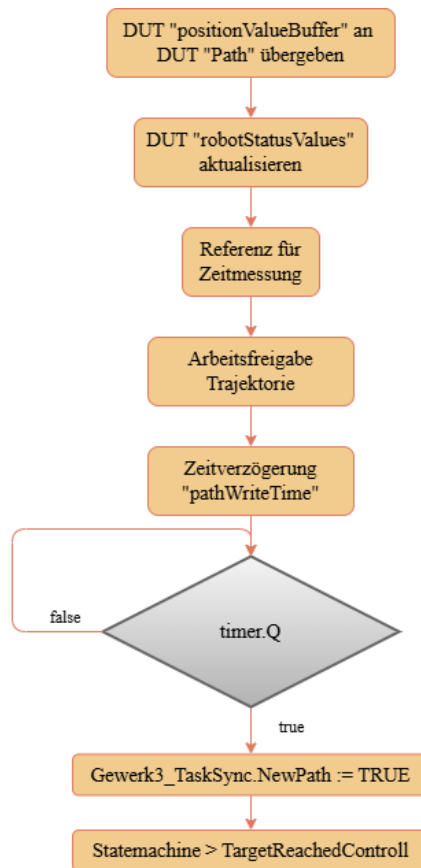


Abbildung 18 Ablaufdiagramm des States SendingReglervalues

Nach der Übergabe der Koordinaten erfolgt eine Aktualisierung der Informationen. Zudem wird eine Referenzzeit aufgenommen, diese wird später für eine Zeitmessung verwendet. Die Trajektoriengenerierung erhält ihre Arbeitsfreigabe und parallel wird eine Zeitverzögerung für das Rückmelden der Übergabe gestartet. Nach Ablauf dieser Zeit erhält die Trajektoriengenerierung ein Semaphore. Dieser ist zuständig für das Scheduling der Zielüberwachung. Der State wechselt auf die *TargetReachedControll*

Der State: TargetReachedControll

Die Überwachung der Zielkoordinate und das Steuern des folgenden Arbeitsverlaufs werden über die TargetReachedControll ermöglicht. Zusätzlich ist hier die Zeitüberwachung eingebettet. Wieder findet sich der Ablauf in dem folgenden Diagramm der Abbildung 19.

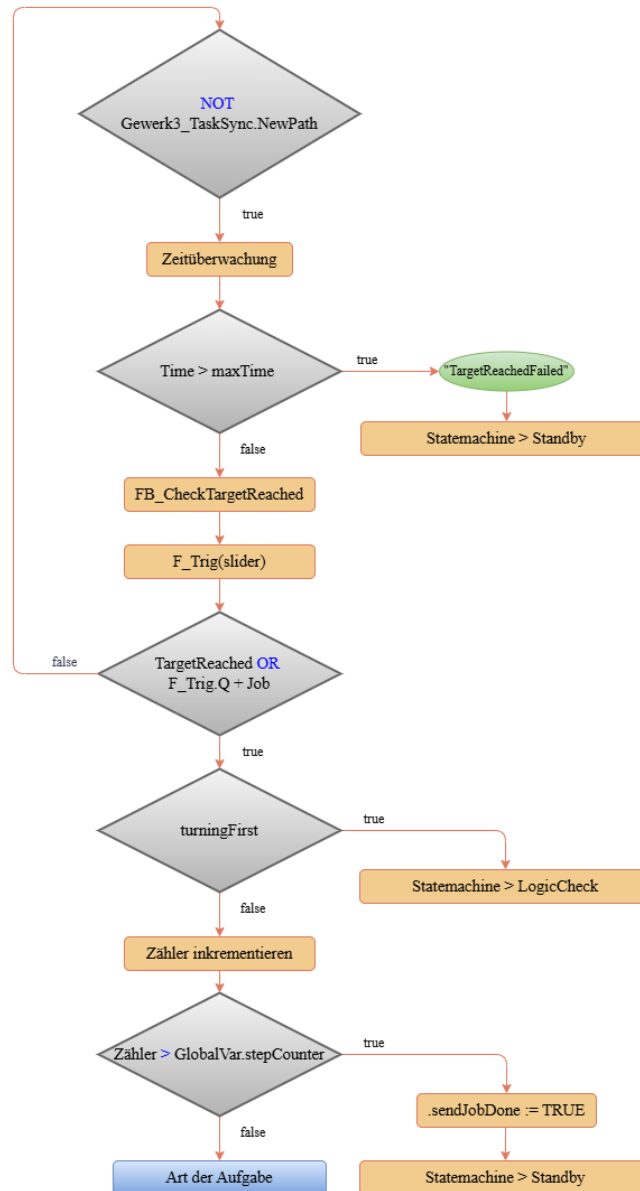


Abbildung 19 Ablaufdiagramm des States TargetReachedControl

Ist die Übergabe der Werte erfolgt und die Trajektoriengenerierung startet den Prozess, kann mit der Zeitüberwachung begonnen werden. Die Berechnung der Zeitdifferenz von der gesetzten Referenzzeit zur aktuellen Zeit erfolgt über folgende Formel (15).

$$\Delta t := t - t_{\text{referenz}} \quad (15)$$

Ist dieses Δt größer als die definierte maximale Arbeitsdauer, so löst die Zeitüberwachung aus. Der RobotController wechselt in den State *Standby* und führt einen Reset der DUT *RobotStatusValues* durch, parallel dazu erhält Gewerk 2 eine Benachrichtigung über die Fehlermeldung.

Die Umschaltung und das Erreichen der Zielkoordinate ist auf zweifachem Wege abgesichert. Im ersten durch die Überprüfung der aktuellen Position und zusätzlich über die Abfrage des Schiebers am Greifer selbst. Über die bereits bekannte Formel (2) lässt sich der Betrag des Restweges aus der aktuellen Position und dem Zielpunkt berechnen. Diese kann, wie in der Konzeptionierung validiert, mit dem aktuellen

threshold aus dem Aufgabentyp verglichen werden. Dieser ist im Vorfeld an die DUT übergeben worden. Hier ist auch unterschieden, ob es sich um eine rotatorische oder eine translatorische Bewegung handelt. Ein Ende der Rotation hat den State LogicCheck zur Folge.

Über eine fallende Flanke auf dem Signaleingang des Schiebers kann die Betätigung bei Erreichen einer Taschenposition erkannt werden. Ist die Art der Fahrt also eine Abhol-, Bring-, oder Scanfahrt ist das Ziel bei der fallenden Flanke erreicht. Ist das Ziel ein Wegpunkt reicht die Prüfung der Koordinate über die Funktion CheckTargetReached. Diese gibt eine boolesche Variable zurück, wenn der Restweg kleiner als der *threshold* ist.

Ist der Wegpunkt oder das Ziel erreicht wird ein Zähler inkrementiert und ist dieser größer als die Anzahl der anzufahrenden Punkte gilt die Fahrt als beendet. Gewerk 2 erhält eine Rückmeldung über die globalen Variablen.

Da Wegpunkte mit spezifischen Aufgaben oder Aktion belegt sein können, folgt die Auswertung der *taskType*, also der verbundenen Aufgabe, hier. Muss ein Bauteil gescannt werden, wird in den State *ScanRequest* übergegangen. Soll ein Bauteil gegriffen oder abgelegt werden, wird der State zu *ControllGripper* gewechselt.

Der State: ControllGripper

In diesem State wird der Greifer angesteuert. Die verwendete Logik zeigt sich in folgendem Ablaufdiagramm dargestellt in der Abbildung 20. Die Namen der DUTs sind hier nicht ausgeschrieben, da diese für die Abbildung zu lang sind, es handelt sich um das DUT *Robot*.

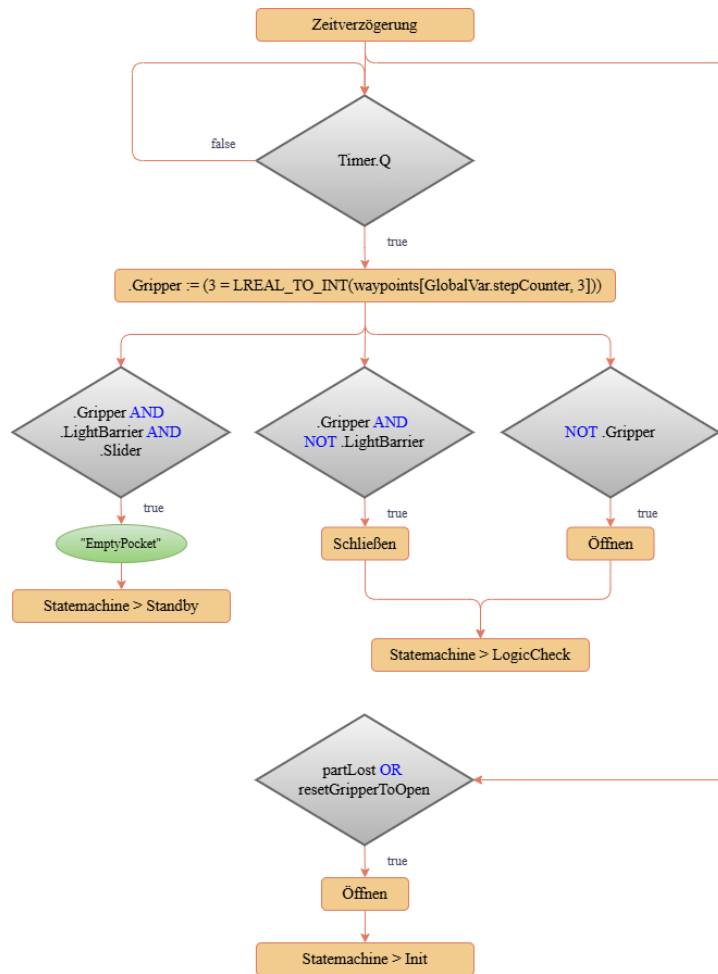


Abbildung 20 Ablaufdiagramm des States ControllGripper

Das Öffnen und Schließen des Greifers erfolgt zeitgesteuert. Nach Ablauf der definierten Verzögerungszeit wird über eine Reihe logischer Abfragen bestimmt, ob der Greifer geöffnet oder geschlossen werden soll.

Die Prüfbedingung ist der Vergleich zwischen Aufgabentyp 3 und der Aktion des aktuellen Wegpunkts. Stimmen diese überein ist die Bedingung *Robot.Gripper* erfüllt. Nach dem Verwenden des Greifers wechselt der State in den LogicCheck.

Hier ist klar zu erkennen, dass die zur Verfügung gestellte Lösung der Greiferansteuerung keine Verwendung fand. Stattdessen ist der Greifer ein direkter Teil der Statemachine geworden. Die Programmierung zeigt sich in einem Teilausschnitt in der folgenden Abbildung 21.

```
// Schließen
IF Robot.Gripper AND NOT Robot.LightBarrier THEN
  GripperVelocity := -32767;
  GripperEnable := TRUE;
  Robot.GripperLoaded := TRUE;
  gripperLoaded := TRUE;
  timer(IN:=TRUE, PT:=gripperTime);
```

Abbildung 21 Verlagerung der ursprünglichen Programmierung in die Statemachine

Über die Geschwindigkeit wird eine Richtung vorgegeben, die Bewegung selbst über das Setzen des Ausgangs *EnableGripper*. Es existiert kein externer Programmaufruf mehr, die Variablen sind direkt mit dem Greifer verknüpft. Hier zeigt sich auch die Aktualisierung des DUTs, der Greifer ist nach dem erfolgreichen Schließen als Beladen markiert.

Findet sich kein Bauteil in der Tasche, so wird eine Fehlermeldung ausgegeben und der State wechselt in den *Standby*. Der Abbruch des Auftrags erfolgt wieder durch Gewerk 2.

Ist der State nach einem *Reset* aufgerufen worden, muss der gewollte *resetGripperToOpen* oder das verlorene Bauteil über ein Öffnen des Greifers abgearbeitet werden. Nach dem Öffnen geht es auch hier wieder in den *Init* und anschließend in den *Standby*.

Der State: ScanRequest und Scanning

Hier sind zwei States vereinfacht zusammengefasst worden, um die Methodik hinter dem Scannen der Bauteile vereinfacht darstellen zu können. Der Ablauf findet sich in der folgenden Abbildung 22.

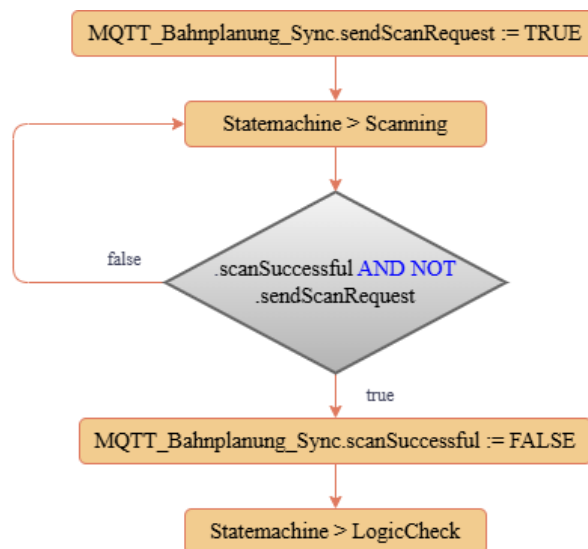


Abbildung 22 ScanRequest und Scanning vereinfacht

Der Controller übermittelt eine Anfrage über *MQTT_Bahnplanung_Sync.sendScanRequest* und wechselt von dem State *ScanRequest* in den State *Scanning*. Bekommt dieser die Rückmeldung über

MQTT_Bahnplanung_Sync.scanSuccessful und das Fehlen von *MQTT_Bahnplanung_Sync.sendScanRequest* wird das Scannen als erfolgreich angenommen und der State wechselt erneut zurück zum *Logic-Check*.

Der State: EStop

Der abschließende State ist der *EStop*. Dieser beinhaltet einzig und allein die Abfrage, ob der *Global-Var.ResetBumper* zurück auf den Wert *true* gesetzt worden ist. Zur Erinnerung, bei einem ausgelösten Bumper ist dieser auf *false* geschrieben worden. Der Nothalt bleibt bis zur Freigabe aktiv und kann nur durch die anderen Gewerke gelöst werden.

3.4 Testbetrieb

Während der Durchführung des Projekts hat sich gezeigt, dass es vorteilhaft ist die Programmänderungen direkt isoliert zu testen. Dabei muss der Testbetrieb nahezu frei von Störquellen und externen Einflüssen sein. Zudem gibt es Programmteile, welche nicht über den regulären Betrieb getestet werden können, weil diese Teile zum damaligen Stand in anderen Gewerken nicht implementiert waren. Ein zusätzliches Erschwernis bei der Erarbeitung des Programms am HAWino ist die Wiederholgenauigkeit von Fahrten oder die Konstruktion spezieller Abläufe zur Einstellung oder zur Datenaufnahme.

Um möglichst flexibel und frei von anderen Gruppen zu testen, ist der Testbetrieb implementiert worden. Dieser erlaubt das Verwenden des HAWinos in seiner ganzen Funktionalität und das isoliert und alleinig über den Programmcode von Gewerk 3.

Einen Überblick über den Testbetrieb ist dargestellt in der folgenden Abbildung 23. Zu erkennen ist die ergänzte Abbildung des vereinfachten Programmablaufs.

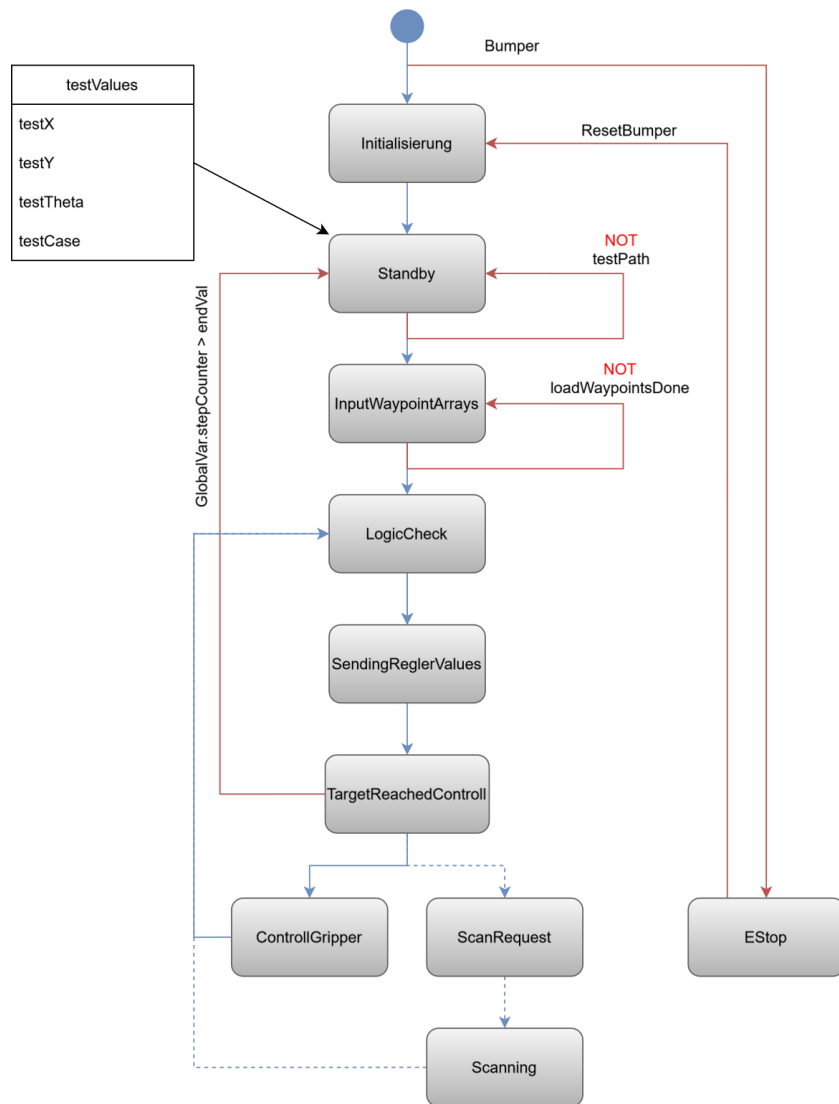


Abbildung 23 RobotController Testbetrieb

Der Testbetrieb ist abgeschnitten von jeglicher Kommunikation und verwendet eigene gewerksinterne Variablen. Die Abläufe sind identisch mit denen des regulären Betriebs, da es sich noch immer um denselben Programmcode handelt. Es sind hierbei lediglich die Verzweigungen überbrückt und mit der Kopie der Kommunikationsvariablen ist der Testbetrieb originalgetreu möglich.

Ist der Nutzer mit einem HAWino verbunden, so gibt es in der Programmierungsumgebung die Option Variablen direkt mit Werten zu belegen. Geschehen kann das über das Schreiben und Forcen von Werten [1]. Werden nun gezielt über einen Testbetrieb vordefinierte Szenarien ausgelöst, so kann der HAWino diese mit dem aktuellen Programmcode ausführen. In der Abbildung 23 ist ein solches Szenario als Block *testValues* dargestellt. Dieser Test kann mit dem manuellen Schreiben der Variable *testPath* in der Programmierungsumgebung ausgelöst werden.

Der Aufbau der *testValues* ist in der folgenden Abbildung 24 dargestellt, hier zu erkennen ist ein Ausschnitt aus der Programmierungsumgebung.

```

//// Test Values ////
//// Testfahrt um Stationen ////
numOfTestWP : DINT := 4;
inputX : ARRAY[0..4] OF LREAL := [1000.0, 4250.0, 4250.0, 500.0, 500.0];
inputY : ARRAY[0..4] OF LREAL := [1500.0, 1500.0, 3400.0, 3400.0, 1500.0];
inputTheta : ARRAY[0..4] OF LREAL := [0.0, 0.0, 0.0, 0.0, 0.0];
inputTask : ARRAY[0..4] OF LREAL := [1.0, 1.0, 1.0, 1.0, 1.0];
testPath, testPathEstop, testJobAbort: BOOL;

```

Abbildung 24 Beispiele für die testValues

Der Aufbau erfolgt exakt wie in der Schnittstellenbeschreibung definiert und von Gewerk 2 übergeben. Es existieren vier Arrays mit den Werten für die Strecke, die Ausrichtung und die Art der Fahrt. Dazu kommen die booleschen Variablen (*testPath*, *testPathEstop*, *testJobAbort*) welche die Kommunikation mit den anderen Gewerken ersetzen. Es ist also auch möglich einen Nothalt zurückzusetzen oder den Fahrauftrag abzuberechnen.

Auf einzelne Testszenarien wird direkt in den betroffenen Kapiteln der Ausarbeitung hingewiesen. Für den RobotController ist der Testbetrieb zur Validierung der Abläufe verwendet worden. Die dabei erstellten Szenarien sind simple Fahrten zu oder zwischen einzelnen Stationen gewesen. Auch das Aufnehmen der Werkstücke und die Fehlersuche der Greifersteuerung sind hiermit bewältigt worden.

4 Trajektoriengenerierung

Dieses Kapitel behandelt die Konzeption sowie Implementierung der Trajektoriengenerierung, um aus den Punkten der Bahnplanung Bewegungspfade im Raum zu erzeugen, die von den HAWinos abgefahren werden können.

4.1 Konzeption und Simulation

Bei der Trajektoriengenerierung geht es darum, einen Algorithmus auszuwählen, der aus den diskreten Wegpunkten der Bahnplanung von Gewerk 2 Trajektorien, also zeitparametrisierte Sollwerte berechnet, die in den Regelkreis gegeben und entsprechend verfolgt werden können. Bereits während der Konzeptionsphase ist zusammen mit Gewerk 2 festgelegt worden, dass die globale Bewegung der Roboter lediglich in einzelnen Abschnitten entlang der x- oder y-Achse erfolgen soll. Zudem ist festgelegt worden, dass die Drehung der Roboter auf die gewünschte Endposition für die Taschen oder die Ladestationen zu Beginn der Fahrt erfolgt. Abbildung 25 zeigt, wie die Bahnverfolgung der HAWinos aussehen soll. Die farbigen Kreise stellen dabei die verschiedenen Radien für das Umschalten der Koordinaten nach Kapitel 3.3.2 dar

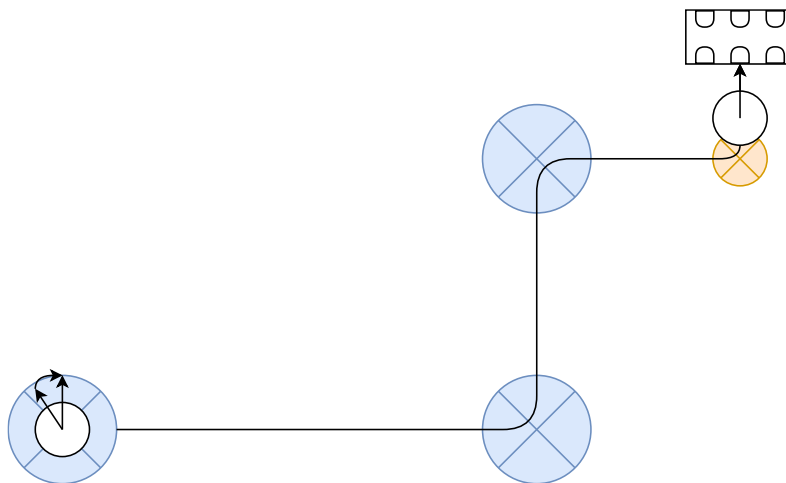


Abbildung 25 Konzept für die Bewegungspfade der HAWinos

Ziel der Trajektoriengenerierung ist es, dynamisch verträgliche, stetige und für den Regelkreis gut verfolgbare Sollwerte für Position und Geschwindigkeit zu erzeugen.

Die Kurven der Bewegungspfade werden durch das Überschleifen der Trajektorien, also das zeitliche Überlagern der Trajektorien der Bewegungsachsen x und y umgesetzt. In Abbildung 25 ist zu erkennen, dass für das Überschleifen der Kurven eine Umschaltung der Koordinaten beim Erreichen eines bestimmten Radius um den Endpunkt erfolgt. Der Radius variiert dabei in Abhängigkeit der Umgebung, sodass im Bereich der Station enger Kurvenradien gewählt werden als im freien Raum (siehe Kapitel 3.3.2, TargetReachedControll).

Das ursprüngliche Konzept für die Trajektoriengenerierung hat vorgesehen, dass die Abweichung von den geraden Verbindungslinien als orthogonale Geschwindigkeit auf die tangentielle Geschwindigkeit addiert wird. Für dieses Konzept müssen Trajektorien für die Strecke und die Geschwindigkeit der Roboter berechnet werden. Die Überprüfung des Konzepts mit MATLAB Simulink hat gezeigt, dass die Berechnung über die Abweichung nicht notwendig ist, da bereits die Sollwerte der berechneten Trajektorien in den Regelkreis gegeben werden können.

Daher ist auf die Berechnung der orthogonalen Geschwindigkeit verzichtet worden. Das gewählte Konzept konzentriert sich auf die Berechnung der Sollstrecke und des Geschwindigkeitsprofils, wodurch die Modellkomplexität reduziert wird. In Kombination mit dem nachgelagerten Regelkreis ermöglicht dieses Vorgehen dennoch eine stabile und stetige Bahnverfolgung. Die Sollstrecke dient der Lageregelung, während das Geschwindigkeitsprofil zur Vorsteuerung für die Geschwindigkeitsregelung genutzt wird.

Grundsätzlich basieren die Berechnungen für die Trajektorien auf den Bewegungsgleichungen für gleichmäßig beschleunigte Bewegungen. Diese sind in Tabelle 7 dargestellt.

Tabelle 7 Bewegungsgleichungen für eine gleichmäßig beschleunigte Bewegung

Funktion	Formel
Zeit-Ort-Gesetz	$s[mm] = 0.5 \cdot a \cdot t^2 + v_0 \cdot t + s_0$ (16)
Zeit-Geschwindigkeits-Gesetz	$v[\frac{mm}{s}] = v_0 + a \cdot t$ (17)

Bei den Fahrten sind zwei verschiedene Geschwindigkeitsprofilformen möglich. Die maximale Beschleunigung und Geschwindigkeit werden festgelegt. Abhängig davon, wie lang die zurückzulegende Strecke ist, kann das Geschwindigkeitsprofil entweder trapez- oder dreiecksförmig sein. Ist die Strecke kürzer als der doppelte Beschleunigungsweg, kann die maximale Geschwindigkeit nicht erreicht werden, wodurch ein dreiecksförmiges Geschwindigkeitsprofil entsteht. Tabelle 8 fasst die für die verschiedenen Profile verwendeten Formeln zusammen. Die Gesamtlänge der Strecke (im Folgenden Weglänge) lässt sich aus den Koordinaten des Start- und Endpunkts der Fahrt berechnen (siehe (1)).

Tabelle 8 Formeln für die Trajektoriengenerierung nach Profil

Funktion	Formel
Profilauswahl	Beschleunigungszeit: $t_a[s] = \frac{v_{max}}{a_{max}}$ (18)
	Beschleunigungsweg: $s_a[mm] = 0.5 \cdot a_{max} \cdot t_a^2$ (19)
Dreieckprofil	Beschleunigungsweg: $s_a[mm] = \frac{Weglänge}{2.0}$ (20)
Bedingung: Weglänge < 2.0 · s_a	Beschleunigungszeit: $t_a[s] = \sqrt{\frac{2.0 \cdot s_a}{a_{max}}}$ (21)

	Zeit mit konstanter Geschwindigkeit: $t_v[s] = 0.0$ Gesamte Zeit: $t_{end}[s] = 2.0 \cdot t_a$ (22)
Trapezprofil	Weglänge mit konstanter Geschwindigkeit: $s_v[mm] = Weglänge - 2.0 \cdot s_a$ (23) Zeit mit konstanter Geschwindigkeit: $t_v[s] = \frac{s_v}{v_{max}}$ (24) Gesamte Zeit: $t_{end}[s] = 2.0 \cdot t_a + t_v$ (25)

In Abbildung 26 ist zu sehen, wie die Umsetzung im Simulationsframework erfolgt ist.

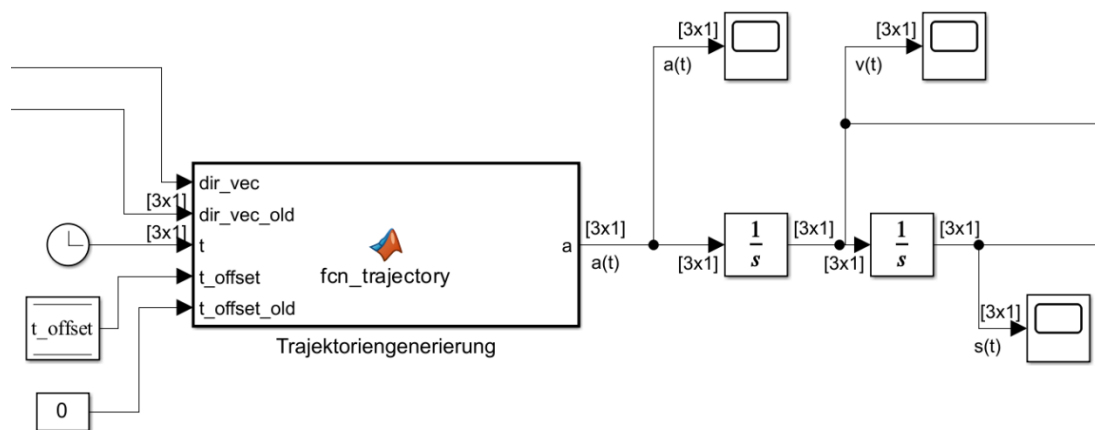


Abbildung 26 Umsetzung der Trajektoriengenerierung in MATLAB Simulink

Im Rahmen der Auswahl der Profile wird berechnet, wie lang die einzelnen Abschnitte der Profile dauern. Nach diesen Zeiten für Beschleunigung und Fahrten mit konstanter Geschwindigkeit wird festgelegt, welcher Wert für die Beschleunigung ausgegeben wird. Über die Integrator-Blöcke in MATLAB Simulink lassen sich die Beziehungen zwischen den Bewegungsgleichungen direkt nutzen. Der ausgegebene Beschleunigungswert wird mittels zweifacher Integration zunächst in eine Geschwindigkeit und dann in eine Strecke umgerechnet. Diese Werte werden an die entsprechenden Stellen im Regelkreis übergeben.

4.2 Implementierung der Trajektoriengenerierung

Dieses Kapitel befasst sich mit der Implementierung des gewählten Konzepts für die Trajektoriengenerierung. Für die Umsetzung ist ein Programm *Trajectory* mit einem eigenen Task erstellt worden. Dieses Vorgehen ist gewählt worden, um den Motor-Task, welcher die Regelalgorithmen beinhaltet und mit einer Zykluszeit von 0.333 ms läuft, nicht zusätzlich mit der Berechnung der Trajektorien zu belasten. Der zugehörige Task hat die Priorität 10 und die Zykluszeit 2 ms. Dies ermöglicht eine ausreichend schnelle Berechnung und Übergabe der Sollwerte an die Regelung. Nachfolgend wird zunächst der Aufbau dieses Programms betrachtet.

Wie im Kapitel *Konzeption und Simulation* beschrieben, hat das Konzept der Trajektoriengenerierung darauf basiert, für die x- und y-Achse Trajektorien zu berechnen und die Drehungen in θ ausschließlich über die Lageregelung durchzuführen. Allerdings hat sich im Verlauf des Projekts gezeigt, dass eine ruhigere Drehung möglich ist, wenn auch für die Drehung des Roboters eine Trajektorie berechnet wird. Daher werden in diesem Kapitel sowohl die Berechnungen der Trajektorie für die Drehungen als auch für die Fahrten in x- und y-Richtung erläutert. Für eine zeitnahe Inbetriebnahme des Systems ist der Regelkreis zunächst mit der Trajektoriengenerierung in x- und y-Richtung ohne Überschleifen umgesetzt worden. Das Überschleifen der Trajektorien für saubere Kurvenfahrten ist im weiteren Verlauf des Projekts ergänzt worden. In diesem Bericht wird in der x- und y-Richtung lediglich die Umsetzung mit Überschleifen der Trajektorien betrachtet.

4.2.1 Beschreibung des Trajectory-Programms

Das *Trajectory*-Programm ruft die Bausteine für die Berechnungen der Trajektorien in x-, y- und θ -Richtung auf. Des Weiteren erfolgt die Festlegung der Reglermodi für das Gain-Scheduling im Lage-regler. Das Programm tauscht den Reglermodus sowie die benötigten und berechneten Koordinaten mit den Programmen *RobotController* und *Main_MotorTask* über das E/A-Abbild und globale Variablen aus. Die verwendeten Variablen sowie die zugehörigen Quell- und Zielprogramme sind in Tabelle 9 zusammengefasst.

Tabelle 9 Austausch zwischen Trajectory, RobotController und Main_MotorTask

Variable	Datentyp	Programm (Quelle)	Programm (Ziel)
stateSinc	INT (vgl. Kapitel 2.3.2)	RobotController	Trajectory
stateSinc	INT (vgl. Kapitel 2.3.2)	Trajectory	RobotController
Path	TrajectoryValues x1, x2, y1, y2, theta1, theta2: LREAL	RobotController	Trajectory
Target	TargetValues x, y, vx, vy, theta, vtheta: LREAL	Trajectory	Main_MotorTask
Reglermode	ENUM_Reglermode NONE := 0, TURNING := 1, GLOBAL := 2, TOCHARGING := 3	Trajectory	Main_MotorTask

Die Variable *stateSinc* vom Typ INT ist eine globale Variable, die verwendet wird, um die Zustände des RobotControllers mit der Generierung der Trajektorie zu synchronisieren. So müssen die Bausteine zur Berechnung der Trajektorien nur aufgerufen werden, wenn tatsächlich eine Trajektorie berechnet werden soll. Die Variable *Path* ist vom Typ *TrajectoryValues* und beschreibt die Anfangs- und Endpunkte

für die Berechnung der Trajektorien. Die Sollwerte aus der Trajektorienberechnung werden mit der Variable `Target` an das Programm `Main_MotorTask` übergeben. Der Reglermode wird für das Gain-Scheduling im Regelkreis verwendet.

Der Ablauf des Programms wird in Abbildung 27 dargestellt. Start- und Zielwerte werden lediglich dann geschrieben, wenn neue Werte vom RobotController gesendet werden, ein Reset durchgeführt oder `stateSinc` der Wert 5 beziehungsweise *SendAgain* zugewiesen wird.

SendAgain wird verwendet, wenn die Start- und Zielwerte für x und y den Wert null annehmen. Dies ist erforderlich, da null für x und y keinen plausiblen Zielwert darstellt, da der Punkt $(0, 0)$ in der Bahnplanung nicht verwendet wird. Für θ ist 0 rad hingegen ein plausibler Wert, da θ den Wertebereich von $-\pi$ rad bis π rad umfasst. Sind die x - und y -Koordinaten jedoch nicht plausibel, darf auch die Winkelstellung nicht übertragen werden.

Dies ist insbesondere nach dem Laden der Konfiguration relevant, wenn die Programme initialisiert werden. In diesem Fall darf der Initialwert der Trajektorienberechnung für θ nicht fälschlicherweise mit 0 rad überschrieben werden. Befindet sich der Roboter nicht in einer Orientierung von 0 rad, soll jedoch später auf 0 rad drehen, würde diese Änderung nicht korrekt erkannt und folglich keine Trajektorie berechnet werden.

Daher ist es erforderlich, ausschließlich plausible Positionen in die Bausteine einzulesen. Andernfalls fordert das Programm `Trajectory` durch das Setzen von `stateSinc` auf *SendAgain* eine erneute Übertragung der Werte an. Dieser Vorgang wird so lange wiederholt, bis die Trajektoriengenerierung valide Start- und Zielwerte erhält.

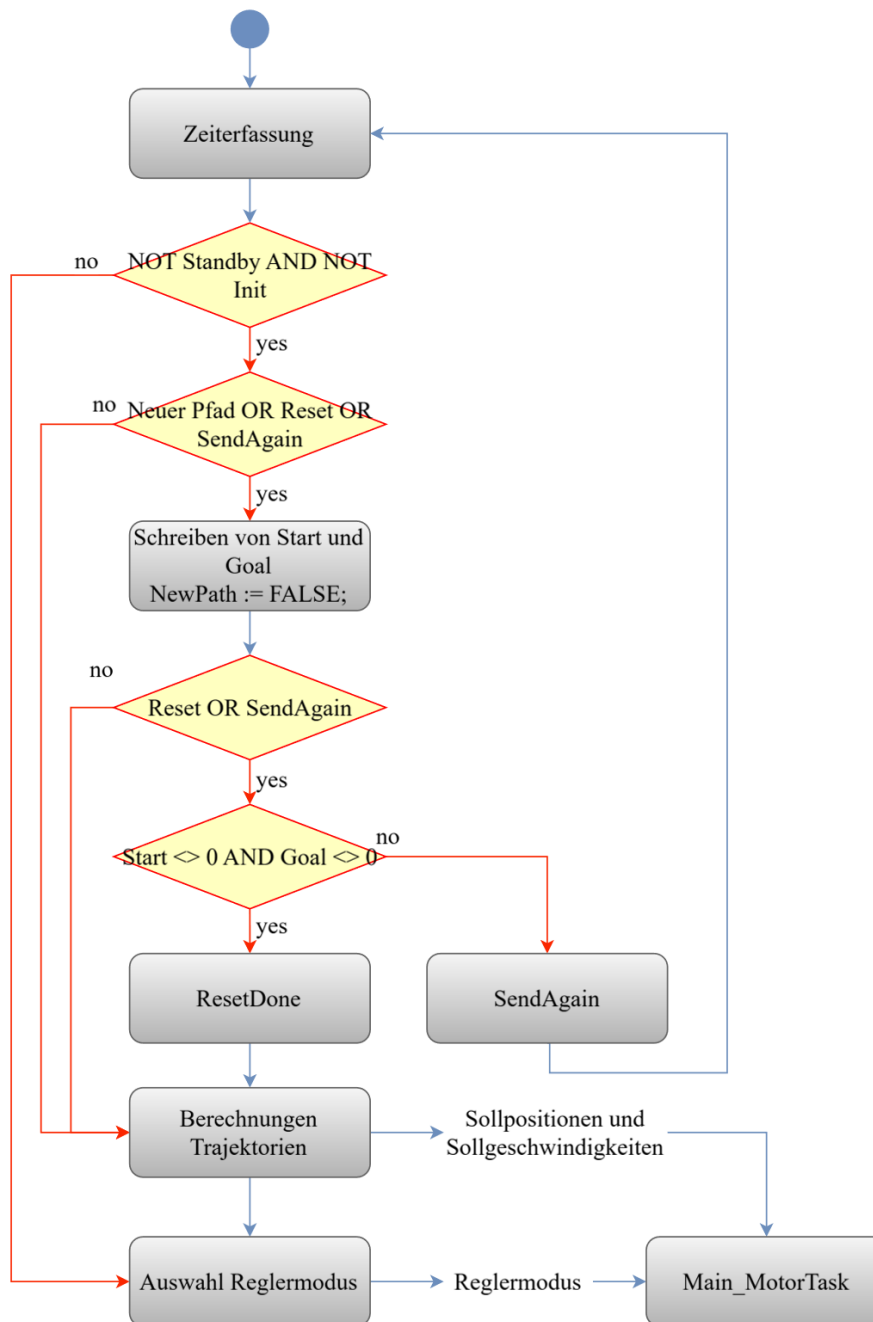


Abbildung 27 Ablaufdiagramm des Programms Trajectory

Sind Start- und Zielwerte geschrieben worden und im RobotController der stateSinc auf *Normalbetrieb* gesetzt, können die Trajektorien berechnet werden. Der Ablauf der Fahrten ist so aufgebaut, dass die HAWinos erst auf die gewünschte Endposition drehen und danach die Fahrten in x- und y-Richtung ausführen. Das bedeutet, dass es immer nur eine Änderung gibt, für die eine Trajektorie berechnet werden muss. Es ändert sich entweder θ , x, oder y. In den Berechnungsbausteinen wird ein *Working*-Flag gesetzt, das signalisiert, dass eine Berechnung läuft. Über dieses Flag wird die Auswahl des Reglermodus in Abhängigkeit von der Achse ermöglicht, für die aktuell eine Trajektorie berechnet wird. Der Ablauf der Logik zur Auswahl des Reglermodus ist in Abbildung 28 zu erkennen.

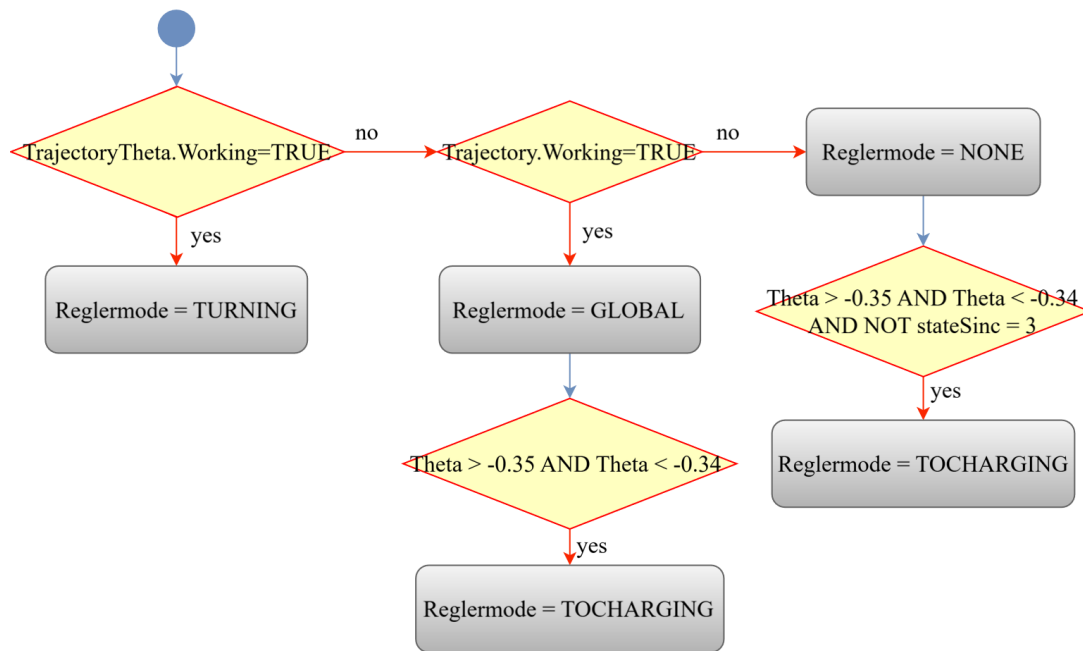


Abbildung 28 Auswahl des Reglermodus im Programm Trajectory

Die Abbildung zeigt, dass bei der Berechnung einer Trajektorie für θ der Reglermodus *TURNING* gewählt wird. Wird keine Trajektorie für θ , sondern für eine Fahrt in x oder y berechnet, wird der Reglermodus *GLOBAL* an das Programm Main_MotorTask übergeben. Dies wird allerdings mit dem Modus *TOCHARGING* überschrieben, falls der Sollwert für θ zwischen -0.35 rad und -0.34 rad liegt, denn dies ist der Winkel, um an die Ladestation heranzufahren. Die Prüfung wird hier als Wertspanne durchgeführt, da es sich um ein LREAL handelt und so nicht jede Nachkommastelle exakt übereinstimmen muss. Dieser Winkel ist experimentell ermittelt worden, indem die Roboter an die Ladestation angeschlossen und jeweils der Winkel aus den Kameramessdaten abgelesen worden ist.

Der Modus *TOCHARGING* wird jedoch nicht für globale Fahrten im Raum verwendet. Obwohl das Konzept ursprünglich besagt hat, dass die HAWinos immer vor der globalen Fahrt auf die benötigte Endposition drehen sollen, ist für die Anfahrt der Ladestation im späteren Verlauf des Projekts eine Änderung vorgenommen worden. In dem Fall drehen die Roboter erst direkt vor der Ladestation auf die benötigte Winkelstellung. Der Modus *TOCHARGING* sorgt nach dieser Drehung für eine saubere Anfahrt der Ladestationen.

Wird keine Trajektorie berechnet, erhält der Regelkreis den Reglermodus *NONE*. Allerdings gilt auch hier, dass für den Fall, dass der Sollwert für θ dem Winkel für die Anfahrt an die Ladestation entspricht, der Modus überschrieben und *TOCHARGING* als Reglermodus verwendet wird. Dadurch wird der Fall berücksichtigt, dass die Trajektorie für die Anfahrt an die Ladestationen schneller berechnet wird, als der HAWino sein Ziel erreicht. Damit wird sichergestellt, dass die Ladestation mit dem korrekten Modus angefahren wird.

4.2.2 Berechnung der Trajektorie für die Drehung in θ

Die Trajektoriengenerierung für die Drehung in θ ist im Projektverlauf erst nach der Umsetzung der Translation ergänzt worden. Sie wird in diesem Bericht dennoch zuerst beschrieben, da die Trajektoriengenerierung für die Fahrten in x- und y-Richtung konzeptionell auf demselben Aufbau basiert und sich darauf aufbauend besser erläutern lässt.

Die Generierung der Trajektorie für die Drehung in θ ist auf zwei Bausteine aufgeteilt. Der erste Baustein ist *FB_TrajectoryOneAxis*. Dieser beinhaltet den Zustandsautomat der Generierung. Dort wird der Baustein *FB_CalculateProfileOneAxis* aufgerufen, welcher die im Konzept aufgeführten Formeln (Tabelle 7, Tabelle 8) nutzt, um die Sollwerte für die Position und Geschwindigkeit zu berechnen.

Abbildung 29 zeigt ein vereinfachtes Ablaufdiagramm des Funktionsbausteins *FB_TrajectoryOneAxis*. Die Abbildung verdeutlicht den sequenziellen Ablauf der Zustandsübergänge sowie die Bedingungen für die Initialisierung und Beendigung einer Trajektorie. Die Auswahl der States erfolgt über die Variable *TrajState*. In diesem Fall kann diese den Wert *Active* haben, wenn eine Trajektorie berechnet wird oder den Wert *NoTraj*, wenn keine Berechnung erfolgt. Es ist zu erkennen, dass zunächst geprüft wird, ob der übergebene Startwert dem Zielwert entspricht. Wäre dies der Fall, muss keine Trajektorie berechnet werden und *TrajState* wird auf *NoTraj* gesetzt. Des Weiteren erfolgt über die Berechnung der Differenz zwischen dem vorangegangenen und dem neuen Ziel eine Prüfung, ob ein neues Ziel vorliegt. Das bedeutet, hat der Roboter zuvor auf 0 rad gedreht, ist dies das alte Ziel, welches für die Bildung der Differenz genutzt wird. Die Differenzberechnung erfolgt mithilfe des *FB_PositionError*. Dieser beinhaltet eine Normalisierung der Winkeldifferenz und wird auch im Regelkreis für die Berechnung der Regeldifferenz genutzt. Daher wird der Aufbau dieses Funktionsbausteins in Kapitel 5.2.1 erläutert.

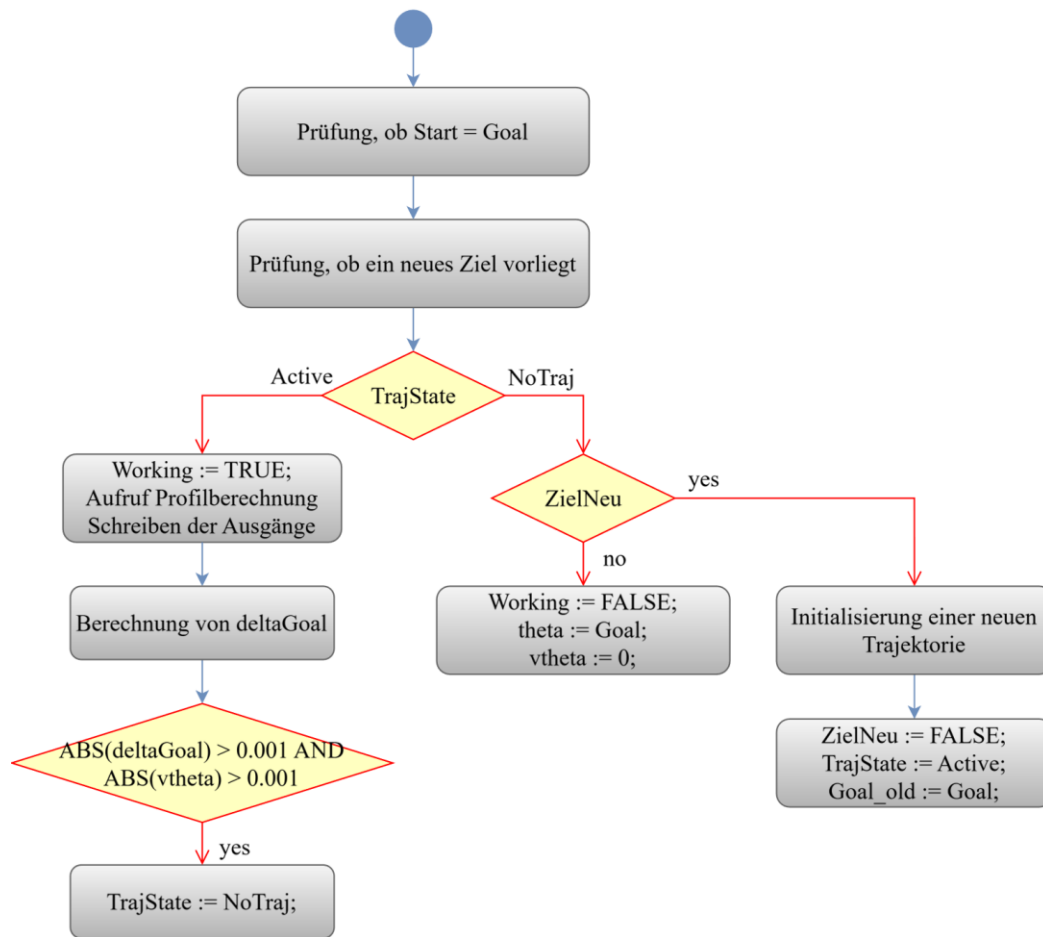


Abbildung 29 Vereinfachtes Ablaufdiagramm des FB_TrajectoryOneAxis

Anschließend an diese Prüfungen folgt die Case-Struktur, die auf der Variable *TrajState* basiert. Ist der State *Active*, wird zunächst die Variable *Working*, welche für die Auswahl des Reglermodus genutzt wird (siehe Abbildung 28), auf *TRUE* gesetzt. Darauf folgt der Aufruf des Bausteins, der die Trajektorie berechnet. Im nächsten Schritt werden die berechneten Werte auf die zugehörigen Ausgänge des Bausteins geschrieben. Im Weiteren wird erneut eine Differenz gebildet, um zu bestimmen, wann die Berechnung der Trajektorie abgeschlossen ist und den Wechsel zum State *NoTraj* durchzuführen. Der State *NoTraj* wird also aufgerufen, wenn keine Trajektorie berechnet wird und auf ein neues Ziel gewartet wird. Daher wird in dem State zunächst geprüft, ob das Flag für ein neues Ziel *TRUE* ist. Ist dies der Fall, wird der Baustein für die Berechnung der Trajektorie mit den neuen Werten für Start und Ziel initialisiert. Dabei werden auch die Werte für die maximale Geschwindigkeit 1 rad/s und Beschleunigung 0.4 rad/s² übergeben. Darauffolgend wird der State zu *Active* gewechselt und der Variable *Goal_old*, welche das alte Ziel speichert, mit dem neuen Ziel beschrieben. Ist *ZielNeu* *FALSE*, wird *Working* auf *FALSE* gesetzt. Des Weiteren werden die Ausgänge des Bausteins mit dem Zielwert beschrieben, da dieser bereits erreicht ist, wenn keine Trajektorie berechnet wird und dem Wert null für die Geschwindigkeit. Durch diese Struktur wird sichergestellt, dass Trajektorien nur bei tatsächlicher Zieländerung berechnet werden.

Nachfolgend wird der Ablauf der Trajektorienberechnung im Funktionsbaustein FB_CalculateProfileOneAxis ebenfalls mithilfe eines vereinfachten Ablaufdiagramms erläutert. Abbildung 30 zeigt den sequenziellen Ablauf der Zustandsübergänge dieses Funktionsbausteins und die Bedingung für die Initialisierung der Berechnung neuer Bewegungsprofile.

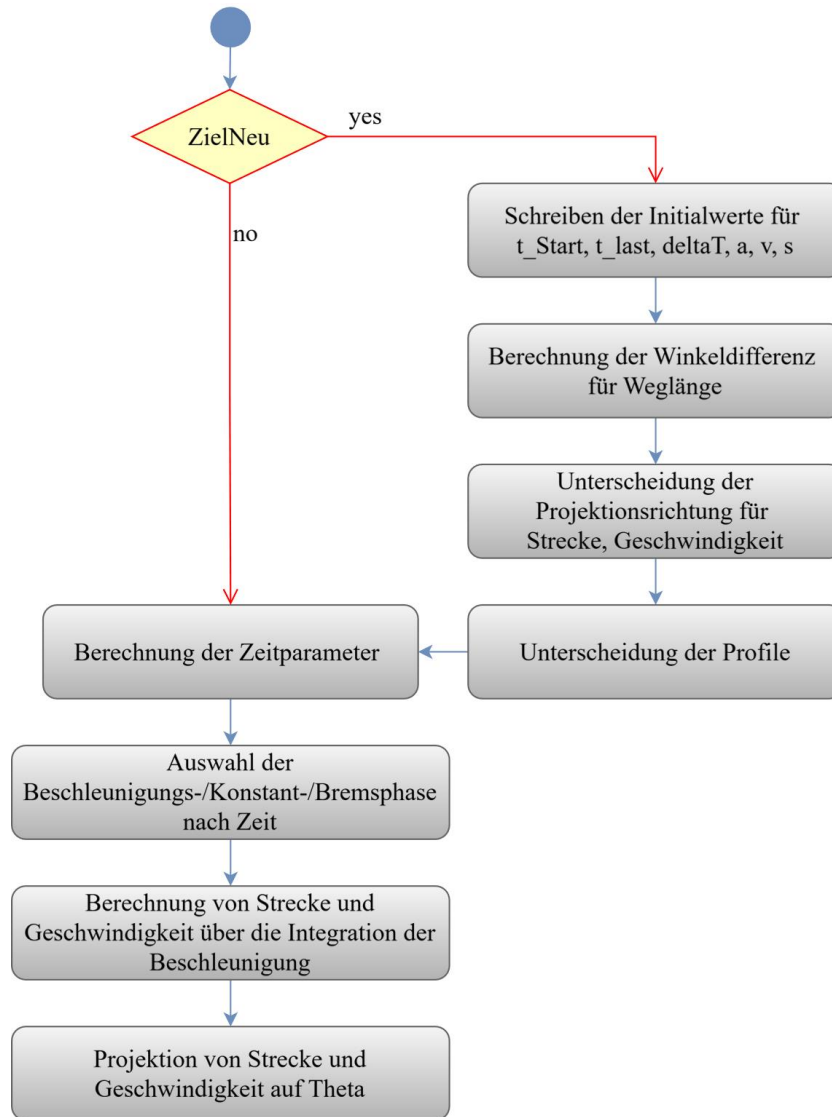


Abbildung 30 Vereinfachtes Ablaufdiagramm des FB_CalculateProfileOneAxis

Hier wird zunächst geprüft, ob das Flag für ein neues Ziel gesetzt ist. Ist dies der Fall, werden die Initialwerte der benötigten Variablen geschrieben. Für die Zeitparameter bedeutet das, dass t_Start und t_last auf die Zeit des Initialisierungsmoments und $deltaT$ auf 0.0 s gesetzt werden. Die Variable t_Start wird benötigt, um berechnen zu können, was die aktuell vergangene Zeit in der Trajektorienberechnung ist. Um das $deltaT$ zu berechnen, welches für die spätere Integration benötigt wird, wird t_last benötigt. Denn der Wert von t_last wird von der realen Zeit abgezogen, um die Zeitdifferenz zum letzten Berechnungsschritt ($deltaT$) zu erhalten. Die Variablen a (Beschleunigung), v (Geschwindigkeit) und s (Strecke) werden zu Beginn jeweils auf den Wert 0.0 gesetzt.

Nachfolgend wird die Winkeldifferenz zwischen dem alten und dem neuen Ziel berechnet. Dafür wird auch hier eine Instanz des *FB_PositionError* genutzt. Diese Differenz ist die Weglänge für die Drehung. Für die Drehrichtung wird geprüft, ob die Winkeldifferenz größer/gleich 0.0 rad ist. Ist dies der Fall, so ist die Projektionsrichtung für die Strecke positiv (1.0) und für die Geschwindigkeit negativ (-1.0). Ist die Differenz kleiner als 0.0 rad, wird die Projektionsrichtung für die Strecke negativ und für die Geschwindigkeit positiv. Das Vorzeichen der Geschwindigkeit ist folglich gegenteilig zu dem der Strecke zu wählen.

Wird dies nicht so gewählt, hat sich in den Tests gezeigt, dass die Geschwindigkeit in der Vorsteuerung gegen den Regler arbeitet. Die Vorsteuerung wirkt dann als Störgröße, die vom Regler ausgeglichen werden muss.

Nach dieser Unterscheidung werden die Profile unterschieden. Dabei erfolgt die Auswahl und Berechnung der Parameter wie bereits in Tabelle 8 beschrieben.

Gibt es kein neues Ziel oder ist die Initialisierung abgeschlossen, werden die Zeitparameter berechnet. Dabei wird Δt wie oben bereits beschrieben aus der Differenz der aktuellen Zeit und t_{last} berechnet. Für die absolute Zeit seit Beginn der Trajektorienberechnung wird die Differenz zwischen der aktuellen und der Startzeit berechnet. Nachfolgend wird die aktuelle Zeit auf t_{last} geschrieben, um diese im nächsten Zyklus für die Berechnung von Δt nutzen zu können.

Abbildung 31 zeigt die Auswahl der Beschleunigungen nach den in der Profilauswahl berechneten Zeitparametern. Diese Beschleunigungen werden darauffolgend nach den Formeln (16) und (17) integriert.

```
// Beschleunigungs-/Konstant-/Bremsphase nach Zeit
IF t0 <= t_a THEN
    a := a_max;
ELSIF t0 < (t_end - t_a) THEN
    a := 0.0;
ELSIF t0 <= t_end THEN
    a := -a_max;
ELSE
    a := 0.0;
    v := 0.0;
    s_t := Way;
END_IF;
```

Abbildung 31 Code zur Auswahl der Beschleunigung

Die Projektion der berechneten Strecke erfolgt durch die Addition der berechneten Strecke multipliziert mit der Projektionsrichtung auf den Startpunkt. Für die Geschwindigkeit wird lediglich die berechnete Geschwindigkeit mit der Projektionsrichtung multipliziert.

Die berechneten Profile für eine Drehung eines HAWinos sind in Abbildung 32 zu sehen. Es ist eine Drehung von π rad beziehungsweise $-\pi$ rad auf 0 rad aufgezeichnet worden. Das Geschwindigkeitsprofil ist für diese Weglänge trapezförmig. Die linke y-Achse erfasst den Winkel und die rechte y-Achse gehört zu der Aufzeichnung der Geschwindigkeit. Die x-Achse bildet die Zeit der Messung ab.

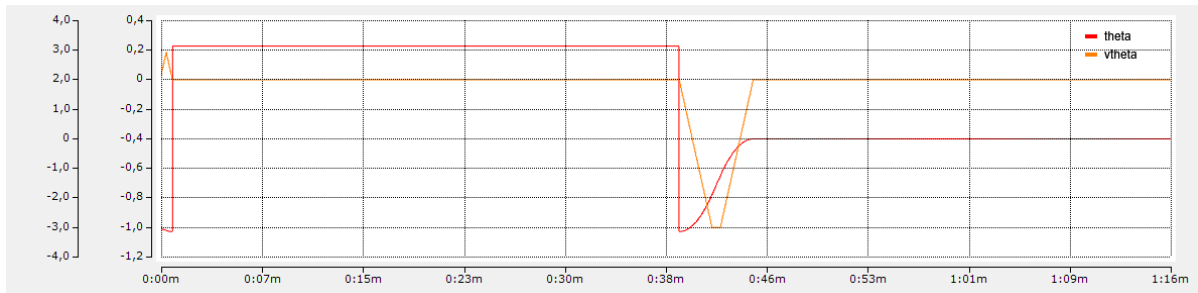


Abbildung 32 Berechnete Trajektorien für eine Drehung in θ

4.2.3 Berechnung der Trajektorie für Fahrten in x- und y-Richtung

Die Trajektoriengenerierung in x- und y-Richtung erfolgt ebenfalls mithilfe von zwei Funktionsbausteinen. *FB_TrajectoryTwoAxis* umfasst den Ablauf der Generierung und ruft den Funktionsbaustein *FB_CalculateProfileTwoAxis* für die Berechnung der Trajektorien auf.

Der Ablauf der Generierung der Trajektorien für Fahrten in x- und y-Richtung folgt grundsätzlich dem in Abbildung 29 gezeigten Vorgehen. Es wird ebenfalls zunächst geprüft, ob ein neues Ziel vorliegt. In diesem Fall wird zudem erfasst und auf einem Merker abgelegt, auf welcher Achse der Koordinatenwechsel erfolgt ist. Dabei wird zudem die Situation abgefangen, dass Änderungen auf beiden Achsen gleichzeitig auftreten. Ist dies der Fall, wird das Änderungs-Flag für die Achse gesetzt, auf der die größere Änderung erfolgt.

Um eine übersichtliche Betrachtung der Zustände des Zustandsautomaten zu ermöglichen, werden diese nachfolgend einzeln erläutert. Abbildung 33 zeigt den Zustand *NoTraj*. Dieser ist bereits aus der Trajektoriengenerierung für die Drehung in θ bekannt.

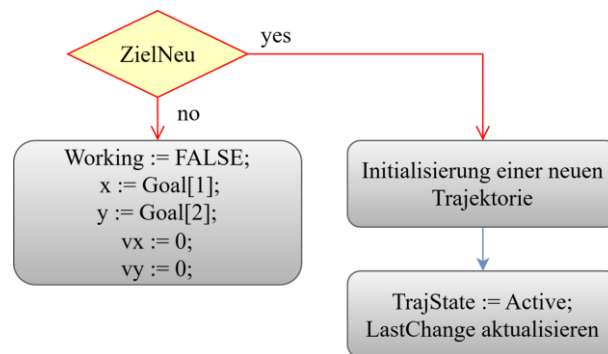


Abbildung 33 Vereinfachtes Ablaufdiagramm des States *NoTraj* im *FB_TrajectoryTwoAxis*

Für die Betrachtung von x und y wird in diesem Fall ergänzt, dass wenn kein neues Ziel vorliegt, die Werte für beide Koordinaten (x, y) auf die Zielwerte und die zugehörigen Geschwindigkeiten auf null gesetzt werden. Liegt ein neues Ziel vor, muss nicht mehr nur eine neue Trajektorie initialisiert, sondern zusätzlich ein Merker gesetzt werden, um zu speichern, auf welcher Achse der Koordinatenwechsel

erfolgt ist. Dafür wird die Variable *LastChange* verwendet. Neue Trajektorien werden mit einer maximalen Beschleunigung von 80 mm/s² und einer maximalen Geschwindigkeit von 400 mm/s initialisiert. Danach erfolgt wie bei θ der Wechsel in den Zustand *Active*.

Durch die Betrachtung beider Achsen (x, y) und das Überschleifen der Trajektorien erweitert sich der Zustand *Active* gegenüber dem für θ um einige Abfragen. Ein vereinfachtes sequenzielles Ablaufdiagramm ist in Abbildung 34 zu sehen. Neben der Berechnung der aktuellen Trajektorie wird das *ZielNeu*-Flag geprüft. Gibt es kein neues Ziel, wird überprüft, ob zusätzlich die berechneten Werte für x und y den vorgegebenen Zielwerten entsprechen. Ist dies der Fall, kann wieder in den State *NoTraj* gewechselt werden.

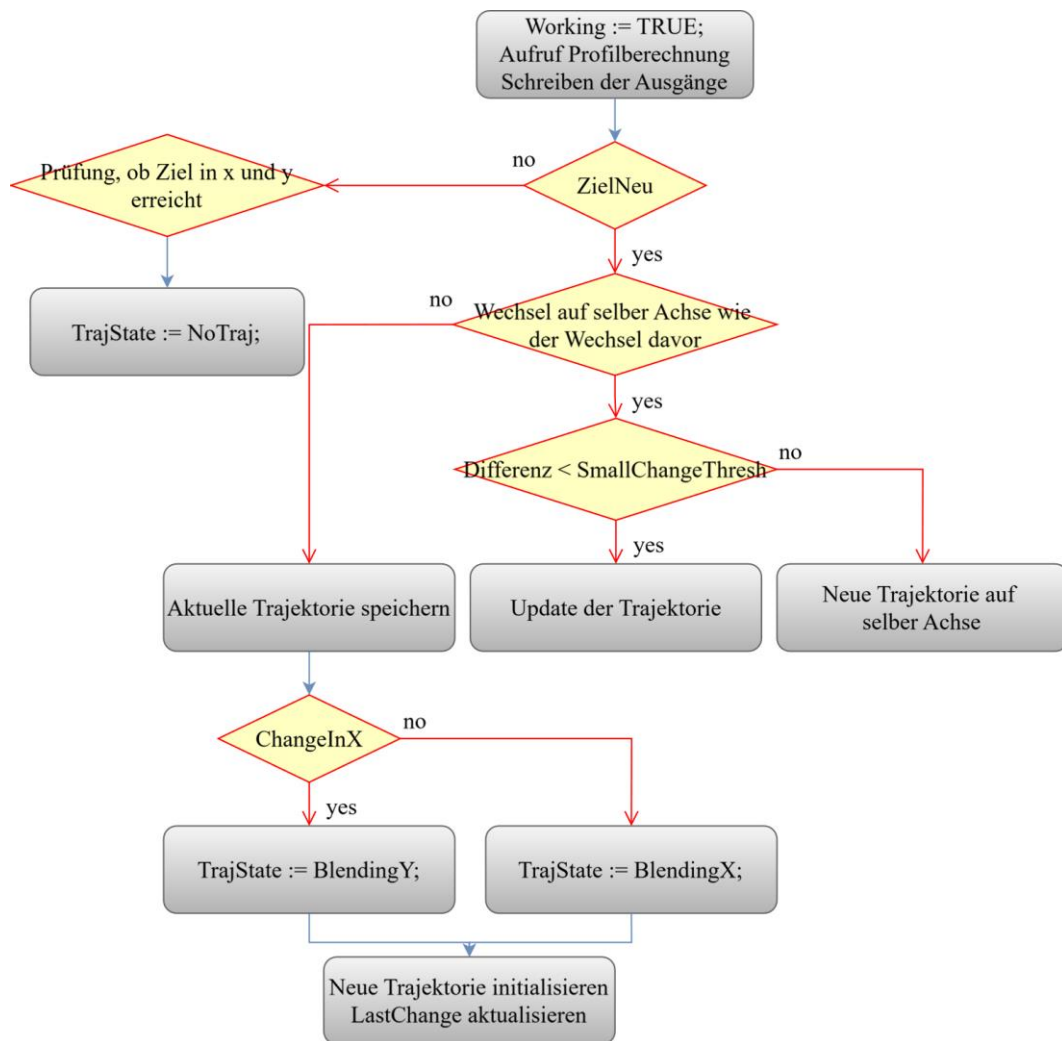


Abbildung 34 Vereinfachtes Ablaufdiagramm des States *Active* im FB *TrajectoryTwoAxis*

Ist das *ZielNeu*-Flag gesetzt, erfolgen verschiedene Prüfungen. Zunächst wird geprüft, ob der Koordinatenwechsel auf derselben Achse erfolgt ist, wie der Wechsel zuvor. Ist dies der Fall, wird überprüft, ob lediglich ein kleiner Wechsel des Zielwerts vorliegt (< 104 mm). Trifft dies zu, wird die aktuelle Trajektorie mit dem neuen Zielwert aktualisiert. Ist die Änderung größer, wird eine neue Trajektorie

initialisiert. Bei der Initialisierung werden als Startwerte die letzten Werte für x und y aus der Berechnung der vorherigen Trajektorie übernommen. Als Zielwerte werden die neuen Zielkoordinaten verwendet.

Erfolgt der Koordinatenwechsel nicht auf derselben Achse wie zuvor, müssen die Trajektorien überschliffen werden. Dafür wird zunächst das aktuelle Profil gespeichert. Danach wird überprüft, ob die Variable *ChangeInX* TRUE, also der Wechsel in der x-Koordinate erfolgt ist oder nicht, um einen Zustandswechsel zu dem entsprechenden Blending-Zustand durchzuführen. Ist der Wert gesetzt, bedeutet dies, dass für x eine neue Trajektorie berechnet werden muss, während die Trajektorie für y vollständig abgearbeitet wird. Es wird in den Zustand *BlendingY* gewechselt. Ist der Wert FALSE, wird für y eine neue Trajektorie berechnet, während die für x weiterläuft, bis sie vollständig abgearbeitet ist (*BlendingX*). Nach dieser Zuweisung des neuen Zustands zu *TrajState*, wird die neue Trajektorie initialisiert und die Variable *LastChange* aktualisiert.

Das vereinfachte Ablaufdiagramm in Abbildung 35 verdeutlicht den Ablauf der Blending-Zustände anhand des Zustands *BlendingX*. Auch in diesem Zustand wird zunächst geprüft, ob das Flag für das Erkennen eines neuen Ziels aktiv ist. Ist dies der Fall, wird zurück in den Active-Zustand gewechselt. Ist kein neues Ziel vorhanden, werden zunächst beide Trajektorien berechnet. Danach wird der x-Koordinate sowie der zugehörigen Geschwindigkeit die entsprechenden Werte aus der Berechnung der alten Trajektorie zugewiesen. Der y-Koordinate und zugehörigen Geschwindigkeit in y-Richtung werden die jeweiligen Werte aus der Berechnung der neuen Trajektorie zugewiesen. Ist der x-Wert der neuen Trajektorie derselbe wie der der alten Trajektorie, kann zurück in den Zustand Active gewechselt werden. Es werden also die Strecken- und Geschwindigkeitsprofile beim Koordinatenwechsel superpositioniert, bis das alte Profil vollständig abgearbeitet ist.

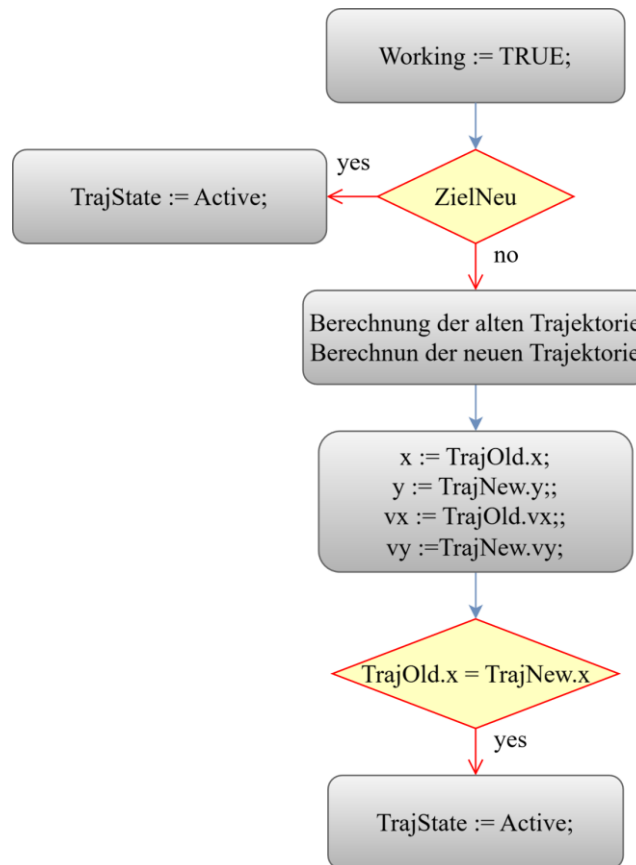


Abbildung 35 Vereinfachtes Ablaufdiagramm des States BlendingX

In der Berechnung der Trajektorien mit dem Funktionsbaustein FB_CalculateProfileTwoAxis gibt es lediglich einen Unterschied zur Berechnung des Profils für eine Achse. Der Unterschied besteht darin, dass zusätzlich zu dem ZielNeu-Flag auch das *UpdateTrajectory*-Flag überprüft wird. Der Ablauf dieser Ergänzung ist in Abbildung 36 dargestellt.

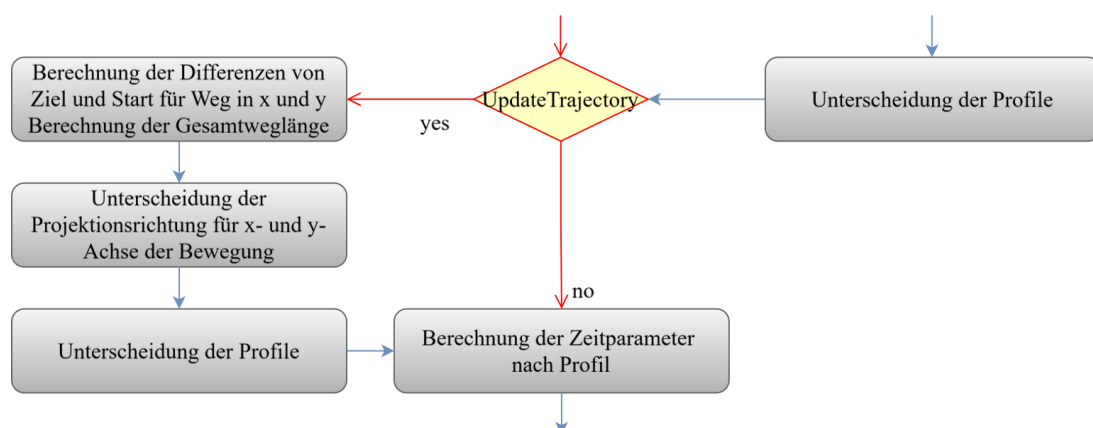


Abbildung 36 Ergänzung im FB_CalculateProfileTwoAxis zum FB_CalculateProfileOneAxis

Es wird abgefragt, ob das Flag für das Aktualisieren der Trajektorie gesetzt ist. Ist die Variable TRUE, wird die Berechnung der Gesamtlänge des Wegs erneuert und die Bestimmung der Projektionsrichtung

sowie die Profilunterscheidung inklusive der Berechnung der zugehörigen Parameter erneut durchgeführt. Im Anschluss läuft die Berechnung wie bereits für den *FB_CalculateProfileOneAxis* beschrieben weiter.

Abbildung 37 zeigt die berechneten Weg- und Geschwindigkeitsprofile, aufgezeichnet über die Zeit, für einen Fahrauftrag eines HAWinos. Die links liegende y-Achse erfasst jeweils die Koordinaten, während die rechte y-Achse die Geschwindigkeitswerte darstellt. Auch das Überschleifen der Trajektorien ist hier zu erkennen. Die zugehörigen Abschnitte sind jeweils durch einen grünen Kasten eingerahmt.

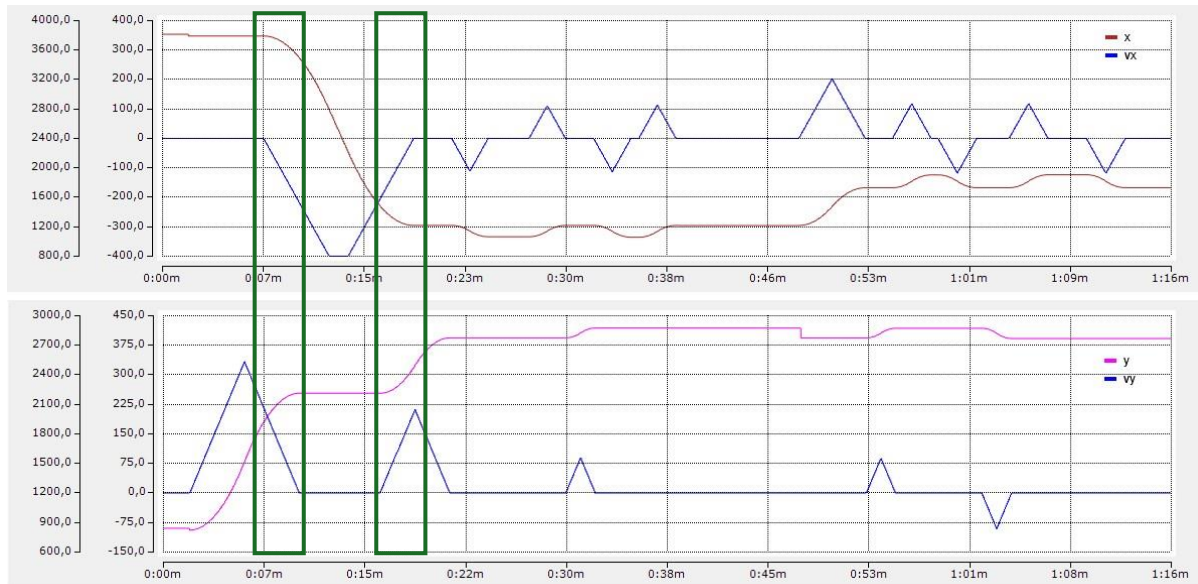


Abbildung 37 Berechnete Weg- und Geschwindigkeitsprofile für einen Fahrauftrag eines HAWinos

5 Bahnregelung

Dieses Kapitel befasst sich mit der Bahnregelung für die HAWinos. Der gesamte Regelkreis wird in dem Programm *Main_MotorTask* ausgeführt. Dieses Programm ist dem *MotorTask* zugeordnet. Dieser hat eine Zykluszeit von 0.333 ms und die Priorität 1. Dies ist so gewählt worden, da die Ansteuerung der Antriebe der wichtigste Task ist.

Zunächst wird das gewählte Konzept und dessen Simulation betrachtet. Im Weiteren wird die Umsetzung von Positions- und Geschwindigkeitsregelung erläutert.

5.1 Konzeption und Simulation

Um eine möglichst stetige, ruhige Fahrt der HAWinos zu erreichen, ist es notwendig, robuste und genaue Regelalgorithmen zu wählen. Nachdem die Trajektoriengenerierung aus den diskreten Eckpunkten der Bahnplanung zeitparametrisierte Sollwerte für den Regelkreis berechnet, ist es das Ziel der Bahnregelung diese Sollwerte möglichst genau und ruhig abzufahren. Dafür gilt es, die jeweils drei Antriebe der Roboter zu regeln.

Ein verbreiteter Ansatz ist, den Regelkreis als Kaskadenregelung aufzubauen. Der Ansatz ist in Abbildung 38 dargestellt.

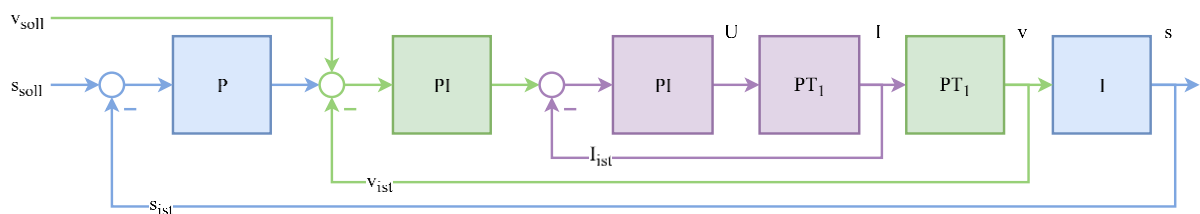


Abbildung 38 Konzept für den Regelkreis

Der äußere, blaue Regelkreis ist der Positionsregelkreis. Diesem wird die Sollposition aus der Trajektorie zugeführt. Als Regler wird hier ein P-Regler verwendet. Durch den P-Regler ergibt sich insgesamt ein Verhalten ähnlich einem I-Glied. Der grüne Abschnitt ist die Geschwindigkeitsregelung. Das Konzept sieht vor einen PI-Regler als Geschwindigkeitsregler zu nutzen. Der Sollwert für den Geschwindigkeitsregler bildet sich aus dem Ausgang des Positionsreglers zusammen mit der Vorsteuerung, die sich aus dem in der Trajektoriengenerierung berechneten Geschwindigkeitsprofil ergibt. Das Verhalten entspricht für diesen Part der Kaskadenregelung einem PT1-Glied. Der violette, innere Regelkreis ist der Stromregelkreis. Auch hier wird ein PI-Regler gewählt und das Verhalten entspricht dem eines PT1-Glieds.

Dieses Konzept ist ebenso wie die Trajektoriengenerierung zunächst in MATLAB Simulink umgesetzt worden. Wobei einer der wichtigsten Punkte das Simulieren der verschiedenen Koordinatentransformationen gewesen ist. Es gibt in diesem Projekt drei Koordinatensysteme, die im Verlauf der Kaskadenre-

gelung ineinander umgerechnet werden müssen. Die Positionsregelung erfolgt im Weltkoordinatensystem. Für die Geschwindigkeitsregelung muss zunächst eine Umrechnung auf das jeweilige Roboterkoordinatensystem und dann eine weitere Umrechnung auf die Antriebe der einzelnen Räder des Roboters erfolgen. Auf diese Umrechnungen wird im nachfolgenden Kapitel eingegangen. Im Anschluss an diese Erläuterungen werden die Lage- und Geschwindigkeitsregelung betrachtet.

5.2 Positionsregelung

Wie bereits in der Konzeption beschrieben erfolgt die Positionsregelung mithilfe eines P-Reglers. Dabei wird für x , y und θ jeweils ein eigener KP-Wert für die Regelung gewählt. Im Verlauf des Projekts hat sich in der Testphase gezeigt, dass für eine ruhigere Fahrt ein Gain-Scheduling sinnvoll ist. Dieses wird im nachfolgenden Kapitel 5.2.2 erläutert. Da das Gain-Scheduling in den Regelalgorithmus integriert ist, wird dieser ebenfalls ausführlich betrachtet.

5.2.1 Berechnung der Regeldifferenz

Ein wichtiger Punkt für die Positionsregelung ist die Berechnung der Regeldifferenz. Denn für die Differenz in θ muss der Winkel normalisiert werden. Die Berechnung der Regeldifferenz beziehungsweise der Positionsdifferenz erfolgt im Funktionsbaustein *FB_PositionError*. Der Code des Bausteins ist in Abbildung 39 zu sehen.

```
// Differenzen der Positionen
errorX := targetPosX - currentPosX;
errorY := targetPosY - currentPosY;
// Winkeldifferenz Rohwert
Delta := targetPosTheta - currentPosTheta;
// Normalisieren der Winkeldifferenz [-pi, pi]
WHILE Delta > Pi DO
    Delta := Delta - 2.0 * Pi;
END_WHILE;

WHILE Delta < -Pi DO
    Delta := Delta + 2.0 * Pi;
END_WHILE;
errorTheta := Delta;
```

Abbildung 39 Code des *FB_PositionError*

Es ist zu erkennen, dass für die x - und y -Koordinaten die Differenzen durch einfache Subtraktion gebildet werden. Für θ wird die berechnete Differenz allerdings auf den Wertebereich $[-\pi, \pi]$ normalisiert. Dies ist wichtig, damit die Differenzen immer dem effizientesten Drehweg entsprechen. Steht der Roboter beispielsweise auf π rad und soll auf -2 rad drehen, wäre die effizienteste Drehung circa $+1.14$ rad. Allerdings würde die normale Differenzberechnung in einer Drehung von ungefähr -5.14 rad resultieren. Dieser Weg ist deutlich länger und ineffizienter. Das verdeutlicht die Notwendigkeit der Normalisierung.

5.2.2 Regelalgorithmus mit Gain-Scheduling

Aus der Trajektorie ergeben sich die Reglermodi *TURNING*, *GLOBAL*, *TOCHARGING* und *NONE*. Im Positionsregler werden zusätzliche Unterscheidungen für das Gain-Scheduling gemacht. Der gesamte Regelalgorithmus ist in Abbildung 40 dargestellt.

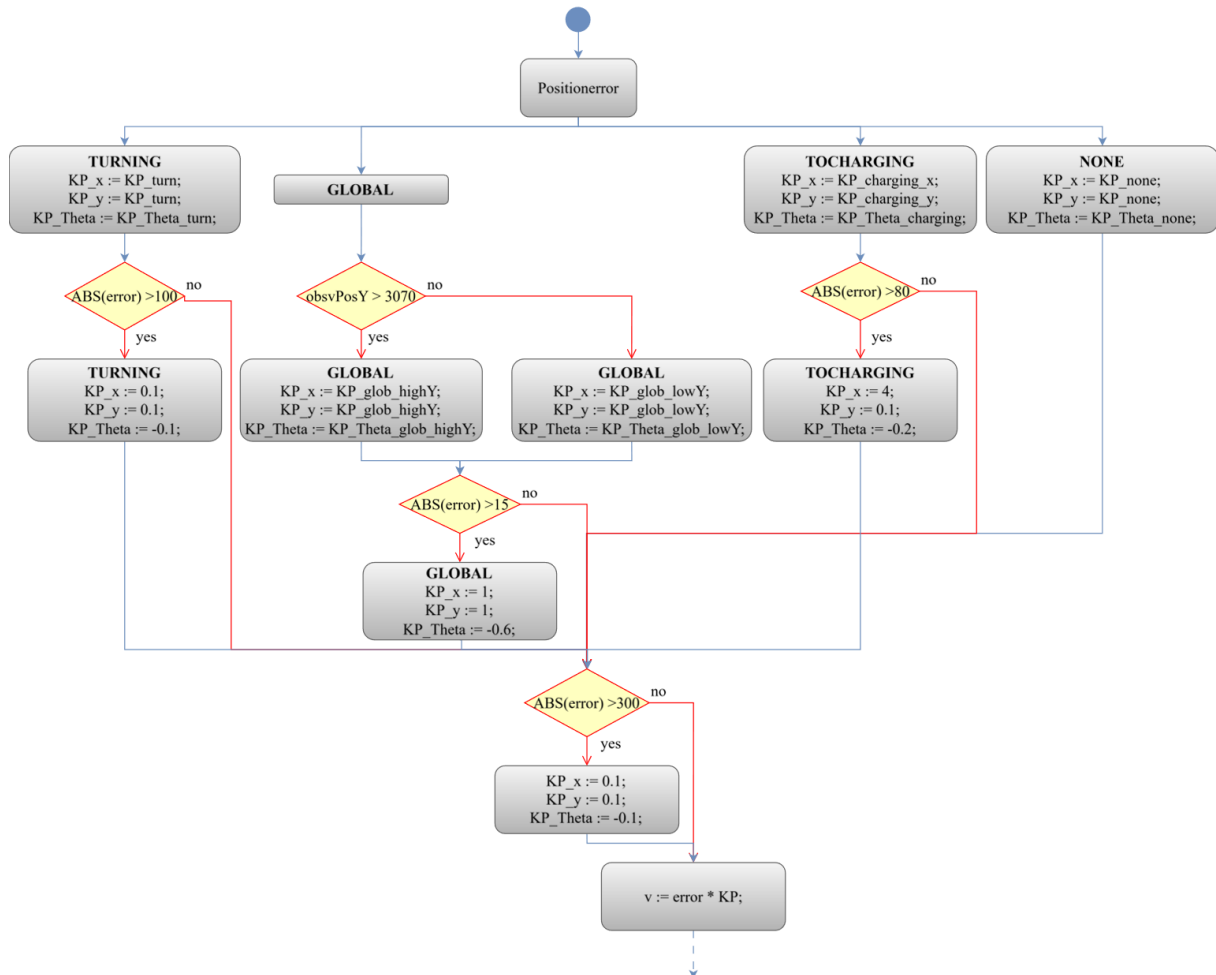


Abbildung 40 Gain-Scheduling im Positionsregler

Es ist zu erkennen, dass nach der Berechnung der Regeldifferenz die Case-Struktur mit der Aufteilung nach den Reglermodi aus der Trajektoriengenerierung erfolgt. Im Modus *GLOBAL* wird zusätzlich unterschieden, in welchem y-Koordinatenbereich des Raums sich die HAWinos befinden. In der Testphase hat sich gezeigt, dass es sinnvoll ist, bei einer globalen Fahrt zu unterscheiden, ob sich die Roboter in der Nähe der Wand befinden, also unterhalb der Schienen fahren, die von der Wand zu den Stationen führen. Bei der Verwendung der ursprünglich gewählten KP-Werte für die Globalfahrten ist es wiederholt zu ruckeligen Fahrten gekommen, wenn die Kamera die Roboter zu lange nicht erkennt. Denn dann wird eine starke Reaktion des Positionsreglers gefordert, die allerdings revidiert wird, sobald der Roboter beim Verlassen des kritischen Bereichs unterhalb der Schienen wieder erkannt wird. Daher werden für den Koordinatenbereich $y > 3070$ mm kleinere KP-Werte genutzt.

Zudem wird in jedem Reglermodus geprüft, ob der Fehler einen Grenzwert überschreitet, da auch dies auf ein längeres Nichterkennen der HAWinos hindeuten kann. So wird auch in diesem Fall keine zu hohe Reaktion des Positionsreglers gefordert und die Roboter fahren ruhige Fahrten.

Auch im gesamten Algorithmus wird überprüft, ob die Regelabweichung zu groß wird (> 300 mm). In dem Fall werden die KP-Werte alle auf 0.1 gesetzt, um die Geschwindigkeit des Roboters zu reduzieren. Dauert dieser Zustand zu lange an, kann aus diesem Modus der Fehler *TargetNotReached* entstehen. Dadurch würde eine neue Bahn geplant werden und der Roboter wieder mit der gewohnten Geschwindigkeit die geplanten Bahnen abfahren.

5.3 Geschwindigkeitsregelung

Die Geschwindigkeitsregelung folgt in dem Kaskadenregelkreis auf die Positionsregelung. Dieses Kapitel befasst sich zunächst mit der Umsetzung der Vorsteuerung für die Geschwindigkeitsregelung. Darauf folgend werden die Koordinatentransformationen erläutert, die bereits im Kapitel der Konzeption erwähnt worden sind. Abschließend wird der eingesetzte Regelalgorithmus erläutert.

5.3.1 Vorsteuerung

Bei der Vorsteuerung wird in diesem Fall zur Stellgröße des Positionsreglers der zum jeweiligen Zeitpunkt gültige Geschwindigkeitswert aus der Trajektoriengenerierung addiert, um den Sollwertverlauf der Geschwindigkeit bereits zu berücksichtigen.

Die Vorsteuerung wird deaktiviert, wenn der Regelmodus TOCHARGING verwendet wird. Dieses Vorgehen hat sich in der abschließenden Testphase als zielführend erwiesen, um eine ruhige und genaue Anfahrt an die Ladestationen zu ermöglichen. Eine weitere Ergänzung für einen möglichst ruhigen Sollwertverlauf in der Geschwindigkeitsregelung ist eine Tiefpassfilterung des vorgesteuerten Sollwerts.

5.3.2 Koordinatentransformationen

Die erste Koordinatentransformation erfolgt als Aufruf der Action *ACT_WorldToRobot* im Programm *Main_MotorTask*. Dieser Aufruf ist im Ablauf nach der Positionsregelung, der Addition der Vorsteuerung auf den Sollwert und der Tiefpassfilterung des Sollwerts für die Geschwindigkeitsregelung eingefügt. Das bedeutet, dass die Transformationen alle für Geschwindigkeiten erfolgen. Bei der ersten Transformation wird von dem Weltkoordinatensystem zum Roboterkoordinatensystem transformiert. Dafür erfolgt eine Drehung des Koordinatensystems mithilfe der aktuellen Orientierung des jeweiligen Roboters über eine sogenannte Rotationsmatrix. Diese wird auch verwendet, um am Ende des Regelkreises die Umrechnung zurück vom Roboterkoordinatensystem in das Weltkoordinatensystem durchzuführen. Die beiden Varianten der Rotationsmatrix sind in Tabelle 10 aufgeführt. Anstelle der in [2] verwendeten

Positionsvektoren werden Geschwindigkeitsvektoren betrachtet. Die Rotationsmatrizen bleiben unverändert, da Rotationen lineare Abbildungen des zugrunde liegenden Vektorraums sind und somit unabhängig davon wirken, ob ein Vektor eine Position oder eine Geschwindigkeit beschreibt.

Tabelle 10 Rotationsmatrizen für die Umrechnung der Koordinatensysteme [2]

Weltkoordinaten zu Roboterkoordinaten	Roboterkoordinaten zu Weltkoordinaten
$\underline{v}^R = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \underline{v}^W \quad (26)$	$\underline{v}^W = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \underline{v}^R \quad (27)$

Im Gegensatz zu der Umsetzung in MATLAB Simulink kann bei der Programmierung in ST nicht mit Matrizen gearbeitet werden, daher wird jede Zeilenmultiplikation der Matrix mit dem entsprechenden Geschwindigkeitsvektor einzeln programmiert.

Die nächste Koordinatentransformation, die Umrechnung der Geschwindigkeiten in Roboterkoordinaten auf die Geschwindigkeiten der einzelnen Antriebe, ist die inverse Kinematik Transformation (IKT). Diese wird ebenfalls als Action umgesetzt, ist allerdings Teil des *FB_SpeedController*, der den Regelalgorithmus für die Geschwindigkeitsregelung beinhaltet (siehe Kapitel 5.3.3). Die Transformation erfolgt dort nach dem Aufruf des Zustandsautomaten des Geschwindigkeitsreglers. Die Rückrechnung erfolgt mit der direkten Kinematik Transformation (DKT). Die verwendeten Formeln sind in Tabelle 11 zu finden.

Tabelle 11 Formeln der IKT und DKT [3]

Inverse Kinematik Transformation	Direkte Kinematik Transformation
$v_1 = -\frac{\sqrt{3}}{2} v_x + \frac{v_y}{2} + R\omega \quad (28)$	$v_x = \frac{v_3 - v_1}{\sqrt{3}} \quad (31)$
$v_2 = 0 - v_y + R\omega \quad (29)$	$v_y = \frac{v_1}{3} - \frac{2}{3} v_2 + \frac{v_3}{3} \quad (32)$
$v_3 = \frac{\sqrt{3}}{2} v_x + \frac{v_y}{2} + R\omega \quad (30)$	$\omega = \frac{v_1 + v_2 + v_3}{3R} \quad (33)$

Die Rücktransformationen liegen alle im Programm *Main_MotorTask* hinter dem Aufruf des *FB_SpeedController*-Bausteins. Sie sind ebenfalls als Actions umgesetzt.

5.3.3 Regelalgorithmus

Der Funktionsbaustein *FB_SpeedController* ist bereits bei der Übernahme des Projekts Bestandteil des Programms gewesen. Der grundsätzliche Ablauf des Bausteins ist nicht verändert worden. Es werden nachfolgend im Zuge der Erläuterung des gesamten Ablaufs vor allem die Änderungen am Algorithmus hervorgehoben.

Der Ablauf der Geschwindigkeitsregelung beginnt mit dem Aufruf des Zustandsautomats für die Ansteuerung und Freigabe der Antriebe durch den Geschwindigkeitsregler. Dieser Zustandsautomat ist

bereits Bestandteil des ursprünglich übernommenen Programms, daher werden hier lediglich die Änderungen erläutert sowie ein kurzer Einblick in die Funktionsweise gegeben.

Der Zustand *Stop* ist aus der Statemachine entfernt worden. Im gegebenen Programm wird dieser Zustand durch das Betätigen des Bumpers ausgelöst: Diese Abfrage des Bumpers ist allerdings im Robot-Controller integriert, sodass das Abschalten der Motoren über den Entzug des *Enable* durch den Robot-Controller gelöst wird. Es bleiben somit die Zustände *Init*, *Idle* und *Working*.

Der Zustand *Init* wird zu Beginn einmal aufgerufen und anschließend immer, wenn ein Motor Reset durchgeführt wird. Dort werden alle Ansteuerungen auf null gesetzt, die Freigabe für alle Motoren entzogen sowie die Integratoren des PI-Reglers zurückgesetzt, bevor in den Zustand *Idle* gewechselt wird. Im Zustand *Idle* werden alle Werte auf null gehalten. Ist im Geschwindigkeitsregler das *Enable* für die Motoren gesetzt wird der Zustand zu *Working* gewechselt. Dort werden die vom Regler ausgegebenen Geschwindigkeitswerte auf INT skaliert, bevor die Freigaben für alle Motoren erteilt werden.

Nachfolgend werden die Sollwerte invertiert und die Action für die IKT aufgerufen (*ACT_IKT*).

Im Anschluss an die Transformation der Geschwindigkeiten erfolgt eine Sollwertbegrenzung. Die bei der Übernahme des Projekts enthaltene Sollwertbegrenzung ist insofern abgeändert worden, dass eine skalierte Sollwertbegrenzung erfolgt. Erreicht ein Sollwert die maximale Geschwindigkeit von 900 mm/s, werden alle anderen Sollwerte mit dem Skalierungsfaktor nach (34) entsprechend angepasst.

$$scale = \frac{\text{maximale Geschwindigkeit}}{\text{größter Sollwert}} \quad (34)$$

Anschließend an die Sollwertbegrenzung werden die Regeldifferenzen berechnet und die Integrale zur Bildung der I-Anteile der Regelung ermittelt. Im nächsten Schritt werden pro Geschwindigkeit die P- und I-Anteile durch Addition zusammengeführt. Der P-Anteil ergibt sich aus der Multiplikation der jeweiligen Regeldifferenz mit dem KP-Wert, während der I-Anteil durch die Multiplikation des KI-Werts mit dem entsprechenden Integral berechnet wird. In diesem Fall besitzen alle Geschwindigkeiten denselben KP-Wert (1) und KI-Wert (220).

Nachfolgend werden gegebenenfalls die Integrale für die Berechnung der I-Anteile zurückgesetzt, wenn die Geschwindigkeitssollwerte oder die Regeldifferenzen betraglich kleiner als 0.02 werden. Dann ist der Stillstand der Roboter erreicht. Werden die Integrale in dem Zustand nicht zurückgesetzt, fangen die Motoren an zu brummen.

Die letzte Action, die im *FB_SpeedController* aufgerufen wird, ist eine Ansteuerungsbegrenzung für die Motoren. Auch hier erfolgt die Änderung zu einer skalierten Begrenzung für die geregelten Motorgeschwindigkeiten. Der Skalierungsfaktor wird dabei ebenfalls wie in (34) berechnet, allerdings handelt es sich hier nicht um den größten Sollwert, sondern die größte geregelte Geschwindigkeit, die auf die Motoren gegeben werden soll.

Wird eine Skalierung notwendig, werden die Integrale aller Motorregler eingefroren. Dies ist erforderlich, da in diesem Zustand keine weitere Reduzierung der Regeldifferenz durch die Regelung möglich ist, die Ansteuerung jedoch bereits begrenzt wird. Durch das gleichzeitige Einfrieren aller Integrale wird ein Anwachsen der I-Anteile unter gesättigten Bedingungen verhindert und sichergestellt, dass nach dem Wegfall der Begrenzung ein konsistentes und stabiles Regelverhalten ohne asymmetrische Integratorzustände zwischen den Motoren erreicht wird.

5.4 Abschließende Betrachtung der Bahnregelung

Das Ergebnis des Zusammenspiels der entwickelten Regelalgorithmen ist in Abbildung 41 zu sehen.

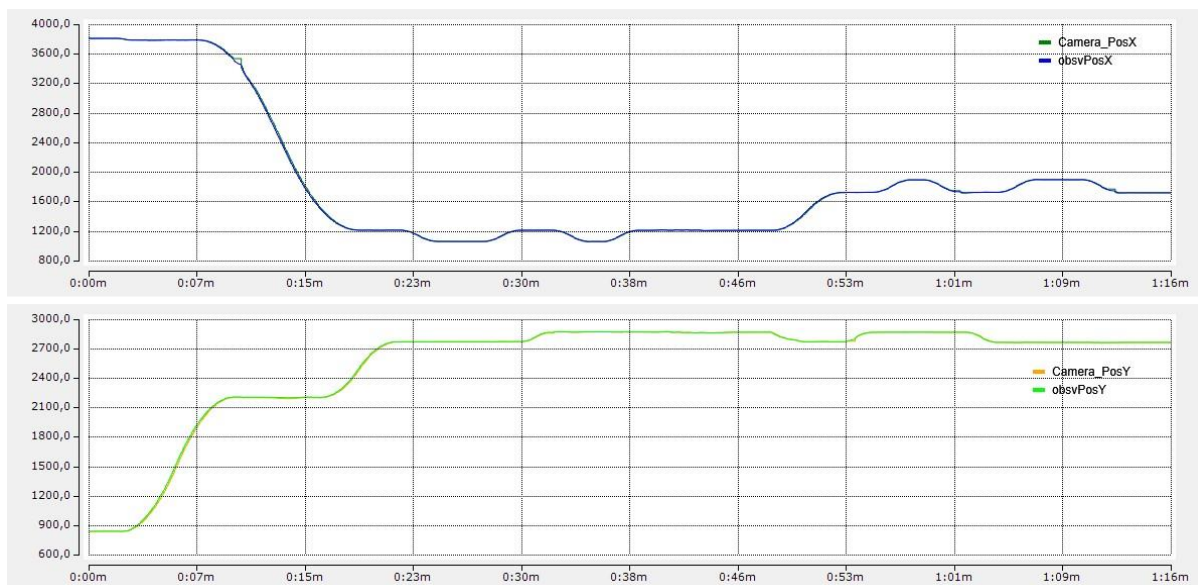


Abbildung 41 Fahrtverlauf in x- und y-Richtung

Die Abbildung zeigt den tatsächlichen Fahrtverlauf für die in Abbildung 37 gezeigten, berechneten Trajektorien in x- und y-Richtung. Dabei sind sowohl die Kamera- als auch die Beobachterpositionen über die Zeit aufgezeichnet worden. Insgesamt ist zu erkennen, dass der HAWino ruhig und stetig den vorgegebenen Trajektorien folgt.

Der zeitliche Verlauf einer Drehung in θ ist in Abbildung 42 zu sehen. Auf der y-Achse ist die Winkelstellung abzulesen. Auch hier ist im Zeitbereich von ca. 38 s bis 45 s ein ruhiger Verlauf der Drehung zu erkennen. Weitere Aspekte dieser Abbildung werden im Kapitel 6.6 betrachtet.

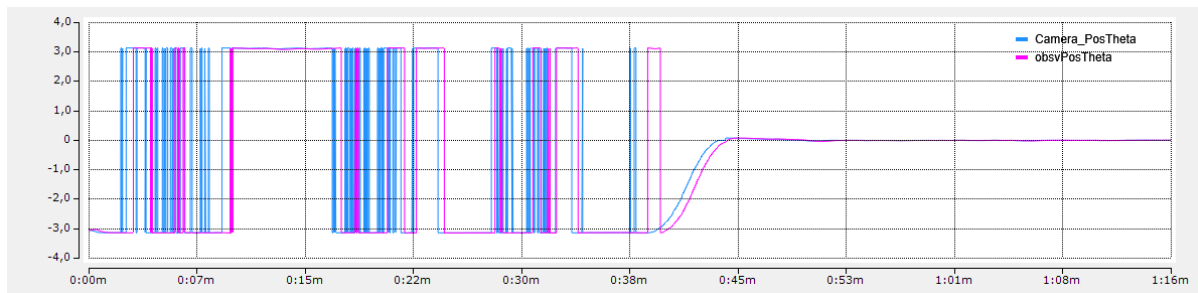


Abbildung 42 Verlauf einer Drehung in θ

Während des Projektverlaufs sind gegenseitige Optimierungsversuche unternommen worden, um die effizienteste Vorgehensweise im Code umzusetzen. Im Rahmen dieses Vier-Augen-Prinzips ist ein Optimierungsversuch des Regelkreises in Form eines PID-Bausteins, der mit Arrays arbeitet, entstanden. Dieser Ansatz ist aufgrund der komplexen Änderungen, die für die Umsetzung notwendig gewesen wären, verworfen worden. Die Erläuterung des Ansatzes ist in Anhang D zu finden.

Insgesamt erfüllt der gewählte Regelalgorithmus die Anforderungen an den Regelkreis. Die Roboter folgen den berechneten Trajektorien und auftretende Störungen werden beispielsweise in der Ermittlung der Kameraposition (vgl. Kapitel 6) werden erfolgreich ausgeglichen, sodass ruhige, stetige Fahrten erreicht werden.

6 Luenberger Beobachter

In diesem Kapitel wird der verwendete Luenberger-Beobachter erläutert. Anschließend wird näher auf dessen Konzeption, die Programmierung und die Optimierung eingegangen.

Der Luenberger-Beobachter [4] ist ein Verfahren zur Schätzung der Zustände eines Systems, wenn nicht alle Zustände direkt messbar sind. Er nutzt die Systemgleichungen und die Ausgangsmessungen, um die Zustände so zu rekonstruieren, dass der Schätzfehler minimal bleibt. Der verwendete Beobachter in der Simulation und der Programmierung besitzt kein Systemmodell, er arbeitet allein mit den Messwerten. Von diesen Messwerten ist einer totzeitbehaftet und stört den Regelkreis.

6.1 Konzeptionierung

Das Konzept des Beobachters wird auch hier wieder in der Simulation validiert. Dafür ist der Luenberger Beobachter in der Simulation des Regelkreises ergänzt worden. Erkennbar ist dies in der folgenden Abbildung 43.

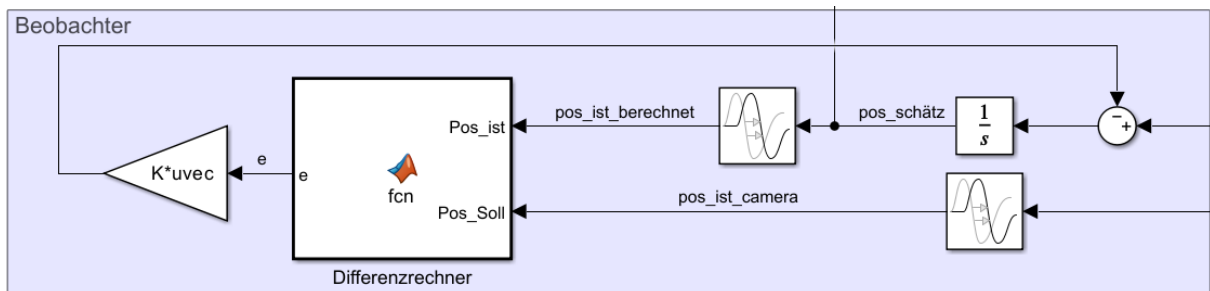


Abbildung 43 Luenberger in der Simulation

Über die Rückführung der Geschwindigkeitsinformation ist es möglich die Auswirkung der Totzeit an den Positionsdaten zu minimieren. Es ist im Vorfeld bekannt, dass die von der Kamera übertragenen Positionen eine Verzögerung besitzen. Die Simulation verwendet dafür eine künstliche und geschätzte Totzeit.

Über die Rückführung der Geschwindigkeit ist es möglich die neue Schätzposition zu berechnen. Verwendet werden hierfür die folgenden Formeln (35) und (36) .

$$\dot{\hat{\underline{x}}}(t) = \underline{v}(t) - \underline{L} \cdot \underline{e}(t) \quad (35)$$

$$\hat{\underline{x}}(t) = \int \dot{\hat{\underline{x}}}(t) dt \quad (36)$$

Dabei sei $\underline{v}(t)$ der aktuelle Wert der Geschwindigkeit, $\underline{L}$ der Proportionalitätsfaktor und $\underline{e}(t)$ der aktuelle Fehler des Beobachters. Aus diesen ergibt sich ein zu integrierender Geschwindigkeitsschätzwert, welcher die Schätzposition $\hat{\underline{x}}(t)$ als Ergebnis trägt. Die dargestellten Größen sind Spaltenvektoren und der Wert für $e_\theta(t)$ wird auf den gültigen Bereich normalisiert.

Der Fehler der Schätzung lässt sich mit der folgenden Formel (37) berechnen.

$$\underline{e}(t) = \underline{x}(t) - \hat{\underline{x}}(t) \quad (37)$$

Hier sei $\underline{x}(t)$ die Position aus der Kamera. Der berechnete Fehler $\underline{e}(t)$ wird wieder in die Schätzung zurückgeführt.

Das Ergebnis der Simulation findet sich in der folgenden Abbildung 44 und zeigt das Zusammenspiel aller Funktionseinheiten inklusive Luenberger Beobachter. Dargestellt ist eine Fahrt bestehend aus zwei Strecken, diese beinhalten eine Umschaltung in x- und y- Richtung.

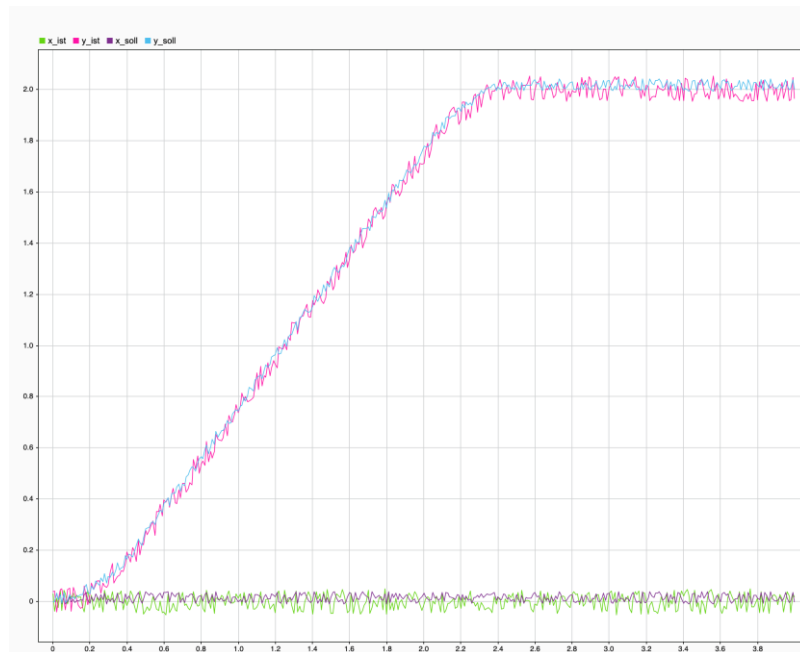


Abbildung 44 Eine Fahrt aus der Simulation

Zu erkennen ist auch der künstlich eingefügte Positionsfehler, welcher sich in einem Rauschen bemerkbar macht. Über die erfolgreiche Regelung der Fahrt kann ein erfolgreiches Implementieren des Beobachters angenommen werden. Die Ausrichtung θ wird von der Simulation nicht abgedeckt, diese ist konstant 0 rad.

6.2 Totzeitermittlung

Bedingt durch die Verarbeitung der Kameradaten in dem Laborrechner selbst und der Übertragung der extrahierten Position kommt es zu einer Totzeit, die sich negativ auf den Regelkreis auswirkt und unerwünschtes Störverhalten verursacht. Über den Luenberger Beobachter ist es möglich die Position des HAWinos mit einer Schätzung in den Regelkreis einzuführen und den Einfluss der Totzeit zu kompensieren.

Um den Beobachter korrekt parametrieren und die Regelgüte sicherstellen zu können, ist es notwendig die Totzeit des Systems zu ermitteln. Dieses Kapitel erläutert zunächst die Ergebnisse der dafür durchgeführten Kameraanalyse, bevor das Experiment für die Totzeitermittlung sowie dessen Ergebnisse dargestellt werden.

6.2.1 Kameraanalyse

Ziel der Kameraanalyse ist es, zu untersuchen, wie gut die Roboter an unterschiedlichen Positionen im Raum erkannt werden. Hierzu werden verschiedene Roboter von der Fensterseite des Raums in Richtung der Tür gefahren. Abbildung 45 und Abbildung 46 zeigen die Kameraaufzeichnungen der x-Position für Roboter 3 und Roboter 4 während einer Fahrt von einem x-Wert von ca. 100 mm bis ca. 7000 mm.

Abbildung 45 zeigt die Kameraaufzeichnung der x-Position (blau) und der y-Position (rosa) für Roboter 3. Dabei ist ersichtlich, dass der Roboter über den gesamten Fahrbereich zuverlässig erkannt wird. In Abbildung 46 ist hingegen die Kameraaufzeichnung der x-Position für Roboter 4 bei derselben Fahrt zu sehen. Hier wird deutlich, dass die Erkennungsqualität mit zunehmendem x-Wert abnimmt.

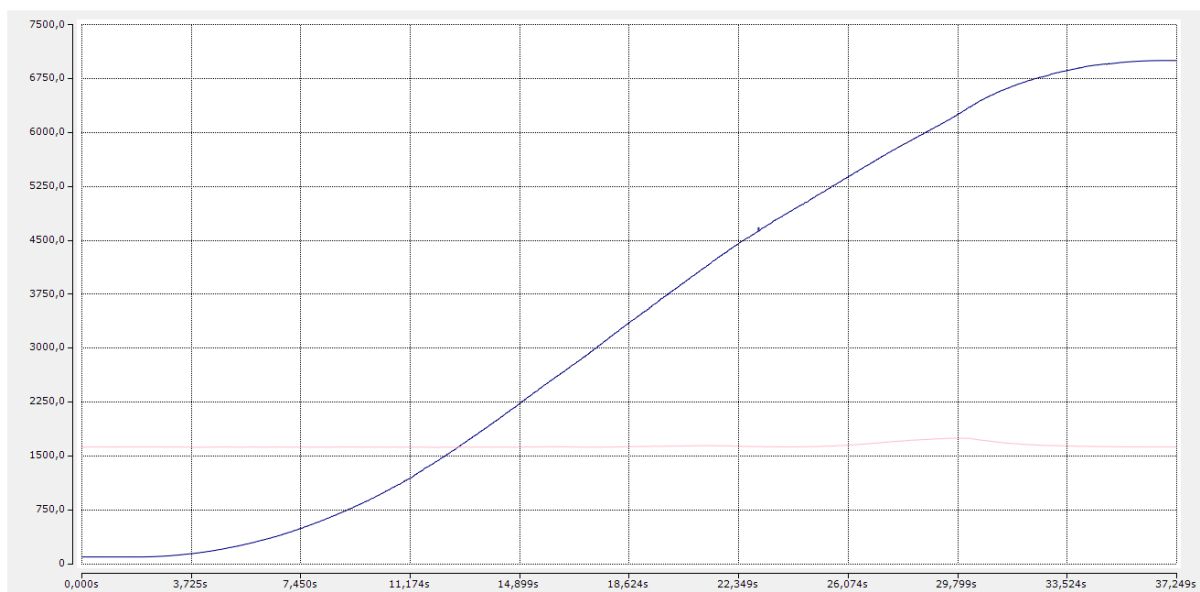


Abbildung 45 Kameraanalyse mit Roboter 3

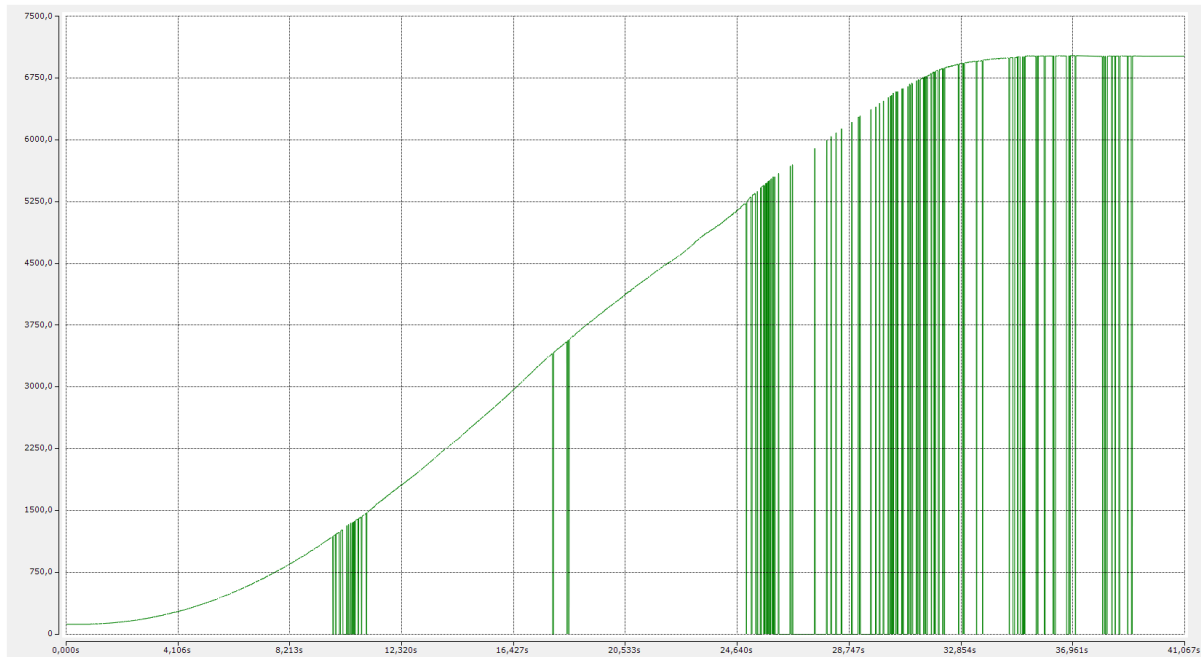


Abbildung 46 Kameraanalyse mit Roboter 4

Daraus lassen sich zwei Schlussfolgerungen ableiten. Zum einen werden nicht alle Roboter gleich gut erkannt, das bedeutet, dass das System nicht nur mit einem Roboter ausgelegt und getestet werden kann. Fortschritte und Änderungen müssen immer an mehreren Robotern getestet werden, um zu gewährleisten, dass die Entwicklungen für alle Roboter funktionieren. Zum anderen scheinen die Lichtverhältnisse im Bereich des Raums mit höheren x-Werten schlechter zu sein als im Bereich der mit kleinen x-Werten, der näher beziehungsweise an den Fenstern zum angrenzenden Laborraum liegt. Es wird vermutet, dass dieses Verhältnis aus dem durch die Fensterfront scheinenden Licht aus eben diesem Laborraum resultiert. Auf Grundlage dieser Erkenntnisse zur Erkennungsqualität wird im nächsten Schritt die Verzögerung der Kameramessung untersucht.

6.2.2 Ermittlung der Totzeit

Für die Ermittlung der Totzeit wird für die Geschwindigkeit in x-Richtung ein Sinus aufgeschaltet. Dieser hat eine Frequenz von 0.1 Hz, also einer Schwingung über 10 s und eine Höhe von 200 mm/s. Abbildung 47 zeigt die entsprechende Codezeile. Hier wird mit 0.0001 Hz multipliziert, um die Zeit zu skalieren, da TimeLREAL hier die Einheit ms hat.

```
vXCartesianToMotors_Vor := SIN(2*pi*0.0001*TimeLREAL)*200;
```

Abbildung 47 Sinus-Funktion für das Totzeitexperiment

Mit dieser Aufschaltung wird Geschwindigkeit und Position des Roboters aufgezeichnet. Die Messwerte werden als csv-Datei exportiert über ein Python-Skript aufgearbeitet und in MATLAB eingelesen. Dort werden die Messwerte weiter vorverarbeitet. Die Geschwindigkeitsmesswerte werden geglättet und die

Positionsmesswerte werden abgeleitet und ebenfalls geglättet. Ein Beispiel eines resultierenden Graphen ist in Abbildung 48 zu sehen.

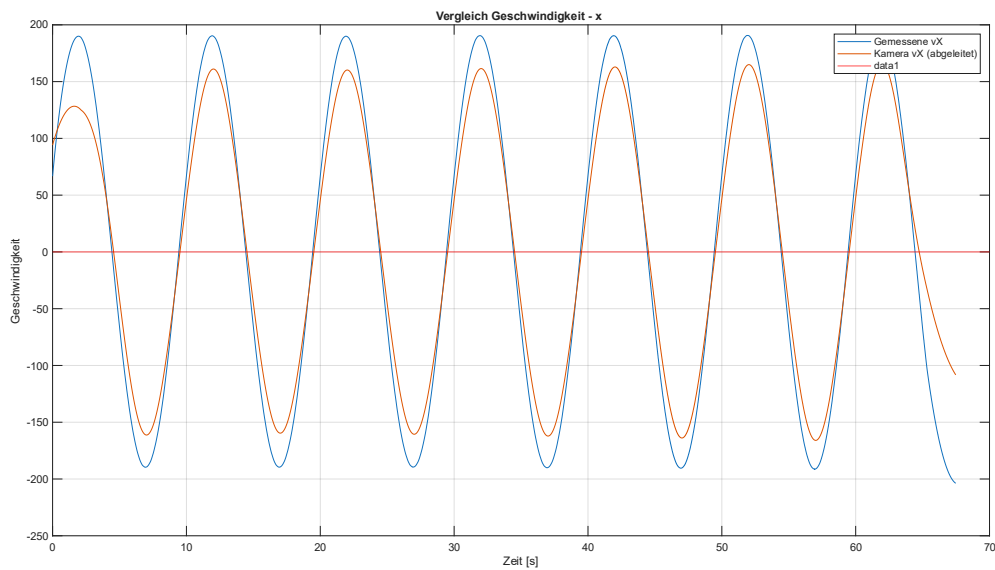


Abbildung 48 Messung für die Totzeitermittlung

Über den Zeitversatz an den Nulldurchgängen kann die Totzeit des Systems ermittelt werden. Im ersten Durchlauf dieses Experiments hat sich gezeigt, dass die Totzeit mit der Laufzeit des Programms driftet und größer wird. Daraufhin ist der Aufruf des Programms *Camera* mit in die *MotorTask* verschoben worden, um sicherzugehen, dass der Kamerabuffer immer rechtzeitig gelesen wird und nicht überläuft. Als Nachweis, dass diese Verschiebung das Problem behebt, ist für beide Konfigurationen das oben beschriebene Experiment 8 Minuten lang durchgeführt und die Totzeiten an den Nulldurchgängen aufgezeichnet worden.

Abbildung 49 zeigt die Totzeitentwicklungen im Vergleich. Diese Analyse zeigt, dass die Totzeit mit dem Programmaufruf der Kamera im PLCTask, also mit einer Priorität von 20 und einer Zykluszeit von 10 ms dafür sorgt, dass die Kameratotzeit innerhalb von 8 Minuten von ca. 220 ms auf ca. 720 ms ansteigt. Dasselbe Experiment mit dem Programmaufruf in der MotorTask zeigt, dass die Totzeit leicht schwankt, sich allerdings insgesamt um ca. 80 ms hält.

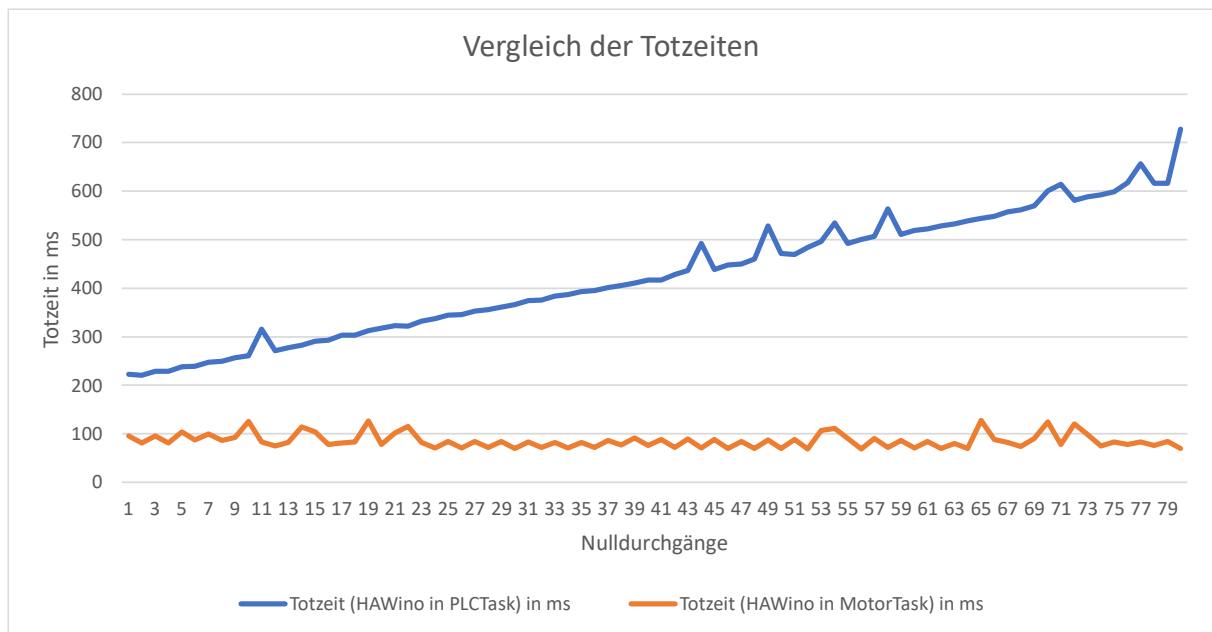


Abbildung 49 Vergleich der Totzeitentwicklungen bei Programmaufruf in verschiedenen Tasks

Auf Basis dieser Messung ist die beschriebene Taskkonfiguration für den gesamten Projektverlauf beibehalten worden.

Für die Totzeitmessung werden zehn Versuche durchgeführt. Dort werden jeweils für sechs Nulldurchgänge die Totzeiten ermittelt. Die insgesamt 60 ermittelten Totzeitwerte sind in Tabelle 12 aufgeführt.

Tabelle 12 Messergebnisse der Versuche für die Ermittlung der Kameratotzeit

Kameratotzeit in ms pro Versuch und Nulldurchgang									
1	2	3	4	5	6	7	8	9	10
76,6	77,5	73,1	77,4	83,8	87,6	81,9	85,4	100	81,4
62,7	74,5	76,6	81	89,8	81,9	86,9	92,9	90	78,2
62,10	90,5	74,9	81,9	85	83,7	81,4	85,3	99,4	83,8
57,5	117,7	114,1	82,4	82,7	133	87,6	94,4	88,6	76,1
117,9	75,1	86,5	83,5	84,5	83,3	130,2	127,2	97,8	79,8
63	74,3	78,5	84,1	81,2	79,4	88,7	102,2	137,7	81,2

Aus den ermittelten Totzeitmesswerten wird ein 95%-Konfidenzintervall für den Mittelwert der Kameratotzeit bestimmt. Der obere Grenzwert dieses Konfidenzintervalls beträgt 106 ms. Um eine robuste Auslegung des Regelkreises zu gewährleisten, wird dieser obere Grenzwert im Folgenden als konservativer Schätzwert für die Kameratotzeit verwendet.

6.3 Umsetzung in der Programmierung

Die Programmierung des Luenberger Beobachters erfolgt in einem Funktionsbaustein mit dem Namen *FB_LuenbergerObserver*. Übergeben werden diesem die Kamerapositionen und Geschwindigkeiten. Die übergebenen Werte werden zuerst in zwei verschiedene Arrays zusammengefasst, dadurch ist ein

späteres Verwenden in einer FOR-Schleife möglich. Die Schleifenstruktur ermöglicht zusätzlich eine Verringerung der Redundanz.

Aus der Kameraanalyse geht hervor, dass das Positionssignal an einigen Stellen keine Werte übermittelt. Stattdessen übergibt die Kamera bei einer Störung einen Nullwert in der x- und y-Achse, sowie in der Ausrichtung. Problematisch wird dies bei der Bestimmung des Schätzwerts, da die schlagartige Wertveränderung den Beobachter mit einem großen Fehler versieht. Dieser Fehler wird an den Regelkreis weitergegeben und es kommt zu Sprüngen in der Fortbewegung. Deutlich dramatischer ist die Auswirkung eines falschen Winkels, dieser wirkt sich stärker auf den Regler aus. Vor der Berechnung wird also über die in der folgenden Formel (38) dargestellten Logik geprüft, ob das Kamerasignal eine gültige Position liefert.

$$validCameraPos := (cameraPos[0] \neq 0.0 \wedge cameraPos[1] \neq 0.0 \wedge |cameraPos[2]| \leq \pi) \quad (38)$$

Hierbei seien *validCmeraPos* eine boolesche Variable zur Abfrage einer gültigen Position und die *cameraPos* ein Array bestehend aus den Positionskordinaten und der Ausrichtung.

In der Initialisierung des HAWinos und der erstmaligen Verwendung des Beobachters wird der Puffer mit dem Namen *tzBuffer* mit den aktuellen Positionswerten gefüllt. Ziel ist es den verwendeten Puffer mit gültigen Werten zu füllen, um aufkommende Schwierigkeiten mit dem Umgang von Nullwerten zu umgehen. Es handelt sich hierbei um eine 3×318 Matrix.

Der Beobachterfehler ermittelt sich über die vorgestellte Formel (37) unter Verwendung des bereits erläuterten Funktionsblocks *FB_PositionError* aus Kapitel 5.2.1. Das Ergebnis dieser Berechnung wird dem Array *error* zugewiesen dabei sind die Winkel stets normiert. Die Programmierung zeigt sich in der folgenden Abbildung 50.

```
// correction
FOR i := 0 TO 2 DO
  IF NOT validCameraPos THEN
    //correction[i] := L[i] * (tzBuffer[i, GlobalConst.Tz - 1] - tzBuffer[i, GlobalConst.Tz - 2]);
    correction[i] := L[i] * 0.0;
  ELSE
    correction[i] := L[i] * error[i];
  END_IF;
END_FOR
```

Abbildung 50 Berechnung der Korrektur

Hier zeigt sich der Vorteil einer Programmierung in Schleifenstruktur. Auch erkennbar ist ein Teil der bereits erklärten Formel (35) zur Bestimmung der Korrektur. Ist der Kamerawert nicht gültig, so wird in der finalen Lösung die Rückführung der ungültigen Fehlerberechnung abgeschnitten. Dies geschieht über eine Multiplikation mit dem Wert 0 und unter der Annahme, dass der Fehler der unterbrochenen Rückführung kleiner ist als der mögliche Positionsfehler bei einer Verwendung der reinen Geschwindigkeit.

Die Schätzung der Position, sowie die Normalisierung des Winkels findet sich in der folgenden Abbildung 51.

```

FOR i := 0 TO 2 DO
  // vor Integral
  xDot[i] := V_t[i] - correction[i];

  // Trapezoidal integration
  tzBuffer[i, 0] := S_t[i] + ((integralBuffer[i] + xDot[i]) / 2.0) * d_t;

  // Integral Startwert
  integralBuffer[i] := xDot[i];

  // Normalisieren
  IF i = 2 THEN
    tzBuffer[2, 0] := ATAN2(SIN(tzBuffer[2, 0]), COS(tzBuffer[2, 0]));
  END_IF

  // letzte Position aktualisieren
  S_t[i] := tzBuffer[i, 0];
END_FOR

```

Abbildung 51 Berechnung des Luenberger Beobachters

Begonnen wird mit der Berechnung der Schätzgeschwindigkeit $\hat{\dot{x}}(t)$. Diese ist dann mit dem numerisch stabileren Trapezintegral [5] integriert worden. Dafür vorgesehen ist die folgende Formel (39).

$$A = \int_a^b f(x) dx \approx (b - a) \cdot \frac{f(a) + f(b)}{2} \quad (39)$$

So seien $f(a)$ und $f(b)$ der letzte Stützwert und der Neue Wert des Integrals. Mit der errechneten Zeitdifferenz kann das Integral im Intervall $[a, b]$ gebildet werden. Dieses Teilstück wird anschließend auf das bereits berechnete addiert.

Der Ausrichtungswinkel wird über die zuvor erwähnte Funktion ATAN2 normalisiert. Der Integralwert wird dem Puffer übergeben und kann wieder in der Berechnung des Schätzfehlers verwendet werden.

Damit die Schätzung des Beobachters den korrekten Pufferwert zum Zeitpunkt t bekommt, muss dieser um die Totzeit verzögert werden. Zu erkennen ist die Verschiebung in der folgenden Abbildung 52.

```

// shift buffer
FOR i := GlobalConst.Tz - 1 TO 1 BY -1 DO
  FOR j := 0 TO 2 DO
    tzBuffer[j, i] := tzBuffer[j, i - 1];
  END_FOR;
END_FOR

```

Abbildung 52 Verschiebung des Puffers um einen Wert

Hier werden nacheinander alle Matrixeinträge um einen Wert verschoben. Damit ist immer gewährleistet, dass die Geschwindigkeit zur passenden Kameraposition verwendet wird.

6.4 Experimentelle Ermittlung des Proportionalitätsfaktors

Damit der Beobachter die Positionsdivergenz und damit den Fehler der Schätzung als Korrekturwert für die Anpassung der Schätzgeschwindigkeit berechnen kann, muss der Proportionalitätsfaktor $\underline{L}$ bestimmt werden. Geschehen ist dies durch verschiedene experimentelle Ansätze.

6.4.1 Verschiebung der Beobachterposition um die Totzeit

Der Beobachter übergibt eine Position zum aktuellen Zeitpunkt, die Position der Kamera ist um die Totzeit verschoben. Der Grundgedanke dieser experimentellen Bestimmung des Faktors ist gewesen, die Position des Beobachters um die Kameratotzeit zu verzögern. Wird eine sinusförmige Positionsvorgabe an einer Achse vorgegeben, so müssten bei einwandfreier Bestimmung beide Positionen im selben Zeitpunkt übereinander liegen. Die Abbildung dieser Position kann direkt im live-scope von Beckhoff erfolgen. Der aktuelle Wert des Proportionalitätsfaktors kann über das Forcen oder Schreiben eines neuen Werts verändert werden, die Auswirkungen der Änderung lassen sich unmittelbar beobachten. So sollte es möglich sein iterativ einen geeigneten Wert für den Faktor zu finden.

Die verzögerte Schätzposition ist zusammen mit der Kameraposition dargestellt worden. Ein Ausschnitt der Visualisierung in Beckhoff ist in der folgenden Abbildung 53 dargestellt. Zu erkennen ist die x-Koordinate des Beobachters und der Kameraposition über die Zeit.

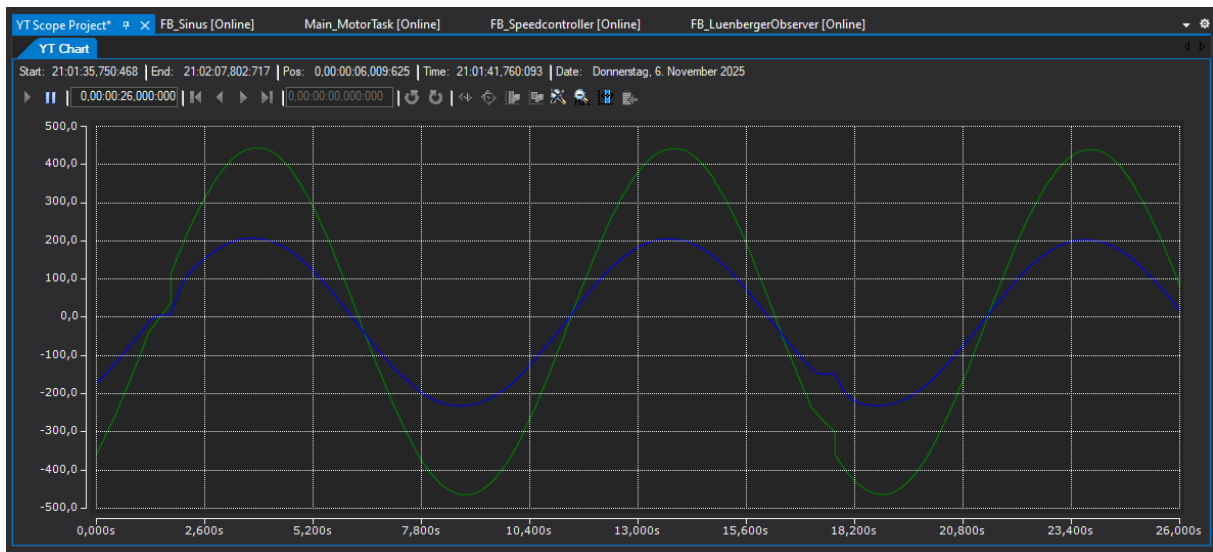


Abbildung 53 Positionsverlauf in Beckhoff für die x-Achse

Die grüne Kurve zeigt den Verlauf der Kameraposition, die Blaue den Verlauf des Beobachters. Eine nennenswerte Veränderung konnte nach dem Verändern der Werte für $\underline{L}$ nicht betrachtet werden.

Das Experiment ist auch mit der Orientierung θ durchgeführt worden. Das Ergebnis und auch die Neuzuweisung eines Werts für $\underline{L}$ sind aufgezeichnet und dokumentiert. Die Beobachtung zeigt sich in der folgenden Abbildung 54.



Abbildung 54 Positionsverlauf in Beckhoff für Theta

Hier ist der Grundgedanke und die mit dem Experiment verbundene Idee deutlicher ersichtlich. Zwischen Sekunde 17 und 20.4 ist der Wert für L_θ von 0.5 auf 15 erhöht worden. Es zeigt sich, dass die beiden Positionen nun übereinander liegen und es kann fälschlicherweise angenommen werden, dass das Experiment funktioniert. Jedoch ist auch zu erkennen, dass an Positionen mit fehlendem Kamerasignal, zwischen Sekunde 10.2 und 13.2, keine Besonderheiten zu erkennen sind und die Beobachterposition bei einer erneuten Störung, Sekunde 27.2 und 30.6, zu schwingen beginnt.

Nach genauerer Überlegung und erneuter Betrachtung der Abbildungen ist der Fehler in diesem Experiment aufgefallen. Der Luenberger Beobachter führt den Fehler der Position zurück und beaufschlagt ihn mit einem Korrekturfaktor. Dieser Faktor ist, vorsichtig gesprochen, die Übersetzung von Position in Geschwindigkeit. Diese Korrektur beeinflusst den nachfolgenden Positionswert. Diese Einstellmöglichkeit kann jedoch nur funktionieren, wenn der Fehler konstant bleibt. Das ist er jedoch nicht. Dieser Denkfehler wäre bei einer nicht sinusartigen Positions Vorgabe aufgefallen, da der Positionswert dem Kamerawert langsam nachgeeilt wäre, bis der Fehler so groß geworden wäre, dass der Beobachter die korrekte Position bestimmen kann. Die abweichende Amplitude wird über die langsame und nacheilende Korrektur erklärt, die Gewichtung wirkt hier wie eine Dämpfung. Der erkennbare Sprung in der Abbildung 54 zeigt damit also die Veränderung der Sensibilität des Beobachters, die entstandenen Schwingungen zeigen den jetzt instabilen Bereich. Hier reagiert der Luenberger Beobachter auf jeden noch so kleinen Wert in der Positions Differenz und beginnt zu schwingen.

Die Annahme es könnte sich um ein Problem in der Periodizität der Sinusschwingung handeln kann widerlegt werden. Diese Betrachtung kann nur unter dem Glauben bestehen, es handle sich um eine konstante Positions Differenz.

Die Ergebnisse des Experimentes sind die Werte $\underline{L} = [3.5 \quad 3.5 \quad 3.0]^T$.

6.4.2 Experimentelle Bestimmung über den Weg

Nach der misslungenen Bestimmung unter Verwendung des vorangegangenen Experiments, ist das Gespräch mit dem Betreuer gesucht worden. Es ergaben sich neue Impulse und Bestimmungsansätze für die Proportionalitätsfaktoren.

Der neue Ansatz ist zu sehen in der folgenden Abbildung 55, dort ist das Experiment zur Bestimmung der Werte für $\underline{L}$ in der y-Achse dargestellt.

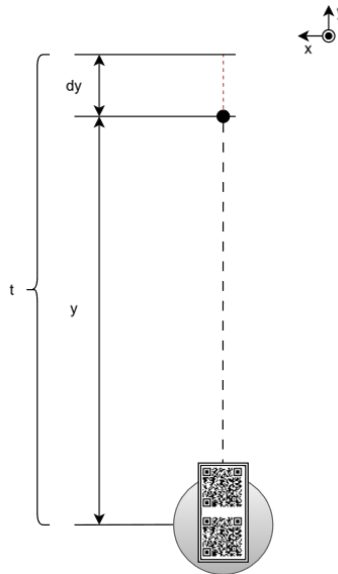


Abbildung 55 Das neue Experiment

Die Geschwindigkeit des HAWinos kann nicht genau bestimmt werden. Sie ist fehlerbehaftet, etwa durch die Form der Räder, die Ungenauigkeiten in den Maßangaben oder Rundungsfehler. Wird diese für die Positionsbestimmung integriert, so wird auch der Fehler in der Position weitergegeben. Dieser Fehler kann experimentell bestimmt und in einen Korrekturfaktor umgewandelt werden. Führt der HAWino für eine definierte Zeit in eine Achsrichtung und wird dann gestoppt, kann der Positionsfehler berechnet werden. Dafür wird die folgende Formel(40) verwendet.

$$L_x = \frac{\Delta s}{t} \quad (40)$$

Hierbei sei L_x der Proportionalitätsfaktor der untersuchten Achse, Δs der sich ergebende Fehler und t die Fahrzeit.

Es sind jeweils fünf Fahrten durchgeführt worden. Über die Abweichungen konnten die Korrekturfaktoren $\underline{L}$ der Fahrt bestimmt werden. Für die Bestimmung des schlussendlichen Faktors ist der Mittelwert der Fahrten verwendet. Die Messwerte finden sich in den Tabellen im Anhang A (Tabelle 13, Tabelle 14, Tabelle 15). Ergeben haben sich für $\underline{L}$ die Werte $[3.14 \quad 3.35 \quad 0.17]^T$;

Die Werte für die x- und y-Achse scheinen korrekt zu bestimmt, der Wert für θ ist nicht korrekt. Hier ließ sich der Fehler in der Bestimmung nicht finden.

6.4.3 Schätzung und Anpassung über Szenario im Controller

Da L_θ durch die Koordinatentransformation einen direkten Einfluss auf die Bewegungen in den anderen Achsen hat, muss dieser möglichst genau bestimmt werden. Das vorangegangene Experiment hat jedoch kein brauchbares Ergebnis für die Verwendung zugelassen. Als letztes Mittel ist auf die Verwendung des Testbetriebs im *RobotController* zurückgegriffen worden. Das dafür erstellte Szenario ist dargestellt in der folgenden Abbildung 56. Hier wird der Aufbau im VPJ aus der Vogelperspektive gezeigt, erkennbar ist der Fahrweg

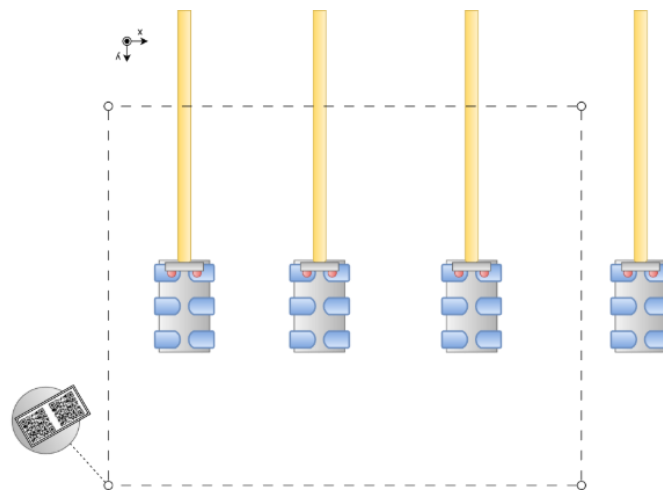


Abbildung 56 Einstellung über ein Szenario

In diesem Szenario wird der HAWino in einer endlosen Schleife über alle Kamerabereiche bewegt. Erst wird vor den Stationen gefahren, dann durch die Bereiche mit unterbrochenem Positionssignal unter den Armen der Stationen hindurch. Berücksichtigt wird hier zudem die Beläuchung.

Für den Luenberger Beobachter sind hier die Anfangswerte des Proportionalitätsfaktors wie folgt gewählt $\underline{L} = [3.14 \quad 3.35 \quad 0.1]^T$.

Während der Fahrt können nun die Werte für $\underline{L}$ angepasst werden. Das Ergebnis wird optisch durch Betrachten des sich einstellenden Verhaltens bewertet. Zur Erreichung einer systematischen Arbeitsweise werden die Werte für θ in Schritten von 0.5 erhöht, bis sich das eingestellte Ergebnis verschlechtert. Das entstandene Intervall wird danach mit der halben Korrekturgröße wiederholt untersucht. Dies geschieht so lange, bis sich das Fahrverhalten nicht länger nennenswert verändert.

Nach dem Einstellen von θ ist diese Vorgehensweise für die x- und y-Achse wiederholt worden. Das Endergebnis für die Faktoren im Luenberger Beobachter liegt bei $\underline{L} = [3.0 \quad 3.1 \quad 2.685]^T$.

6.5 Optimierungsversuche

In der verbleibenden Projektzeit und während der Faktorbestimmung ist an diversen Optimierungsmöglichkeiten gearbeitet worden. Diese werden hier angemerkt und knapp erklärt.

Die ersten trivialen Ansätze zur Optimierung sind Filter. Der Tiefpass-Filter ist hier in einer ersten schlichten Form programmiert. Ziel ist es die unterbrochene Kameraposition über einen solchen auszugleichen. Bei der Verwendung hat sich gezeigt, dass der Filter die invaliden Kamerawerte abfängt, jedoch keine spürbare Verbesserung mit sich bringt. Je länger das Kamerasignal entfallen ist, desto schneller sind auch die Positionen in ihrem Betrag minimiert worden. Eine Verkleinerung des Glättungsfaktors α hat hierbei eine negative Auswirkung auf die validen Kamerapositionen. Davon ist zunehmend auch der Regelkreis betroffen. Selbes gilt für die Verwendung in der Rückführung. Der Ansatz ist im Beobachter verworfen worden, beweist sich jedoch als wirkungsvoll im Regelkreis selbst. Aus diesem Grund ist der programmierte Funktionsblock *FB_TiefpassFilter* in den Regelkreis implementiert.

Mit den Erfahrungen aus dem Tiefpass-Filter ist versucht worden die Kamerapositionen mit anderen Mechanismen zu filtern. Hierbei fanden die Ansätze aus der Encoder-programmierung, zu finden im Anhang C, Verwendung. Es ist der Mittelwert aus den letzten Positionen gebildet worden. Dafür verwendet wird die folgende Formel (41).

$$\bar{x} = \frac{1}{n} \cdot \sum_{i=0}^n x_i \quad (41)$$

Hierbei seien n die Anzahl der verwendeten Positionen, x repräsentiert den Positionswert.

Es ist direkt aufgefallen, dass der Einfluss dieses Ansatzes noch schlechter ist als der Tiefpass-Filter. Das Ausbleiben der Positionen hielt länger an, wirkte aber trotzdem gedämpfter. Der Regelkreis wirkte zunehmend instabil und bei raschen Beschleunigungsvorgängen versagte die Positionsbestimmung gänzlich. Das Verändern der Filtergröße hatte nur das verringern der Verschlechterung zum Ergebnis.

Aus Neugier ist die Funktion *FB_MovingAvgStd* entstanden. Diese berechnet neben dem Mittelwert für ein Inputarray aus den Positionen auch deren Standardabweichungen über die folgende Formel (42).

$$s = \sqrt{\frac{1}{n-1} \cdot \sum_{i=0}^n (x_i - \bar{x})^2} \quad (42)$$

Während einer Fahrt ist damit die Position aus der Kamera beobachtet worden und so entstand der folgende Ansatz.

Die Fahrt besitzt eine Richtung, diese lässt sich aus den vorherigen Werten im *tzBuffer*, welcher die vergangenen Schätzpositionen des Beobachters beinhaltet, errechnen. Wird der letzte Wert in diesem mit dem ersten subtrahiert, so ergibt sich ein vorzeichenbehaftetes Ergebnis mit dem Namen *dir*.

Mit der statistischen Information über den Mittelwert und der Abweichung der Werte voneinander, sowie der Richtung kann bei einem Ausfall der Position der Positionswert geschätzt werden. Dies geschieht unter Verwendung der folgenden Formel (43).

$$P_n := P_{n-1} + dir * s \quad (43)$$

Hierbei seien P_n die neue Kameraposition, dir die Richtung der Bewegung und s die Standardabweichung des Puffers der Kameraposition. Dieser Puffer wird neben dem Puffer der Schätzwerte verwendet.

Die Probefahrten mit dieser Systematik waren von guter Qualität. Auch in Bezug auf θ , welches nach der Berechnung zusätzlich auf den Intervall $[-\pi, \pi]$ normalisiert ist.

Gegen diesen Ansatz spricht das Abwandern der Positionen in den kritischen Bereichen. Blieb eine Position für längere Zeit ungültig kann der Positionsfehler rasant anwachsen. Im Stillstand ist dies besonders bemerkbar, so kommt es dazu, dass der HAWino in kritischen Positionsbereichen oder bei Beginn der Fahrt zu unvorhersehbaren Verhalten neigt.

Da die Ergebnisse während der Fahrt in den von den Armen der Stationen verdeckten Bereichen gut waren, ist ein ähnlicher Ansatz verfolgt worden. Es begann die Prüfung, ob die Rückführung des Positionsfehlers trotzdem möglich ist, jedoch über den bereits berechneten Fehler der letzten Position. Hierfür wird der Positionsfehler vor der Multiplikation mit dem Faktor $\underline{L}$ in einem Puffer gespeichert. Ist die Kameraposition ungültig, so wird der letzte Fehler zurückgegeben.

Auch dieser Ansatz liefert gute Ergebnisse, jedoch besteht ein gravierender Nachteil. Wird die Abbildung 56 erneut betrachtet, so fällt auf, dass der HAWino unter einem Arm hindurch fährt, um in eine Stationsposition anzufahren. Parallel dazu wird in einer Achse beschleunigt und in der anderen entschleunigt, um das Überschleifen zu ermöglichen. Es bleibt die gültige Position aus und der Positionsfehler im Luenberger Beobachter verändert sich nur in einer Achse. Der HAWino antwortet mit schlagartigen Bewegungen und ungewolltem Fahrverhalten. Als Folge dessen wird auch dieser Ansatz gänzlich verworfen.

Die bayes'schen Spezialfälle für Aufgaben dieser Art kamen im Rahmen eines Fachgespräches mit dem Betreuer zur Ansprache, von diesen ist abgeraten worden. Nach kurzer Recherche stellt sich heraus, dass es sich hierbei um Kalman-Filter handeln muss. Es begann der Versuch in der verbleibenden Zeit des Verbundprojektes ein solchen zu programmieren. Dabei scheiterte es an der Zeit und an dem fehlenden Systemmodell.

Der Luenberger Beobachter blieb unverändert und brauchbare Systematiken zur Optimierung finden an anderer Stelle Verwendung. Die Anpassungen des Regelkreises finden sich als Zusatz im Anhang D.

6.6 Fehlerbetrachtung und abschließende Gedanken

Die Versuche haben gezeigt, dass die Werte für die Proportionalitätsfaktoren im Rahmen einer Ungenauigkeit über die Versuchsvariation nahe beieinander liegen. Auch kann bei den Fahrten keine extreme Verhaltensauffälligkeit bemerkt werden, welche nicht durch den Regelkreis oder die Filter abgefangen wird.

In der Abschlusspräsentation war angemerkt, dass der Luenberger Beobachter noch immer nicht in der Lage ist die Position in θ korrekt zu bestimmen. Dafür verwendet worden ist die folgende Abbildung 57. Dargestellt ist hier ein Stück der abgefahrenen Trajektorie in θ .

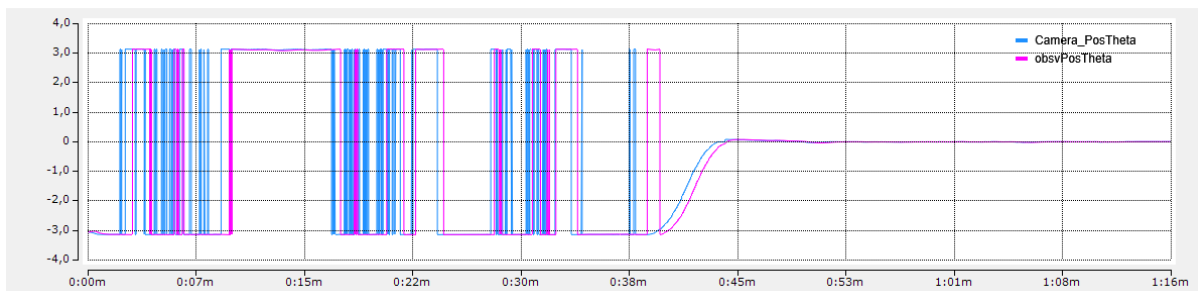


Abbildung 57 Verschiebung in θ

Hier zeigt sich im hinteren Teil der Abbildung, dass eine Verzögerung im Drehwinkel existiert. Jedoch besteht dieses Problem nur bei der Rotation. Vergleichsweise findet sich eine Fahrt der x-Achse in der folgenden Abbildung 58.

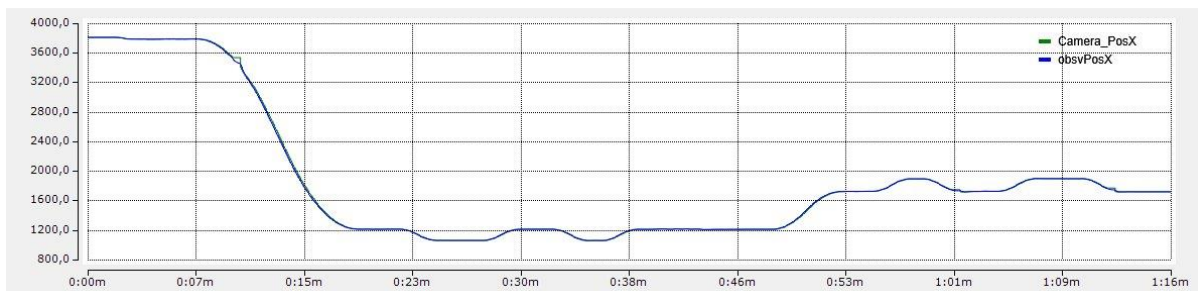


Abbildung 58 Verschiebung in x

Hier liegen die Koordinaten von Kamera und Beobachter übereinander, die Totzeit kann durch die Größe der Darstellung nicht erkannt werden.

In der vorherigen Abbildung ist diese aber klar zu erkennen, trotz der 8 s Skalierung. Wird jetzt die Abbildung vermessen und über den Dreisatz die Zeitverzögerung der Winkeländerung zwischen Beobachter und Kamera berechnet, so ergibt sich mit folgender Rechnung (44) ein Ergebnis.

$$t = \frac{a \cdot b}{c} \cdot 10^3 = \frac{8s \cdot 4mm}{41mm} \cdot 10^3 \approx 640ms \quad (44)$$

Hier seien a die Länge eines Zeitabschnittes, b die gemessene Breite der Verzögerung auf der Abbildung und c die gemessene Skalenbreite auf der Abbildung.

Eine Verzögerung von 640ms ist das sechsfache der Totzeit. Das bedeutet, dass der Beobachter mit einem L_θ von 0 besser funktioniert als mit dem verwendeten L_θ von 2.685. Auch ist dies in den Experimenten klar widerlegt worden. Bei einem L_θ von 0.1 ist eine qualitative und zufriedenstellende Fahrt nicht möglich. Dieser Bereich ist experimentell mehrfach ausgeschlossen.

Es ist nach Fehlern in der Programmierung gesucht worden, jedoch sind die Berechnungen durch die Verwendung der FOR-Schleife identisch. Läge der Fehler hier, dann würden die Positionen nicht stimmen.

Eine Überlegung aus den Versuchen findet hier Wiederverwendung. Wenn der Wert für L_θ nicht die korrekte Größe besitzt, dann muss bei einer Konstanten fahrt die Korrektur der Ausrichtung durch die anwachsende Differenz der Werte im Laufe der Zeit ausgeglichen werden.

Zur Überprüfung der Hypothese ist der HAWino mit konstanter Geschwindigkeit und den finalen Proportionalitätsfaktoren in der Drehachse θ auf der Stelle gedreht worden. Das Ergebnis findet sich in der folgenden Abbildung 59. Hier ist die Kameraposition in grün und die Position aus dem Beobachter in blau dargestellt.

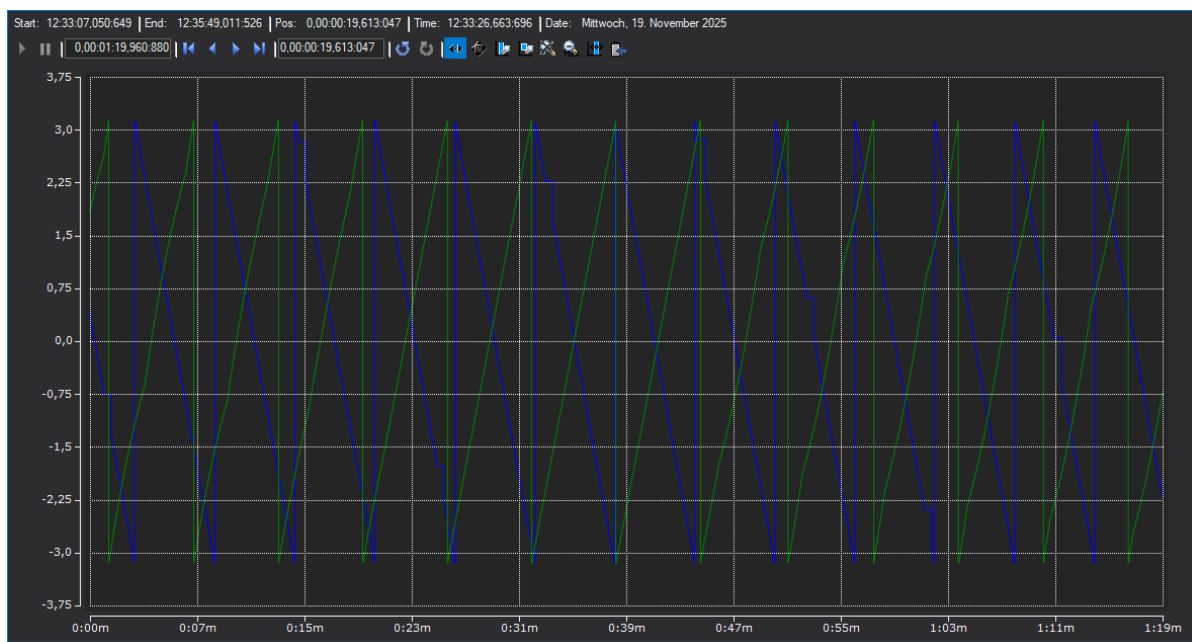


Abbildung 59 Rotation um die eigene Achse

Die Verläufe der Graphen sind irrelevant, interessant ist, dass der Abstand der Übergänge von $\pm\pi$ immer größer wird. Dieses Überholen sich sogar mittig der Abbildung, der Beobachter wird immer langsamer. Selbst das Verändern der Werte für L_θ zum Zeitpunkt der Aufnahme verhilft nicht zu einem erklärbaren Ergebnis.

Der Abstand der Spitzen voneinander beträgt konstant 2 mm. Damit kann argumentiert werden, dass der Fehler systematisch in das System eingebracht wird.

Abschließend kann die Ursache des Fehlers doch noch eingegrenzt werden.

Nach sehr langer Überlegung und einer ehrgeizigen Fehlersuche zeigt sich, dass die Problematik einen trivialen Ursprung hat. Die Korrekturrichtung in θ ist mit einem Vorzeichenfehler behaftet. Eine positive Winkeldifferenz hat eine negative Korrektur in der Geschwindigkeit zur Folge, der Beobachter bremst. Er bremst systematisch und wirkt gegen die Kamera. Mit großem Bedauern fällt dieser Fehler erst in der Auswertung auf. Wegen der fehlenden Implementierung des Winkels θ in der Simulation konnte dieser Fehler zusätzlich nicht früher entdeckt werden.

7 Fazit

Im Rahmen der Abschlusspräsentation konnte nachgewiesen werden, dass der HAWino in der Lage ist, die vorgesehenen Fahrmanöver zuverlässig zu bewältigen. Dabei hat sich gezeigt, dass mehrere HAWinos parallel betrieben und Aufträge systematisch abgearbeitet werden können. Sowohl auf Störfälle im Prozessablauf als auch auf verzögerte Kamerapositionen konnte angemessen reagiert werden.

Die frühzeitige Ausarbeitung einer Simulation sowie die vorgelagerte Validierung der entwickelten Konzepte haben maßgeblich dazu beigetragen, potenzielle Problemstellungen früh zu identifizieren und das zugrunde liegende Modell gezielt zu vereinfachen. Die hierbei gewonnenen Erkenntnisse und Erfahrungen haben sich insgesamt positiv auf den Projektverlauf ausgewirkt.

Der Aufbau des Testbetriebs hat konsequent auf dieser Validierungslogik basiert und während der Durchführung eine wertvolle Unterstützung dargestellt. Auf diese Weise konnten die einzelnen Teilsysteme des HAWinos innerhalb dieses Gewerks gezielt getestet und iterativ verbessert werden. Insbesondere die Regelung, der Regelkreis, die Trajektoriengenerierung sowie der Beobachter konnten durch separate, individuelle Versuche optimiert werden.

Die Komplexität der Fahrten konnte durch die Trennung von rotatorischer und translatorischer Fahrt reduziert werden. Die Zusammenführung einzelner Wegpunkte zu Trajektorien sowie die Übergabe geeigneter Sollwerte an den Regler erwiesen sich als problemlos umsetzbar. Hervorzuheben sind hierbei insbesondere die Einsparung von Rechenleistung sowie der enge Austausch mit dem RobotController. Das zentrale Element der Trajektoriengenerierung stellen die verwendeten Überschleifkurven dar, welche in der praktischen Fahrenwendung äußerst zufriedenstellende Ergebnisse geliefert haben.

Der implementierte Regelkreis zeigte sich als äußerst robust und in der Lage, Störungen unterschiedlicher Art zuverlässig zu kompensieren. Ein wesentliches Merkmal stellt dabei das eingesetzte Gain-Scheduling dar, durch welches der Regler situationsabhängig angepasst werden kann. Die Ergebnisse der Abschlusspräsentation verdeutlichten, dass selbst Störungen in Verbindung mit Totzeiten erfolgreich ausgeglichen werden konnten.

Der eingesetzte Luenberger-Beobachter hat sich als hilfreich für die Schätzung der Position ergeben. Insbesondere in der x- und y-Richtung konnten zufriedenstellende Schätzergebnisse erzielt werden. Als Einschränkung ist jedoch ein erst spät erkannter Fehler in der Rückführung der Orientierung θ zu nennen. Auch wenn die Ausrichtung nicht korrekt geschätzt werden konnte, stellt dieser Umstand dennoch einen nachhaltigen Lerneffekt dar, der aus dem Verbundprojekt mitgenommen werden kann.

Zusammenfassend lässt sich festhalten, dass die gestellte Aufgabe insgesamt zufriedenstellend gelöst worden ist. Die große gestalterische Freiheit sowie die Arbeit in einer Gruppe dieser Größe stellten eine Herausforderung dar, sind jedoch zugleich als bereichernd und motivierend empfunden worden.

Literaturverzeichnis

- [1] „R_TRIG“, *Beckhoff Information System - German*, 3. Januar 2026. Verfügbar unter: https://infosys.beckhoff.com/index.php?content=../content/1031/tcplclib_tc2_standard/74391563.html&id=. [Zugegriffen: 3. Januar 2026]
- [2] Z. Şanal, „Koordinatentransformation“, in *Mathematik für Ingenieure*, Wiesbaden: Springer Fachmedien Wiesbaden, 2020, S. 403–418. doi: 10.1007/978-3-658-31733-1_7. Verfügbar unter: http://link.springer.com/10.1007/978-3-658-31733-1_7. [Zugegriffen: 15. Juni 2025]
- [3] Prof. Dr. M. Seitz, „Vorlesung - Bewegungssteuerungen zur Automatisierung fertigungstechnischer Prozesse“, Mannheim, 10. November 2021.
- [4] Prof. Dr. F. Wenck, „Mehrgrößenregelung“, *Moodle*, 3. Januar 2026. Verfügbar unter: <https://moodle.haw-hamburg.de/login/index.php>. [Zugegriffen: 3. Januar 2026]
- [5] „Praktische Informatik 1“, 3. Januar 2026. Verfügbar unter: <https://www.peter-junglas.de/fh/vorlesungen/praktinfWI1/html/kap6-5.html>. [Zugegriffen: 3. Januar 2026]

Anhang

A Tabellen der Fehlerauswertung

Tabelle 13 Messwerte für die X-Achse

Nr	Strecke [mm]	Integral [mm]	Zeit [s]	Differenz [mm]	L
1	3000,00	3064,13	20,44	64,13	3,14
2	3000,00	3078,25	20,56	78,25	3,81
3	3000,00	3041,95	20,31	41,95	2,07
4	3000,00	3069,19	20,47	69,19	3,38
5	3000,00	3037,52	20,26	37,52	1,85
Mittelwert:					3,14

Tabelle 14 Messwerte für die Y-Achse

Nr	Strecke [mm]	Integral [mm]	Zeit [s]	Differenz [mm]	L
1	3000,00	3073,80	20,52	73,80	3,60
2	3000,00	3001,48	20,05	1,48	0,07
3	3000,00	3003,98	20,04	3,98	0,20
4	3000,00	3071,15	20,50	71,15	3,47
5	3000,00	3068,15	20,32	68,15	3,35
Mittelwert:					3,35

Tabelle 15 Messwerte für Ausrichtung Theta

Nr	Strecke [rad]	Integral [rad]	Drehungen	Restwert [rad]	Zeit [s]	v [rad/s]	Differenz [rad]	L
1	10,00	11,23	1	4,95	10	1	1,23	0,12
2	10,00	12,14	1	5,86	10	1	2,14	0,21
3	10,00	11,26	1	4,97	10	1	1,26	0,13
4	10,00	12,30	1	6,02	10	1	2,30	0,23
5	10,00	11,66	1	5,38	10	1	1,66	0,17
Mittelwert:								0,17

B Ladestation

Die Ladestation der Roboter ist nicht Teil dieses Projekts, hat jedoch eine große Menge an Arbeitszeit in Anspruch genommen. Der Ausfall der Ladestation hat verhindert, dass die Roboter geladen werden konnten und somit auch verhindert, dass die Roboter problemlos getestet werden konnten. Der Ladevorgang war ein Erschwernis. Die folgenden Unterkapitel nehmen Bezug auf die aufgetretenen Probleme, die Beobachtungen und erste eigene Lösungsansätze.

B.-1 Problembeschreibung

Das Hauptproblem der Ladestation ist die Verbindung mit dem Server. Wird die Ladestation eingeschaltet, so zeigt sie eine Fehlermeldung. Diese Fehlermeldung ist in der folgenden Abbildung 60 dargestellt.



Abbildung 60 Fehlermeldung der Ladestation

Es ist zu erkennen, dass die Station keine Verbindung zum Server unter 192.168.0.255 aufbauen kann. Nach dem Auftreten dieses Fehlers ist es nicht länger möglich, die HAWinos zu laden, da die Schaltfläche der Ladestation nicht länger betätigt werden kann. Auch unter Verwendung der IP-Adresse in einem Webbrowser ist es nicht möglich, das HMI (Human Machine Interface) zu öffnen. Dort ist auch die dargestellte Fehlermeldung zu sehen. Das Problem tritt nur nach dem Einschalten auf und hält unbestimmt lange an.

Im weiteren Verlauf des Projekts ist aufgefallen, dass es gelegentlich zu Situationen kommt, in denen das HMI der Ladestation keinerlei Veränderungen anzeigt. Die Bedienelemente funktionieren, jedoch verändert sich die Anzeige, wie beispielsweise der Ladestatus der einzelnen HAWinos, nicht länger.

Teile des Bildschirms wirken wie eingefroren. Wird die Ladestation aus diesem Zustand neu gestartet, so ergibt sich das vorher beschriebene Hauptproblem. Es kann wiederholt keine Verbindung aufgebaut werden und wieder ist die Dauer des Fehlers nicht abschätzbar.

B.-2 Analyse des Systems

Das System ist mehrfach über verschiedene Ansätze untersucht worden in der Hoffnung den Fehler eigenständig lösen zu können. Dabei ist zu Beginn auf zeitsparende und triviale Möglichkeiten zurückgegriffen worden. Die Ladestation konnte unter 192.168.0.09 im Command-prompt von Windows erreicht werden und hat gegen alle Erwartungen reagiert und Daten empfangen.

Im nächsten Schritt ist untersucht, ob es einen möglichen Konflikt mit der Lizenzierung gibt. Aus der eigenen Berufserfahrung ist bekannt, dass eine HMI-Anwendung nicht nur alleinig lizenzgebunden, sondern auch Uhrzeitsynchron angebunden werden muss. Geprüft wird jetzt die Uhrzeiten des Panel-PCs, welche dann mit der Uhrzeit auf den Laborrechnern verglichen wird. Die Ergebnisse sind in der folgenden Abbildung 61 dargestellt und zeigen eine Abweichung von einer Stunde zu den Laborrechnern.

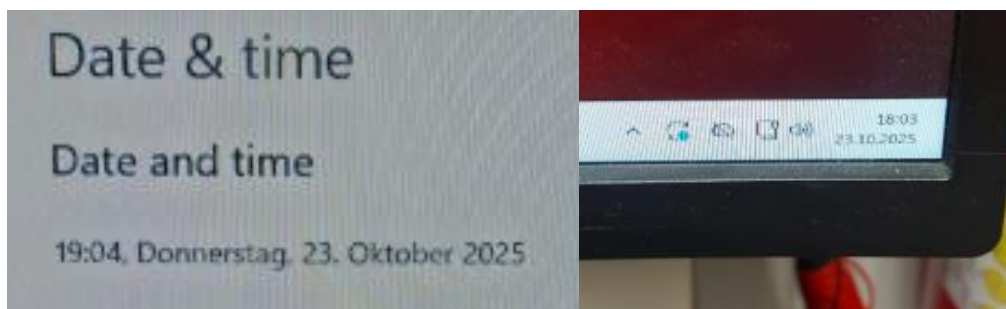


Abbildung 61 Vergleich der Uhrzeiten auf dem HMI und den Laborrechnern

Auch das Synchronisieren der Uhrzeiten hat nach einem Neustart keinerlei Veränderung im Fehlerbild der Ladestation bewirken können. Auch ist bemerkt worden, dass die Uhrzeiten nach dem Neustart direkt und automatisch zurückgesetzt ihren Ursprungswert erhalten. Damit ist eine Veränderung dieser im Vorfeld unwirksam und ohne tieferegreifende Systemrechte unmöglich.

Vor dem Ende des Projekts viel der Fokus auf die Software WireShark. Mit dieser ist es möglich die Kommunikation zwischen den Systemen zu beobachten. Im Rahmen der Überwachung eines anderen Problems ist zufällig die Entstehung des Fehlers der Ladestation aufgezeichnet worden. Wieder ist diese eingefroren und ließ sich nicht länger bedienen. Ein Blick in die Software hat das in der folgenden Abbildung 62 dargestellte Bild ergeben. Dort sind die Kommunikationsversuche der Ladestation sichtbar.

3652	25518.180497	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59621 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.180534	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59619 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.110846	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59619 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.128485	192.168.0.9	192.168.0.251	TCP	54	58953	+ 48898 [ACK] Seq=25841 Ack=31281 Win=65167 Len=0
3652	25518.135489	192.168.0.9	192.168.0.10	TCP	54	51584	+ 8016 [ACK] Seq=1297087 Ack=205531001 Win=5535 Len=0
3652	25518.157343	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59620 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.360897	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59620 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.586236	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59622 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.586886	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59622 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.586233	192.168.0.9	192.168.0.251	APPS	92	APPS Request	
3652	25518.586236	192.168.0.9	192.168.0.10	TLSv1.2	155	Application Data	
3652	25518.591122	192.168.0.251	192.168.0.9	APPS	100	APPS Request	
3652	25518.601167	192.168.0.10	192.168.0.9	TLSv1.2	155	Application Data	
3652	25518.602740	192.168.0.9	192.168.0.251	TCP	60	59623 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.606175	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59621 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.610160	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59619 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.611505	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59621 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.611505	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59621 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.613056	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59619 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.632201	192.168.0.9	192.168.0.251	TCP	54	58953	+ 48898 [ACK] Seq=25879 Ack=31277 Win=65121 Len=0
3652	25518.642084	192.168.0.9	192.168.0.10	TCP	54	51584	+ 8016 [ACK] Seq=1297087 Ack=205531002 Win=5534 Len=0
3652	25518.806900	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59620 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.862199	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59620 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.862202	192.168.0.251	192.168.0.251	TCP	60	59624 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.866084	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59624 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3652	25518.868476	192.168.0.9	192.168.0.251	TCP	60	[TCP Port numbers reused] 59622 + 1010 [SYN] Seq=0 Min=64240 Len=0 MSS=1460 WS=256 SACK_PERM	
3652	25518.870397	192.168.0.251	192.168.0.9	TCP	60	1010	+ 59622 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Abbildung 62 „Kommunikationsversuche der Ladestation bei einem Fehlerfall“

Der Datenverkehr ist zusätzlich gespeichert und steht als Datei zur Verfügung.

B.3 Erkenntnisse

Anhand der Analyse der Netzwerkdaten, sowie der Widerlegung der sogenannten *Systemzeit-Hypothese* kann eine erste fundierte Vermutung zur Ursache der beobachteten Problematik formuliert werden.

Zunächst bestand die Vermutung, dass es sich um eine sogenannte Race-Condition im Bootvorgang handelt, also einen zeitlichen Konflikt zwischen konkurrierenden Prozessen während des Einschaltens. Diese Annahme konnte jedoch nicht eindeutig bestätigt werden. Der Fehler tritt nicht bei jedem Systemstart auf, lässt sich jedoch auch während des laufenden Betriebs beobachten. Damit ist eine reine Race-Condition im Bootvorgang als alleinige Ursache unwahrscheinlich.

Die Auswertung der WireShark-Daten zeigt mehrere Auffälligkeiten. Die Ladestation sendet eine ungewöhnlich hohe Anzahl an Verbindungsanforderungen. Das Zielsystem reagiert darauf jedoch unmittelbar mit Verbindungsrücksetzungen. Kurzzeitig ist ein TLS-Hello (Transport Layer Security) sichtbar, der Verbindungsaufbau wird jedoch sofort abgebrochen.

Auf Basis dieser Beobachtungen kann folgende Hypothese formuliert werden. Einige Systeme könnten eine hohe Frequenz von Verbindungsversuchen als potenziellen Angriff interpretieren und blockieren den Client anschließend vorsorglich. In diesem Zusammenhang ist an die zu hohe Übertragungsrate der Kamera gedacht worden. Möglicherweise erkennt der Server die Ladestation aufgrund der Anzahl der Verbindungsversuche als Sicherheitsrisiko. Diese Annahme könnte erklären, warum der Verbindungsaufbau nicht zuverlässig stattfindet, allerdings ist das Problem nicht reproduzierbar genug, um eine eindeutige Ursache zu identifizieren.

Darüber hinaus ist zu berücksichtigen, dass ein Großteil der beteiligten Systeme aus Studien- oder Abschlussarbeiten beziehungsweise aus anderen studentischen Ausarbeitungen stammt. In diesem Kontext könnte es vorkommen, dass einige IP-Adressen fehlerhaft konfiguriert sind. Ein solcher Fehler hätte im ungünstigsten Fall Auswirkungen auf das sogenannte Address Resolution Protocol (ARP) innerhalb des Netzwerks. Ein Konflikt im ARP, also ein ARP-Race, könnte dazu führen, dass ein Gerät eine Verbindung akzeptiert, während ein anderes sie unmittelbar zurücksetzt. Das beobachtete Muster, bei dem eine

Verbindungsrücksetzung direkt nach dem Verbindungsversuche erfolgt, passt sehr gut zu diesem Szenario. Allerdings konnte diese Vermutung bisher nicht bestätigt werden. Außerdem ist unklar, wie eine Verbindung überhaupt aufgebaut werden kann, wenn ein solcher Konflikt vorliegt.

Zusammenfassend lässt sich feststellen, dass derzeit keine direkte Lösung für das Problem vorliegt. Die Fehlererscheinung tritt weiterhin auf und auf Wunsch des Betreuers ist dieses Kapitel ein Anhang des Berichtes.

B.-4 Abschließende Anmerkungen

Abschließend soll angemerkt werden, dass während der Fehlersuche auch der Code der Ladestation eingesehen worden ist. Es ist aufgefallen, dass die verwendete Statemachine in Simulink Stateflow über keine konsistente Absicherung verfügt. Beispielsweise ist es technisch möglich beide Stationen parallel in den CAC-Check zu schicken. Das ist durch den Laborbetreuer verboten worden, aber dennoch technisch möglich.

Werden doch zwei HAWinos parallel und zeitgleich angeschlossen, kommt es jedes Mal zu dem Fehlerbild in den folgenden Abbildung 63 und Abbildung 64. Diese lassen sich oftmals nicht ohne Hilfe der Betreuer beheben.

Für entstandene Fragen oder die Aushändigung der gesammelten Daten steht Herr Alexander Prehn weiterhin zur Verfügung.

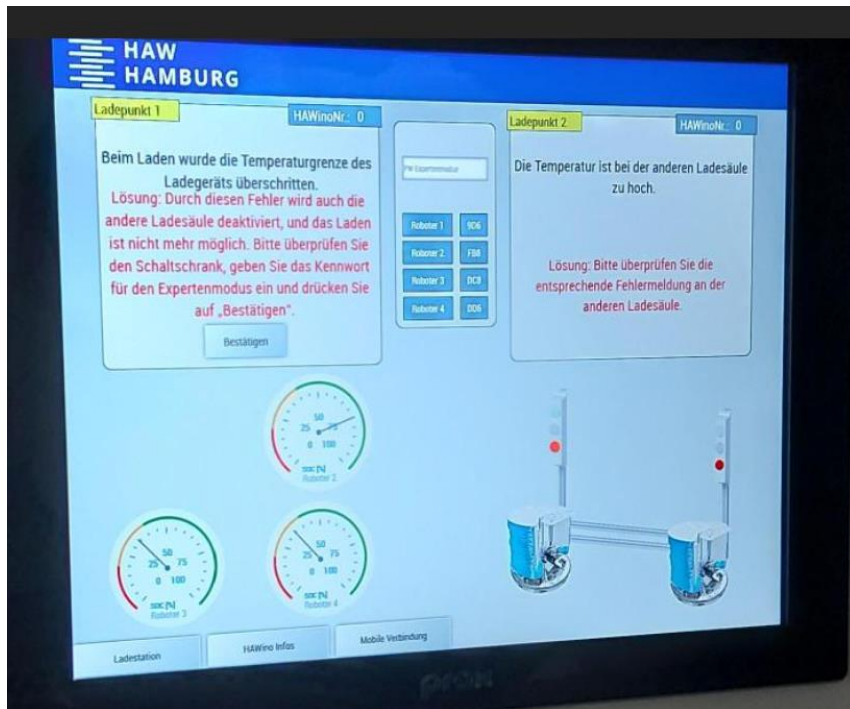


Abbildung 63 Ladestation: Temperaturfehler 1

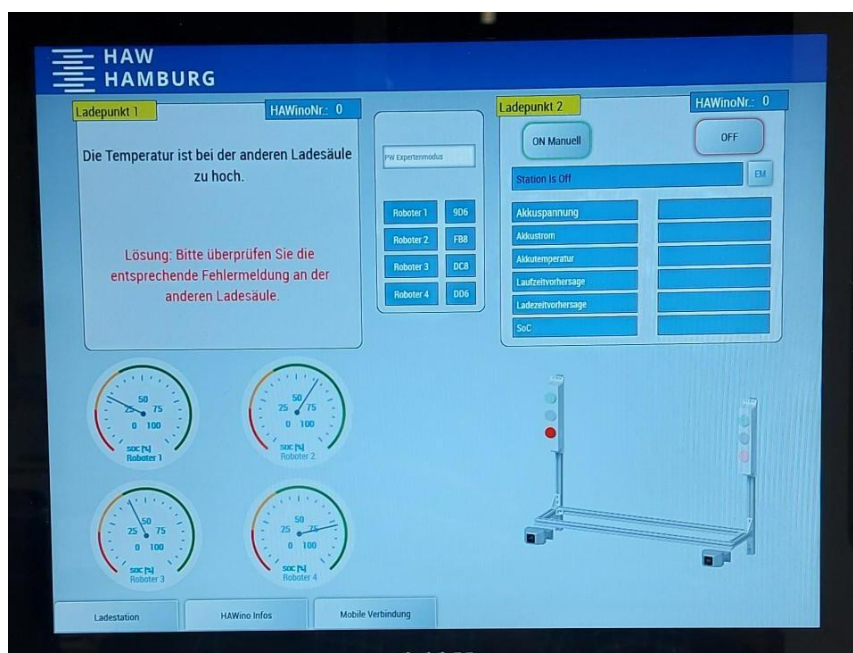


Abbildung 64 Ladestation: Temperaturfehler 2

C Encoder

Auf Grund der parallelen Bearbeitung der Arbeitsaufgabe und der anfänglichen Unschlüssigkeit über die Entstehung der anwachsenden Totzeit ist neben der Kamera auch die Geschwindigkeitsmessung in der Korrektheit angezweifelt worden. Liefert der im HAWino verbaute Encoder zufällige inkorrekte Daten, so ist die Totzeitmessung gestört.

Um die Geschwindigkeitsmessung zu verbessern, ist viel Zeit in die Programmierung einer Encoder-Task investiert worden, welche jedoch keine Verwendung fand. In diesem Kapitel sollen die Bemühungen kompakt dargestellt werden, bezogen wird sich bei der Beschreibung nur auf das Endergebnis.

Die Encoder-Task beinhaltet drei Funktionsbausteine, diese lesen an den zugewiesenen Motoren die Signale aus. Aus diesen Signalen können die Radgeschwindigkeit ausgerechnet werden.

C.-1 Der Funktionsbaustein FB_GetEncoderSpeed

Zunächst wird die Zeit seit dem letzten Tick, dem Erkennen eines Signales, aktualisiert. Dies geschieht über folgende Formel (45).

$$\Delta t = \Delta t + cycleTime \quad (45)$$

Anschließend werden Flankendetektoren für die beiden Encoder-Signale *Encoder_A* und *Encoder_B* aufgesetzt, dabei sowohl die steigende als auch die fallende Flanke. Bei der Detektierung einer Flanke (RisingA OR RisingB OR FallingA OR FallingB) wird die Richtung des Encoders über die Kombination der aktuellen und vorherigen Zustände (*Encoder_A*, *Encoder_B*, *LastA*, *LastB*) ermittelt. Die Logik basiert auf dem quadraturkodierte Signal des Encoders, das vier mögliche Zustände hat.

Je nach Übergang zwischen diesen Zuständen wird der *TickCounter* inkrementiert oder dekrementiert.

Anschließend folgt die Berechnung der Winkelgeschwindigkeit über die Formel (46).

$$\omega = \frac{TickCounter - LastTickCounter}{PPR} \cdot 2\pi \cdot GearRatio \cdot \frac{1}{\Delta t} \quad (46)$$

Hierbei werden die Ticks, welche als Winkelstücke verstanden werden können, durch die Anzahl der möglichen Pulsanzahl pro Umdrehung geteilt. Dieser Quotient wird mit dem Übersetzungsverhältnis multipliziert und durch das Δt dividiert. Abschließend folgt die Umrechnung auf die Einheit *rad/s*.

C.-2 Beobachtungen, Fehlersuche und Ergebnis

Die Geschwindigkeiten variieren stark, bei konstanten Probefahrten ist es nicht möglich konstante Winkelgeschwindigkeiten auszulesen. Aus dem Funktionsblock ist eine Task erzeugt worden, diese ist dann in ihrer Priorität und Geschwindigkeit auf die maximal möglichen Werte eingestellt worden. Es folgte die Flankenerkennung und diverse Filterschaltungen. Auch ist versucht worden die Zeiten für die Berechnung in konstanten Zeitfenstern abzutasten.

Die Lösungsansätze erstrecken sich bis in die numerische Mathematik. Verwendung findet hier erstmals verschiedene Methoden zur numerischen Differentiation. Programmiert sind dabei diverse Anfangswertprüfungen und Schutzmechanismen.

In dem Gespräch mit dem Betreuer hat sich abschließend herausgestellt, dass die Steuerung nicht schnell genug für die Auswertung der Signale ist. Das Zählen der *Ticks* muss deshalb auf Hardware-Ebene geschehen. Diese Hardware ist bereits verbaut und findet Verwendung im HAWino. Die Hypothese, es kann sich bei der Totzeitvergrößerung um ein Geschwindigkeitsproblem handeln, gilt als widerlegt. Die nutzbaren Ansätze dieses Versuches finden sich an anderen Projektteilen wieder.

D Programmierung des Regelkreises über PID

Im Rahmen einer gegenseitigen Korrektur ist der folgende Optimierungsansatz entstanden.

Es ist ein Funktionsblock mit dem Namen *FB_PID_for_Array* programmiert. Dieser ist universal gestaltet, das bedeutet über eine Zeichenkette von drei Buchstaben sind alle Kombinationsmöglichkeiten der P-, I-, und D-Elemente darstellbar. Die Elemente setzen sich zusammen aus den einzelnen Summanden der folgenden Formel (47).

$$y(t) = K_P \cdot e(t) + K_I \cdot \int e(\tau) d\tau + K_D \cdot \frac{de(t)}{dt} \quad (47)$$

Hierbei seien $y(t)$ die Stellgröße und $e(t)$ die Regeldifferent. K_P, K_I und K_D sind die Reglerparameter.

Diese Summanden können einzeln berechnet und Fallbezogen addiert werden. Hiermit kann der Regelkreis und dessen Einstellung erheblich erleichtert werden. Über die zusätzliche Verwendung von FOR-Schleifen ist der Programmieraufwand reduziert. Der Funktionsblock *FB_PID_for_Array* ist schematisch in der folgenden Abbildung 65 dargestellt.

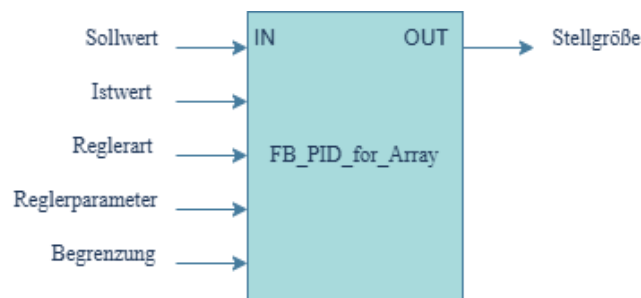


Abbildung 65 Darstellung des *FB_PID_for_Array*

Dem Regler werden die Soll- und Ist-Werte, die gewählte Reglerstruktur sowie die zugehörigen Reglerparameter übergeben. Zusätzlich werden Begrenzungen für den Integralanteil definiert. Als Ausgabe liefert der Regler die entsprechende Stellgröße.

Die eingelesenen Positionswerte sowie die Sollwerte der Trajektorie werden mithilfe von Actions zu Arrays zusammengefasst. Diese Arrays werden gemeinsam mit der Reglerart, den Reglerparametern und den definierten Begrenzungen an den Positionsregler übergeben. Das vom Positionsregler erzeugte Ausgabearray dient anschließend als Eingangsgröße für den Geschwindigkeitsregler. Dieser funktioniert über denselben Funktionsblock. Für diesen sind neue Reglerparameter sowie eine neue Anteils-kombinationen definiert. Die Ausgabe des Geschwindigkeitsreglers wird in einzelne Stellgrößen aufgeteilt und an die jeweiligen Motoren weitergegeben.

Ziel dieses Programmierversuchs ist die Verbesserung der Reglereinstellung sowie des Regelverhaltens im Vergleich zum zuvor eingesetzten Regler. Hierbei sollen neben verschiedenen Parameterwerten auch unterschiedliche Reglertypen untersucht werden. Der Fokus liegt insbesondere auf dem Einsatz von

Differentialanteilen in der Regelung und nicht primär auf der Verwendung von Filtern. Die systematischen Probleme im Luenberger Beobachter sollen so abgefangen werden. Dazu kommt der Gedankengang, die bestehende Programmierung für den Reglekreis auf ein einzelnes Programm zu reduzieren.

Der Beginn der Programmierung fand zu einem späten Zeitpunkt des Projekts statt und hat für die Implementierung einen zu hohen Arbeitsaufwand gehabt. Das Verbinden der Ein- und Ausgänge, alle bisherigen Programmstrukturen und zum Teil auch die Trajektoriengenerierung hätten angepasst werden müssen. Hiervon betroffen ist ein großer Teil des Grundbestands des *Starterkits*. Die Umstrukturierung ist daran gescheitert. Es ist die Entscheidung getroffen worden, das bestehende Konzept zu verbessern und nicht zu erneuern.